

بازیابی اطلاعات

دانشگاه آزاد شیراز • دانشکده مهندسی کامپیوتر

ترم دوم • سال تحصیلی ۱۴۰۴-۱۴۰۵

فصل بیستم: Web Crawling and Indexes

استاد درس

دکتر امین اسکندری

تیم طراحی محتوای آموزشی و ویراستاری (علمی و ادبی)

امیرمحمد اسدجو

علی نیکوان

علی مجاهد

امیرحسین همتی

حمید نامجو

فهرست مطالب

20.1 Overview

20.1.1 Features a crawler must provide

20.1.2 Features a crawler should provide

20.2 Crawling

20.2.1 Crawler architecture

20.2.2 DNS resolution

20.2.3 The URL frontier

20.3 Distributing indexes

20.4 Connectivity servers

20.5 Chapter 20 Summary

20.6 End of Chapter Exercises

20، 20.1 و ویژگی‌های کرالر وب: از تئوری تا عمل 🕷

تصور کنید در تیم مهندسی گوگل کار می‌کنید و مسئول بهبود موتور جستجوی این شرکت هستید. هر روز میلیاردها صفحه وب توسط ربات‌های گوگل (Crawlers) بازدید و ایندکس می‌شوند. این یک فرآیند بسیار پیچیده است، زیرا وب با سرعت فوق‌العاده در حال تغییر است.

چالش‌های اصلی که تیم شما با آن‌ها روبرو است:

- **دامنه‌های کرالر:** وبسایت‌هایی با محدودیت‌های مختلف (مانند فروشگاه‌های آنلاین با میلیون‌ها محصول).
- **تله‌های کرالر:** صفحاتی که به‌طور نامتناهی تولید می‌شوند و ربات‌ها را در حلقه‌های بی‌نهایت قرار می‌دهند.
- **سرورهای پرسرعت:** وبسایت‌هایی که بار ترافیکی بالایی دارند و به‌روزرسانی مداوم نیاز دارند.

🕷 ربات‌های گوگل: از تئوری تا عمل

ربات‌های گوگل (Googlebots) باید ویژگی‌های کلیدی که در کتاب ذکر شد را داشته باشند:

- **استحکام:** مقاومت در برابر **دام‌های کرالر** که به‌طور نامتناهی تولید می‌شوند.
- **ادب:** رعایت فایل robots.txt و محدود کردن نرخ درخواست به هر دامنه.
- **توزیع‌پذیری:** امکان اجرا روی چندین ماشین برای پوشش سریع‌تر وب.
- **مقیاس‌پذیری:** افزایش نرخ کرالینگ با اضافه کردن منابع.
- **کارایی:** استفاده بهینه از منابع سیستم (CPU، ذخیره، پهنای باند).
- **کیفیت:** اولویت‌دهی به صفحات **مفیدتر** (مثلاً با بر اساس اعتبار دامنه).
- **تازگی:** به‌روزرسانی مداوم صفحات بر اساس **نرخ تغییر** آن‌ها.
- **گسترش‌پذیری:** معماری ماژولار برای پشتیبانی از پروتکل‌ها یا فرمت‌های داده جدید.

این ویژگی‌ها به ربات‌های گوگل اجازه می‌دهند تا با چالش‌های بزرگ وب مقابله کنند و در عین حال از بار سرورها و شبکه به صورت بهینه استفاده کنند.

🚫 تله‌های کرالر: چالش‌های پیشرفته

یکی از بزرگترین چالش‌ها برای کرال‌های وب، **تله‌های کرالر** (Spider Traps) است. این صفحات به‌طور عمدی یا ناشیانه تولید می‌شوند تا ربات‌ها را به در حلقه‌های بی‌نهایت بیندازند.

انواع رایج تله‌های کرالر:

- **تله‌های تقویمی:** صفحاتی با لینک‌های بی‌پایان که کاربر را به صفحات بی‌نهایت هدایت می‌کنند.
- **تله‌های زمانی:** صفحاتی با رویدادهای زمانی بی‌پایان که کاربر را در وب نگه می‌دارند.
- **تله‌های نرم‌افزاری:** صفحاتی که نیاز به نصب نرم‌افزار برای نمایش محتوای کامل دارند.
- **تله‌های منطقی:** ساختارهای پیچیده با مسیرهای تکراری که ربات‌ها را در مسیرهای بی‌پایان به دام می‌اندازند.

ربات‌های گوگل با الگوریتم‌های پیشرفته خود را در برابر این تله‌ها مقاوم می‌کنند. برای مثال، اگر یک صفحه بیش از 1000 لینک خروجی داشته باشد، ربات ممکن است آن را به عنوان یک تله تقویمی شناسایی و از کرالینگ کامل آن خودداری کند.

برای کرایلینگ وب در مقیاس بزرگ (مانند کل وب)، گوگل از معماری **توزیع شده** استفاده می‌کند. در این معماری:

- **URL Server**: مسئولیت تخصیص URLها به ربات‌های مختلف برای کرایلینگ موازی.
- **Crawler**: کامپیوترهایی که صفحات وب را بارگیری و تحلیل می‌کنند.
- **Links Database**: پایگاه داده‌ای که لینک‌های بین صفحات را ذخیره می‌کند.
- **Document Store**: ذخیره محتوای صفحات پس از تحلیل.
- **Indexer**: سیستمی‌هایی که ایندکس‌های جستجو را ایجاد و به‌روزرسانی می‌کنند.

این معماری به گوگل اجازه می‌دهد تا میلیاردها صفحه را به صورت موازی کرایلینگ کند و با سرعت بسیار بالا ایندکس‌ها را به‌روزرسانی کند. هر بخش می‌تواند روی ماشین‌های مختلف اجرا شود و به صورت مستقل به‌روزرسانی شود.

🌐 کاربردهای عملی: از موتورهای جستجو تا توصیه‌گر محتوا

کرایلینگ وب تنها مرحله اول از زنجیره بازیابی اطلاعات است. مرحله بعد، **ایندکس‌گذاری و جستجو** است که در آن کاربران می‌توانند به اطلاعات دسترسی پیدا کنند. در این مرحله نیز، رویکردهای یادگیری ماشین نقش کلیدی دارند.

کاربردهای عملی که از این رویکردها استفاده می‌کنند:

- **موتورهای جستجو**: برای رتبه‌بندی نتایج جستجو بر اساس ارتباط با پرس‌وجو کاربر.
- **سیستم‌های توصیه**: برای ارائه پیشنهادهای شخصی‌سازی بر اساس تاریخچه و رفتار کاربر.
- **پلتفرم‌های تجارت الکترونیک**: برای ترکیب داده‌های مختلف (جستجو، خرید، بازدید محصول) برای ارائه پیشنهادهای جامع.
- **شبکه‌های تحلیل داده**: برای درک الگوهای پیچیده در رفتار کاربران و بهبود الگوریتم‌ها.

این کاربردها نشان می‌دهند که چگونه کرایلینگ وب و ایندکس‌گذاری می‌توانند پایه‌ای برای سیستم‌های هوشمندتر باشند که تجربه کاربری را به شکل معنادارتری به کاربران ارائه دهند.

🕷 20.2 کرایلینگ: کاوشگران دنیای وب

🌐 سفر به دنیای وب: چگونه ربات‌های گوگل میلیاردها صفحه را کشف می‌کنند؟

تصور کنید در حال طراحی یک سیستم برای گوگل هستید که باید تمام صفحات وب را کشف و ایندکس کند. این فرآیند که به آن "کرایلینگ" می‌گویند، مانند سفر یک کاوشگر به دنیای ناشناخته وب است.

ربات‌های گوگل (Googlebots) این سفر را به این صورت انجام می‌دهند:

1. **نقطه شروع**: از چند URL معروف مانند `google.com`، `wikipedia.org` و `twitter.com` شروع می‌کنند. این‌ها "مجموعه دانه" (seed set) نامیده می‌شوند.
2. **بازدید از صفحه**: ربات به این آدرس‌ها سر زده و محتوای صفحه را دریافت می‌کند.
3. **تحلیل محتوا**: متن صفحه برای ایندکس‌گذاری استخراج می‌شود و تمام لینک‌های داخل صفحه شناسایی می‌شوند.
4. **ادامه سفر**: لینک‌های جدید به صفی به نام "مرز" (URL Frontier) "URL اضافه می‌شوند تا در آینده بازدید شوند.

این فرآیند به صورت بازگشتی ادامه می‌یابد و ربات‌های گوگل روزانه میلیاردها صفحه جدید را کشف و ایندکس می‌کنند.

در نگاه اول، کرایلینگ ممکن است ساده به نظر برسد، اما در عمل با چالش‌های بزرگی روبرو است. برای درک این چالش‌ها، به مثال زیر توجه کنید:

شرکت "دیجی‌کالا" می‌خواهد تمام محصولات خود را در گوگل ایندکس کند. با میلیون‌ها محصول و هزاران صفحه جدید هر روز، این کار به سادگی امکان‌پذیر نیست. ربات‌های گوگل باید:

- **مقیاس‌پذیر باشند:** بتوانند میلیاردها صفحه را در زمان معقولی پردازش کنند.
- **کارا باشند:** از منابع سیستم بهینه استفاده کنند.
- **مودب باشند:** به سرورها فشار نیاورند.
- **مقاوم باشند:** در برابر خطاها و تله‌های وب مقاومت کنند.
- **قابل گسترش باشند:** بتوانند با تغییرات وب سازگار شوند.

برای مثال، کرالر Mercator که پایه بسیاری از کرال‌های تجاری است، می‌تواند صدها صفحه در ثانیه را پردازش کند. این یعنی برای ایندکس کردن یک میلیارد صفحه، فقط چند هفته زمان نیاز است!

💡 ادب در دنیای دیجیتال: چرا ربات‌ها باید مودب باشند؟

شاید فکر کنید ربات‌ها می‌توانند هر طور که می‌خواهند به سرورها درخواست ارسال کنند، اما در واقعیت، این کار می‌تواند به سرورها آسیب بزند. برای درک این موضوع، به این مثال توجه کنید:

تصور کنید یک فروشگاه کوچک آنلاین دارید که روزانه ۱۰۰ بازدیدکننده دارد. ناگهان یک ربات به صورت همزمان ۱۰۰۰ درخواست به سرور شما ارسال می‌کند. این کار باعث می‌شود سرور شما از کار بیفتد و مشتریان واقعی نتوانند به فروشگاه شما دسترسی پیدا کنند.

برای جلوگیری از این مشکلات، کرال‌های حرفه‌ای باید این قوانین را رعایت کنند:

- فقط یک اتصال همزمان به هر سرور.
- تأخیر چند ثانیه‌ای بین درخواست‌های متوالی به یک سرور.
- رعایت سیاست‌های فایل robots.txt که مشخص می‌کند کدام بخش‌های سایت نباید کرال شوند.

این قوانین به عنوان "ادب کرالر" (Crawler Politeness) شناخته می‌شوند و برای سلامت اکوسیستم وب ضروری هستند.

🔄 کرایلینگ مداوم: چگونه گوگل اطلاعات را به‌روز نگه می‌دارد؟

وب به سرعت در حال تغییر است و صفحات جدید دائماً ایجاد می‌شوند. برای اینکه موتورهای جستجو اطلاعات به‌روزی ارائه دهند، باید به صورت مداوم وب را کرال کنند.

برای مثال، سایت‌های خبری مانند irna.ir یا isna.ir هر چند دقیقه یک بار به‌روزرسانی می‌شوند. ربات‌های گوگل این سایت‌ها را بیشتر از صفحاتی که به ندرت تغییر می‌کنند (مانند یک مقاله علمی قدیمی) بازدید می‌کنند.

در **کرایلینگ مداوم** (Continuous Crawling)، URL‌های بازدیدشده دوباره به مرز URL اضافه می‌شوند تا در آینده دوباره بازدید شوند. این فرآیند به گوگل اجازه می‌دهد تا همیشه جدیدترین اطلاعات را در اختیار کاربران قرار دهد.

In [1]:

```
import time
import random
from collections import deque, defaultdict
from urllib.parse import urlsplit

class WebCrawlerSimulator:
    """
    شبیه‌سازی یک کرالر وب ساده که
    - از یک مجموعه دانه شروع میکند
    - اضافه میکند URL لینک‌ها را استخراج و به مرز
    - برای هر میزبان، فقط یک درخواست همزمان انجام می‌دهد
    - بین درخواست‌ها به یک میزبان، تأخیر رعایت می‌شود
    """

    def __init__(self, seed_urls, politeness_delay=2):
        """
        مقداردهی اولیه کرالر
        - seed_urls: لیست URL اولیه
        - politeness_delay: تأخیر حداقل بین درخواست‌ها به یک میزبان (ثانیه)
        """

        self.seed_urls = seed_urls
        self.politeness_delay = politeness_delay
        self.url_frontier = deque(seed_urls) # های بازدیدنشده URL صف
        self.fetched_urls = set() # های بازدیدشده URL مجموعه
        self.last_fetch_time = defaultdict(float) # آخرین زمان درخواست به هر میزبان
        self.host_queues = defaultdict(deque) # ها برای هر میزبان URL صف

    def extract_host(self, url):
        """URL استخراج نام میزبان از"""
        return urlsplit(url).netloc.lower()

    def simulate_fetch_page(self, url):
        """
        شبیه‌سازی دریافت صفحه و استخراج لینک‌ها
        را دریافت و پارس کند HTML در واقعیت، این تابع باید
        """

        print(f"دریافت صفحه {url}")
        # شبیه‌سازی زمان پردازش
        time.sleep(0.1)

        # تولید لینک‌های تصادفی برای شبیه‌سازی
        host = self.extract_host(url)
        new_links = []
        for i in range(random.randint(0, 3)):
            new_links.append(f"http://{host}/page_{random.randint(1, 100)}")
        return new_links

    def is_polite_to_fetch(self, host):
        """بررسی اینکه آیا می‌توان به این میزبان درخواست فرستاد یا خیر"""
        current_time = time.time()
        last_time = self.last_fetch_time[host]
        return (current_time - last_time) >= self.politeness_delay

    def crawl(self, max_pages=10):
        """
        اجرای فرآیند کرالینگ تا حداکثر تعداد صفحات مشخص‌شده
        """

        print(f"...اولیه URL {len(self.seed_urls)} شروع کرالینگ از")
        pages_fetched = 0

        while self.url_frontier and pages_fetched < max_pages:
            # ها در صف‌های میزبان URL توزیع
            while self.url_frontier:
                url = self.url_frontier.popleft()
                if url in self.fetched_urls:
                    continue
                host = self.extract_host(url)
                self.host_queues[host].append(url)
                self.fetched_urls.add(url) # جلوگیری از بازدید مجدد

            # پردازش هر میزبان که شرایط ادب را دارد
            hosts_ready = [host for host in self.host_queues if self.is_polite_to_fetch(host)]

            if not hosts_ready:
                # اگر هیچ میزبانی آماده نیست، کمی صبر کن
                time.sleep(0.5)
                continue

            # انتخاب یک میزبان آماده
            host = random.choice(hosts_ready)
            if not self.host_queues[host]:
                continue

            url = self.host_queues[host].popleft()
            self.last_fetch_time[host] = time.time()

            # شبیه‌سازی دریافت صفحه و استخراج لینک‌ها
            new_links = self.simulate_fetch_page(url)
            pages_fetched += 1

            # اضافه کردن لینک‌های جدید به مرز
            for link in new_links:
                if link not in self.fetched_urls:
                    self.url_frontier.append(link)

            print(f"صفحات بازدیدشده: {pages_fetched}/{max_pages} | لینک‌های جدید: {len(new_links)}")
            print("-" * 50)

        print(f"کرالینگ به پایان رسید. تعداد کل صفحات بازدیدشده: {pages_fetched}")
```

مثال اجرا

```
if __name__ == "__main__":
    seed_urls = [
        "http://example.com/home",
        "http://sample.org/index",
        "http://test.net/main"
    ]

    crawler = WebCrawlerSimulator(seed_urls, politeness_delay=1)
    crawler.crawl(max_pages=8)

    print("\nنکته:")
    print("این شبیه‌سازی، مکانیزم اصلی کرایلینگ را نشان می‌دهد")
    print("URL (URL Frontier) مدیریت مرز -")
    print("رعایت ادب میزبان (یک اتصال، تأخیر بین درخواست‌ها) -")
    print("های یکسان URL جلوگیری از بازدید مجدد از -")
    print("و پایگاه داده است HTTP، HTML در پیاده‌سازی واقعی، نیاز به پردازش")
```

...اولیه URL شروع کرایلینگ از 3
دریافت صفحه: http://test.net/main
لینک‌های جدید: 2 | صفحات بازدیدشده: 1/8

دریافت صفحه: http://example.com/home
لینک‌های جدید: 1 | صفحات بازدیدشده: 2/8

دریافت صفحه: http://sample.org/index
لینک‌های جدید: 0 | صفحات بازدیدشده: 3/8

کرایلینگ به پایان رسید. تعداد کل صفحات بازدیدشده: 3

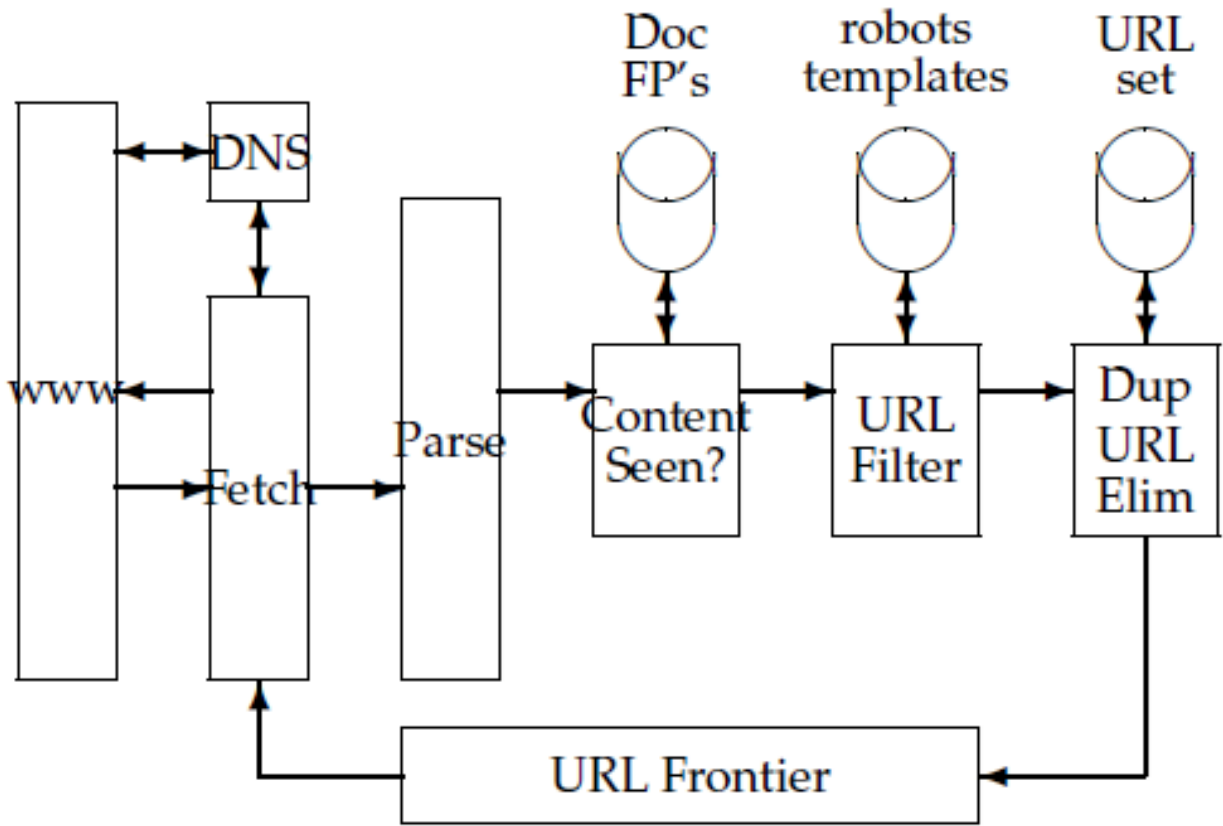
نکته:
این شبیه‌سازی، مکانیزم اصلی کرایلینگ را نشان می‌دهد:
- URL (URL Frontier) مدیریت مرز
- رعایت ادب میزبان (یک اتصال، تأخیر بین درخواست‌ها)
- های یکسان URL جلوگیری از بازدید مجدد از
- و پایگاه داده است HTTP، HTML در پیاده‌سازی واقعی، نیاز به پردازش

20.2.1 معماری کرالر وب: آناتومی یک کاوشگر دیجیتال

پشت صحنه گوگل: چگونه میلیاردها صفحه در روز ایندکس می‌شوند؟

تصور کنید در تیم مهندسی گوگل کار می‌کنید و مسئول طراحی کرالری هستید که باید هر روز میلیاردها صفحه وب را ایندکس کند. این کرالر مانند یک ارتش کوچک از ربات‌های هوشمند عمل می‌کند که هر کدام وظیفه خاصی دارند.

معماری این کرالر از چندین ماژول اصلی تشکیل شده است که با هم هماهنگ کار می‌کنند:



► Figure 20.1 The basic crawler architecture.

شکل 20.1: معماری پایه خزنده وب .

مرز (URL Frontier) URL مانند یک صف انتظار بی‌پایان است که تمام URL‌هایی که باید بازدید شوند را نگهداری می‌کند. در کرایینگ مداوم، حتی URL‌هایی که قبلاً بازدید شده‌اند نیز به این صف بازمی‌گردند تا در آینده دوباره بررسی شوند.

برای مثال، وقتی گوگل سایت دیجی‌کالا را کral می‌کند، تمام صفحات محصولات در این صف قرار می‌گیرند. با توجه به اینکه قیمت‌ها و موجودی محصولات به طور مداوم تغییر می‌کنند، این URL‌ها به طور منظم به صف بازگردانده می‌شوند تا اطلاعات به‌روز باقی بمانند.

🌐 **ماژول DNS: دفترچه تلفن اینترنت**

قبل از اینکه کralر بتواند صفحه‌ای را دریافت کند، باید آدرس IP سرور را پیدا کند. این کار توسط ماژول DNS انجام می‌شود که مانند دفترچه تلفن اینترنت عمل می‌کند.

برای مثال، وقتی کralر می‌خواهد صفحه `https://www.bing.com/search?q=crawling` را دریافت کند، ابتدا باید آدرس IP سرور `bing.com` را پیدا کند. این فرآیند ممکن است چند میلی‌ثانیه طول بکشد، اما برای کral‌رهایی که میلیاردها درخواست ارسال می‌کنند، این زمان اضافی می‌تواند تأثیر زیادی بر کارایی داشته باشد.

📡 **ماژول Fetch: پیک اینترنت**

ماژول Fetch مانند یک پیک دیجیتال است که به سرورها مراجعه کرده و محتوای صفحات وب را تحویل می‌گیرد. این ماژول از پروتکل HTTP برای برقراری ارتباط با سرورها استفاده می‌کند.

برای مثال، وقتی کralر گوگل می‌خواهد صفحه اصلی سایت ورزش سه را دریافت کند، یک درخواست HTTP به سرور ورزش سه ارسال می‌کند و سرور محتوای HTML صفحه را به کralر برمی‌گرداند.

📄 **ماژول Parse: خواننده هوشمند**

ماژول Parse مانند یک خواننده هوشمند عمل می‌کند که محتوای HTML را تحلیل کرده و اطلاعات مهم را استخراج می‌کند. این ماژول متن اصلی صفحه، لینک‌ها و سایر اطلاعات مفید را جدا می‌کند.

برای مثال، وقتی کralر صفحه اصلی سایت تپسل (`tapcell.ir`) را تحلیل می‌کند، این ماژول تمام لینک‌های موجود در صفحه را استخراج کرده و برای بررسی‌های بعدی به مرز URL اضافه می‌کند. همچنین، متن اصلی صفحه برای ایندکس‌گذاری به سیستم جستجو ارسال می‌شود.

🚫 **حذف محتوای تکراری: جلوگیری از اتلاف منابع**

یکی از چالش‌های بزرگ کralرها، جلوگیری از پردازش محتوای تکراری است. این کار با استفاده از اثر انگشت (Fingerprint) یا Shingling انجام می‌شود.

برای مثال، ممکن است یک مقاله خبری در چندین سایت خبری مختلف منتشر شود. کرالر باید بتواند تشخیص دهد که این محتوا تکراری است و از پردازش مجدد آن خودداری کند. این کار با محاسبه اثر انگشت محتوا و مقایسه آن با اثر انگشت‌های موجود انجام می‌شود.

🛡️ احترام به حریم خصوصی: پروتکل Robots.txt

وب‌مسترها می‌توانند با استفاده از فایل robots.txt مشخص کنند که کدام بخش‌های سایتشان نباید توسط کرالرها بازدید شود. این فایل مانند یک تابلو "ورود ممنوع" برای ربات‌ها عمل می‌کند.

برای مثال، فایل robots.txt سایت آمازون ممکن است مشخص کند که کرالرها نباید صفحات پرداخت و اطلاعات شخصی کاربران را بازدید کنند. این فایل باید مستقیماً قبل از دریافت صفحه بررسی شود تا از نقض قوانین سایت جلوگیری شود.

در اینجا یک نمونه از فایل robots.txt سایت توییتر آمده است:

```
* :User-agent
Disallow: /search
Disallow: /sms
Disallow: /sessions
Disallow: /settings

User-agent: Googlebot
Allow: /hashtags
/Allow: /i
/Allow: /intent
```

🌐 کرالینگ توزیع‌شده: کار تیمی در مقیاس جهانی

برای کرالینگ وب در مقیاس بزرگ، شرکت‌هایی مانند گوگل و مایکروسافت از معماری توزیع‌شده استفاده می‌کنند. در این معماری، کرالرها در سراسر جهان توزیع شده‌اند و هر کرالر مسئولیت کرالینگ مجموعه‌ای از سایت‌ها را بر عهده دارد.

برای مثال، گوگل ممکن است کرالرهایی در اروپا داشته باشد که کرالینگ سایت‌های اروپایی، و کرالرهایی در آسیا که سایت‌های آسیایی را کرال می‌کنند. این توزیع جغرافیایی به کاهش تأخیر و افزایش کارایی کمک می‌کند.

در این معماری، یک "تقسیم‌کننده میزبان" (Host Splitter) مشخص می‌کند که هر URL به کدام کرالر ارسال شود. این تقسیم‌بندی معمولاً بر اساس هش نام دامنه انجام می‌شود تا توزیع بار به صورت یکنواخت انجام شود.

⚡ چالش‌های عملی: از تئوری تا پیاده‌سازی

پیاده‌سازی یک کرالر در مقیاس بزرگ با چالش‌های زیادی همراه است:

- **مدیریت حافظه:** مرز URL ممکن است حاوی میلیاردها آدرس باشد که نیاز به مدیریت حافظه هوشمند دارد.
- **تحمل خطا:** اگر یک کرالر از کار بیفتد، باید مکانیزمی برای ادامه کار از آخرین نقطه وجود داشته باشد.
- **توزیع بار:** باید اطمینان حاصل شود که هیچ سروری تحت فشار بیش از حد قرار نگیرد.
- **به‌روزرسانی مداوم:** وب دائماً در حال تغییر است و کرالر باید بتواند این تغییرات را پیگیری کند.

برای مقابله با این چالش‌ها، شرکت‌های بزرگ از سیستم‌های پیچیده‌ای استفاده می‌کنند که شامل مکانیزم‌های ذخیره‌سازی توزیع‌شده، الگوریتم‌های هوشمند برای اولویت‌بندی URL‌ها و سیستم‌های نظارتی پیشرفته است.

20.2.2 تحلیل DNS: دفترچه تلفن اینترنت 🌐

📱 از آمازون تا گوگل: چرا DNS برای کرالرها حیاتی است؟

تصور کنید می‌خواهید به دوستان پیام دهید، اما فقط شماره تلفن او را دارید، نه نامش. حالا برعکس: فقط نام دوستان را می‌دانید، اما شماره تلفنش را نه. این دقیقاً همان چیزی است که در اینترنت اتفاق می‌افتد.

وقتی در مرورگر خود `www.amazon.com` را تایپ می‌کنید، کامپیوتر شما نمی‌داند که این سایت در کجای اینترنت قرار دارد. باید نام دامنه به آدرس IP عددی (مثل `174.129.25.17`) تبدیل شود. این فرآیند را **تحلیل DNS** (DNS Resolution) می‌نامیم.

🕒 گلوگاه سرعت: چرا DNS کرالرها را کند می‌کند؟

برای یک کاربر عادی، تحلیل DNS فقط چند میلی‌ثانیه طول می‌کشد و اصلاً قابل توجه نیست. اما برای یک کرالر که باید در ثانیه صدها صفحه را دریافت کند، این فرآیند می‌تواند یک فاجعه باشد!

چرا؟

- تحلیل DNS ممکن است **چند ثانیه** طول بکشد
- برای دامنه‌های ناشناخته یا مشکوک، این زمان حتی بیشتر هم می‌شود
- اگر کرالر بخواهد ۱۰۰۰ صفحه در ثانیه دریافت کند، هر تأخیر چند ثانیه‌ای یک مانع بزرگ است

📁 راه‌حل اولیه: کش کردن نتایج DNS

اولین راه‌حلی که به ذهن می‌رسد، **کش کردن** نتایج DNS است. مانند اینکه شماره تلفن دوستان را در دفترچه تلفن خود یادداشت کنید تا دیگر نیازی به جستجو نداشته باشید.

اما این راه‌حل محدودیت‌هایی دارد:

- به دلیل **محدودیت‌های ادب کرالینگ** (فاصله بین درخواست‌ها به یک سرور)، نرخ استفاده مجدد از کش کاهش می‌یابد
- آدرس‌های IP ممکن است تغییر کنند، بنابراین کش‌ها باید به‌طور منظم به‌روز شوند
- برای کرالرهای بزرگ، حتی کش هم کافی نیست

🚫 مشکل عمیق‌تر: رفتار همگام کتابخانه‌های DNS

مشکل اصلی اینجا است: کتابخانه‌های استاندارد DNS **همگام** (Synchronous) عمل می‌کنند. یعنی وقتی یک کرالر درخواست DNS ارسال می‌کند، تمام فعالیت‌های دیگر متوقف می‌شوند تا پاسخ دریافت شود.

این مانند این است که در یک رستوران، گارسون فقط بتواند یک سفارش را بگیرد و تا زمانی که غذا برای آن میز آماده نشده، از میزهای دیگر سفارش نگیرد!

🛣️ راه حل گوگل: DNS غیرهمگام سفارشی

شرکت‌هایی مانند گوگل و آمازون که کرال‌های بسیار بزرگی دارند، این مشکل را با پیاده‌سازی **resolver DNS سفارشی و غیرهمگام** حل کرده‌اند. این سیستم چگونه کار می‌کند؟

1. هر رشته کرالر، یک پیام DNS ارسال کرده و به کار خود ادامه می‌دهد (منتظر نمی‌ماند)
 2. یک **رشته اختصاصی DNS** به‌طور مداوم پاسخ‌ها را دریافت کرده و به رشته‌های مربوطه تحویل می‌دهد
 3. اگر پاسخی در زمان مشخصی (مثلاً ۱ ثانیه) دریافت نشد، درخواست مجدداً ارسال می‌شود
 4. این تلاش‌ها معمولاً تا ۵ بار تکرار می‌شوند و زمان انتظار به‌صورت نمایی افزایش می‌یابد (از ۱ ثانیه تا ۹۰ ثانیه)
- این طراحی به کرالر اجازه می‌دهد تا به جای انتظار برای پاسخ‌های DNS، به کارهای دیگر ادامه دهد و در نتیجه سرعت کرالینگ به‌شدت افزایش می‌یابد.

🎯 نتیجه‌گیری: چرا این موضوع مهم است؟

بدون راه‌حل‌های هوشمند برای تحلیل DNS، کرال‌های مدرن نمی‌توانستند با سرعت مورد نیاز عمل کنند. این به‌ویژه برای شرکت‌هایی مانند گوگل که باید میلیاردها صفحه را در روز ایندکس کنند، حیاتی است.

در واقع، بهینه‌سازی DNS یکی از رازهای موفقیت کرال‌های بزرگ است که به آنها اجازه می‌دهد وب را با سرعت فوق‌العاده‌ای پوشش دهند.

20.2.3 URL: رهبر ترافیک اینترنت 🚦

🌐 چالش گوگل: چگونه میلیاردها صفحه را منظم کرال کنیم؟

تصور کنید رهبر تیم کرالینگ گوگل هستید. هر ثانیه، هزاران صفحه جدید به وب اضافه می‌شود و شما باید تصمیم بگیرید کدام صفحه را اول بازدید کنید. این مثل مدیریت ترافیک یک کلان شهر است که در آن همه جاده‌ها یک‌باره شلوغ شده‌اند.

شما با دو چالش بزرگ روبرو هستید:

1. **اولویت‌دهی هوشمند:** صفحاتی که مهم هستند و به سرعت تغییر می‌کنند (مانند خبرهای فوری در bbc.com یا قیمت‌های روز در amazon.com) باید سریع‌تر از صفحات کم‌اهمیت‌تر کرال شوند.
2. **ادب در ترافیک:** شما نباید با ارسال همزمان صدها درخواست به یک سرور (مثلاً apple.com در زمان عرضه یک محصول جدید)، آن را از کار بیندازید.

🛣️ راه‌حل: معماری دو لایه مرز URL

برای حل این چالش، کرال‌های مدرن از یک سیستم صف هوشمند به نام **مرز URL** (URL Frontier) استفاده می‌کنند. این سیستم مانند یک مرکز کنترل ترافیک پیشرفته عمل می‌کند که از دو بخش اصلی تشکیل شده است:

- **صف‌های جلو (Front Queues):** این صف‌ها مانند خطوط ویژه وسایل نقلیه اضطراری عمل می‌کنند. هر کدام از این صف‌ها برای سطح اولویت خاصی طراحی شده‌اند. برای مثال، URL‌های مربوط به خبرهای فوری در بالاترین صف قرار می‌گیرند، در حالی که URL‌های وبلاگ‌های شخصی در صف‌های با اولویت پایین‌تر قرار می‌گیرند.
- **صف‌های عقب (Back Queues):** این صف‌ها مانند چراغ‌های راهنمایی برای هر خیابان (میزبان) خاص عمل می‌کنند. هر صف عقب فقط شامل URL‌های یک میزبان خاص است. این تضمین می‌کند که در هر لحظه فقط یک درخواست به یک سرور خاص ارسال شود.

🔧 هماهنگ‌کننده اصلی: هیپ زمان‌بندی

بین این دو لایه، یک **هیپ (Heap)** قرار دارد که مانند یک ساعت هوشمند عمل می‌کند. این هیپ برای هر میزبان، زودترین زمانی که می‌توان دوباره به آن درخواست ارسال کرد را ذخیره می‌کند.

برای مثال، اگر آخرین بار در ساعت ۱۰:۰۰:۰۰ به `nytimes.com` درخواست ارسال شده و این درخواست ۲ ثانیه طول کشیده باشد، هیپ ممکن است زمان مجاز بعدی را ساعت ۱۰:۰۰:۲۰ (یعنی ۱۰ برابر زمان درخواست قبلی) تنظیم کند.

🔄 سفر یک URL از ورود تا خروج

بیایید سفر یک URL خبری از `reuters.com` را دنبال کنیم:

1. **ورود و اولویت‌دهی:** URL خبری به سیستم وارد شده و به دلیل اهمیت بالای خبر، به بالاترین صف جلو اضافه می‌شود.
2. **انتقال به صف عقب:** وقتی یک صف عقب خالی می‌شود، یک "کنترل‌کننده هوشمند" URL را از صف‌های جلو (با تمایل به صف‌های با اولویت بالاتر) برداشته و آن را به صف عقب مربوط به `reuters.com` اضافه می‌کند.
3. **انتظار برای زمان مجاز:** کرالر به هیپ نگاه می‌کند و می‌بیند که آیا زمان مجاز برای ارسال درخواست به `reuters.com` فرا رسیده است یا خیر.
4. **کراالینگ:** پس از رسیدن به زمان مجاز، کرالر URL را از صف عقب برداشته و صفحه را دریافت می‌کند.
5. **به‌روزرسانی زمان:** پس از اتمام کراالینگ، زمان مجاز بعدی برای `reuters.com` در هیپ به‌روزرسانی می‌شود.

💾 مدیریت حجم عظیم: وقتی حافظه RAM کافی نیست

در کراالینگ در مقیاس وب، مرز URL ممکن است به میلیاردها آدرس برسد که بسیار بزرگ‌تر از حافظه RAM یک سرور است. راه‌حل این است که بخش اعظم مرز URL روی **دیسک** ذخیره شود.

این مانند یک پارکینگ خیلی بزرگ است که اکثر خودروها در آن منتظر می‌مانند و فقط تعداد محدودی در مرکز کنترل ترافیک (حافظه RAM) برای پردازش فوری قرار دارند. این طراحی به کرالرها اجازه می‌دهد تا کل وب را مدیریت کنند، حتی با منابع محاسباتی محدود.

```
In [8]: import heapq
import time
import random
from collections import deque, defaultdict

class URLFrontierSimulator:
    """
    :با دو لایه URL شبیه‌سازی مرز
    برای اولویت‌دهی بر اساس کیفیت و نرخ تغییر : (Front Queues) صف‌های جلو -
    """
```

برای رعایت ادب (یک میزبان در هر صف): (Back Queues) صف‌های عقب -
هیپ زمان‌بندی: برای مدیریت زمان مجاز درخواست به هر میزبان -
"""

```
def __init__(self, num_front_queues=3, num_back_queues=10):
    # لایه اولویت‌دهی
    self.front_queues = [deque() for _ in range(num_front_queues)] # F صف جلو
    self.num_front = num_front_queues

    # لایه ادب
    self.back_queues = [deque() for _ in range(num_back_queues)] # B صف عقب
    self.host_to_back_queue = {} # نگاشت میزبان -> شماره صف عقب
    self.next_back_queue = 0 # برای تخصیص چرخشی صف عقب جدید

    # هیپ زمان‌بندی: (زمان مجاز, شماره صف عقب)
    self.time_heap = []

    # سایر تنظیمات
    self.host_last_fetch = {} # آخرین زمان درخواست به هر میزبان
    self.host_fetch_duration = defaultdict(lambda: 1.0) # Fetch مدت زمان تخمینی

def extract_host(self, url):
    """(ساده‌شده) URL استخراج نام میزبان از"""
    if "://" in url:
        url = url.split("://", 1)[1]
    return url.split("/")[0].lower()

def assign_priority(self, url):
    """
    (بالاترین = self.num_front-1، پایین‌ترین = ۰) URL تخصیص اولویت به
    در واقعیت، این تابع بر اساس تاریخچه تغییر و کیفیت صفحه تصمیم می‌گیرد.
    """

    host = self.extract_host(url)
    # شبیه‌سازی: دامنه‌های خبری اولویت بالاتری دارند
    if any(keyword in host for keyword in ["news", "bbc", "cnn"]):
        return self.num_front - 1
    # صفحات معمولی اولویت تصادفی دریافت می‌کنند
    return random.randint(0, self.num_front - 2)

def add_url(self, url):
    """جدید به مرز URL افزودن"""
    priority = self.assign_priority(url)
    self.front_queues[priority].append(url)
    print(f"+ {url} (اولویت {priority}) به صف جلو URL افزودن")

def _get_or_create_back_queue(self, host):
    """دریافت یا ایجاد صف عقب برای یک میزبان"""
    if host in self.host_to_back_queue:
        return self.host_to_back_queue[host]

    # تخصیص یک صف عقب جدید
    back_queue_id = self.next_back_queue % len(self.back_queues)
    self.next_back_queue += 1
    self.host_to_back_queue[host] = back_queue_id

    # اولین زمان مجاز برای این میزبان
    current_time = time.time()
    heapq.heappush(self.time_heap, (current_time, back_queue_id))

    return back_queue_id

def _refill_back_queue(self, back_queue_id):
    """پر کردن یک صف عقب خالی از صف‌های جلو (با اولویت بالاتر)"""
    # جستجوی صف‌های جلو از اولویت بالا به پایین
    for priority in range(self.num_front - 1, -1, -1):
        if self.front_queues[priority]:
            url = self.front_queues[priority].popleft()
            host = self.extract_host(url)
            target_back_id = self._get_or_create_back_queue(host)

            # را به صف عقب صحیح خودش اضافه کن URL
            self.back_queues[target_back_id].append(url)

            # اگر صفی که پر کردیم همان صف درخواستی بود، کار تمام است
            if target_back_id == back_queue_id:
                return True
    # در غیر این صورت، به جستجو برای پر کردن صف درخواستی ادامه بده
    return False

def get_next_url(self):
    """بعدی برای کراپینگ (با رعایت ادب و اولویت) URL دریافت"""
    # اگر هیچ زمان‌بندی وجود ندارد، سعی کن یک صف را برای شروع پر کنی
    if not self.time_heap:
        # تمام صف‌های عقب را با بالاترین اولویت ممکن پر کن
        for i in range(len(self.back_queues)):
            self._refill_back_queue(i)

    # ای وجود ندارد URL اگر هنوز هیپ خالی است، یعنی هیچ
    if not self.time_heap:
        return None

    # دریافت زمان مجاز بعدی
    earliest_time, back_queue_id = heapq.heappop(self.time_heap)
    current_time = time.time()

    # اگر هنوز زمان مجاز نرسیده، منتظر بمان
    if current_time < earliest_time:
        time.sleep(earliest_time - current_time)
        current_time = earliest_time

    # اگر صف عقب خالی است، سعی کن آن را پر کنی
    if not self.back_queues[back_queue_id]:
        if not self._refill_back_queue(back_queue_id):
```

ی باقی نمانده URL اگر نتوانستیم آن را پر کنیم، یعنی هیچ #
return None

اگر بعد از پر کردن، هنوز هم صف خالی بود، به سراغ زمان بعدی برو
if not self.back_queues[back_queue_id]:
 return self.get_next_url()

از صف عقب URL دریافت
url = self.back_queues[back_queue_id].popleft()
host = self.extract_host(url)

به روزرسانی زمان‌ها
self.host_last_fetch[host] = current_time
fetch_duration = self.host_fetch_duration[host]
next_allowed_time = current_time + 10 * fetch_duration # سیاست Mercator

زمان مجاز بعدی را به هیپ اضافه کن
heapq.heappush(self.time_heap, (next_allowed_time, back_queue_id))

return url

مثال استفاده

if __name__ == "__main__":
 # با 3 صف اولویت و 5 صف ادب URL ایجاد مرز
 frontier = URLFrontierSimulator(num_front_queues=3, num_back_queues=5)

های نمونه URL افزودن

sample_urls = [
 "http://news-example.com/latest",
 "http://blog-site.org/post1",
 "http://cnn-breaking.news/update",
 "http://regular-site.com/page",
 "http://bbc-world.news/today",
 "http://another-blog.net/article"
]

print("ها به مرز URL افزودن")
for url in sample_urls:
 frontier.add_url(url)

print("\nشبیه‌سازی کرالینگ")

حالا حلقه اصلی کرالینگ را اجرا می‌کنیم

for i in range(6):
 url = frontier.get_next_url()
 if url:
 print(f"📄 کرالینگ: {url}")
 # شبیه‌سازی زمان پردازش
 time.sleep(0.1)
 else:
 print("ی برای کرالینگ باقی نمانده URL هیچ 📄")
 break

print("\n💡 نکته:")
print("را نشان می‌دهد URL این شبیه‌سازی مکانیزم اصلی مرز")
print("- (اولیت‌دهی بر اساس میزبان (دامنه‌های خبری اولویت بالاتر) -")
print("- (رعایت ادب با صف‌های میزبان‌محور -")
print("- (مدیریت زمان‌بندی با هیپ -")
print("- (در پیاده‌سازی واقعی، اولویت‌ها بر اساس تاریخچه تغییر صفحه محاسبه می‌شوند")

ها به مرز URL افزودن:

- ➕ http://news-example.com/latest (اولویت 2) URL افزودن
- ➕ http://blog-site.org/post1 (اولویت 1) URL افزودن
- ➕ http://cnn-breaking.news/update (اولویت 2) URL افزودن
- ➕ http://regular-site.com/page (اولویت 0) URL افزودن
- ➕ http://bbc-world.news/today (اولویت 2) URL افزودن
- ➕ http://another-blog.net/article (اولویت 0) URL افزودن

شبیه‌سازی کرالینگ:

- 📄 کرالینگ: http://news-example.com/latest
- 📄 کرالینگ: http://cnn-breaking.news/update
- 📄 کرالینگ: http://bbc-world.news/today
- 📄 کرالینگ: http://blog-site.org/post1
- 📄 کرالینگ: http://regular-site.com/page
- 📄 کرالینگ: http://another-blog.net/article

💡 نکته:

را نشان می‌دهد URL این شبیه‌سازی مکانیزم اصلی مرز:

- اولویت‌دهی بر اساس میزبان (دامنه‌های خبری اولویت بالاتر) -
- رعایت ادب با صف‌های میزبان‌محور -
- مدیریت زمان‌بندی با هیپ -

.در پیاده‌سازی واقعی، اولویت‌ها بر اساس تاریخچه تغییر صفحه محاسبه می‌شوند

چند کرالر واقعی👉

In [12]:

```
import time
import random
import re
from collections import deque, defaultdict
from urllib.parse import urljoin, urlparse

# --- توابع کمکی برای شبیه‌سازی ---

def simulate_dns_lookup(host):
    """با تأخیر تصادفی DNS شبیه‌سازی حل"""
    print(f"{host}... IP در حال پیدا کردن آدرس [DNS] 🕒")
    time.sleep(random.uniform(0.1, 0.5)) # شبیه‌سازی تأخیر شبکه
    return f"192.168.1.{random.randint(1, 254)}"

def simulate_http_fetch(url):
    """شبیه‌سازی دریافت صفحه با تأخیر تصادفی"""
    print(f"{url}... [HTTP] 🌐 در حال دریافت صفحه از")
```

```
شبيه‌سازی تأخير دریافت # uniform(0.5, 1.5))
# شبیه‌سازی تأخیر دریافت
# ساختگی برای شبیه‌سازی برمی‌گردانیم HTML یک محتوای
return f"""
<html>
<head><title>Page at {url}</title></head>
<body>
    <h1>Welcome to {urlparse(url).netloc}</h1>
    <p>This is a sample page.</p>
    <a href="{urljoin(url, '/page1.html')}">Link 1</a><br>
    <a href="{urljoin(url, '/page2.html')}">Link 2</a><br>
    <a href="http://external-site.com/about">External Link</a>
</body>
</html>
"""

def extract_links(html_content, base_url):
    """(ساده‌شده) HTML استخراج لینک‌ها از محتوای"""
    # پیدا کردن تمام لینک‌ها در تگ <a href="...">
    links = re.findall(r'<a\s+(?:[^\s]*)href="([^\"]*)"', html_content, re.IGNORECASE)
    # تبدیل لینک‌های نسبی به مطلق
    absolute_links = [urljoin(base_url, link) for link in links]
    return list(set(absolute_links)) # حذف لینک‌های تکراری

def check_robots_txt(url, robots_cache):
    """URL برای یک robots.txt بررسی فایل"""
    host = urlparse(url).netloc
    if host in robots_cache:
        print(f"📄 [Robots] استفاده از کش [Robots] برای {host}")
        return robots_cache[host]

    print(f"📄 [Robots] در حال دریافت robots.txt برای {host}...")
    time.sleep(0.2) # شبیه‌سازی دریافت فایل
    # robots.txt شبیه‌سازی یک فایل
    robots_cache[host] = {'allowed': True}
    return robots_cache[host]

# --- کلاس اصلی کرالر ---

class WebCrawler:
    """
    «ادب» URL یک کرالر وب ساده که مفاهیم اصلی مانند مرز
    را پیاده‌سازی می‌کند robots.txt حذف تکراری و احترام به
    """
    def __init__(self, seed_urls, max_pages=20, politeness_delay=3):
        """
        مقداردهی اولیه کرالر
        seed_urls: های شروع URL لیست
        max_pages: حداکثر تعداد صفحات برای دریافت
        politeness_delay: تأخیر به ثانیه بین درخواست‌ها به یک میزبان
        """
        self.max_pages = max_pages
        self.politeness_delay = politeness_delay

        # ها که باید بازدید شوند URL صفی از URL مرز
        self.url_frontier = deque(seed_urls)

        # هابی که قبلاً دیده‌ایم (برای جلوگیری از تکرار URL مجموعه‌ای از)
        self.visited_urls = set()

        # هر میزبان robots.txt کش برای فایل‌های
        self.robots_cache = {}

        # صف برای هر میزبان برای رعایت ادب (یک میزبان، یک صف)
        self.host_queues = defaultdict(deque)

        # زمان آخرین درخواست به هر میزبان
        self.last_fetch_time = defaultdict(float)

        # آمار کرالر
        self.crawled_count = 0
        self.total_links_found = 0

    def _is_polite(self, host):
        """آیا می‌توان به این میزبان درخواست فرستاد؟"""
        last_time = self.last_fetch_time[host]
        return (time.time() - last_time) >= self.politeness_delay

    def _add_to_frontier(self, new_urls, source_url):
        """URL اضافه کردن لینک‌های جدید به مرز"""
        for url in new_urls:
            # حذف تکراری: اگر قبلاً دیدیم یا در مرز است، اضافه نمی‌کنیم 1.
            if url in self.visited_urls or url in self.url_frontier:
                continue

            # 2. robots.txt احترام به
            if not check_robots_txt(url, self.robots_cache)['allowed']:
                print(f"🚫 {url} لینک به دلیل محدودیت robots.txt رد شد.")
                continue

            # 3. اضافه به مرز
            self.url_frontier.append(url)
            print(f"➕ {url} لینک جدید (از {source_url}) به مرز اضافه شد")

    def crawl_page(self, url):
        """کراال کردن یک صفحه: دریافت، تحلیل و اضافه کردن لینک‌ها"""
        print(f"\n--- {url} شروع کراال صفحه [CRAWL] ---")

        host = urlparse(url).netloc
        # DNS مرحله 1: حل
        ip_address = simulate_dns_lookup(host)

        # مرحله 2: دریافت صفحه
        html_content = simulate_http_fetch(url)
```

```
# مرحله 3: تحلیل صفحه و استخراج لینک‌ها
new_links = extract_links(html_content, url)

# مرحله 4: اضافه کردن لینک‌های جدید به مرز
self._add_to_frontier(new_links, source_url=url)

# به‌روزرسانی آمار و وضعیت
self.visited_urls.add(url)
self.crawled_count += 1
self.total_links_found += len(new_links)
self.last_fetch_time[host] = time.time()

print(f"--- [CRAWL] کral صفحه {url} تمام شد. {len(new_links)} لینک جدید پیدا شد. ---")

def start_crawling(self):
    """شروع فرایند کralینگ اصلی"""
    print("...کralر وب شروع به کار کرد 🚀")

    while self.url_frontier and self.crawled_count < self.max_pages:
        # از مرز URL مرحله 1: انتخاب یک
        current_url = self.url_frontier.popleft()

        host = urlparse(current_url).netloc

        # مرحله 2: رعایت ادب
        if not self._is_polite(host):
            print(f"{host}... منتظر ماندن برای ادب به میزبان 🕒")
            time.sleep(self.politeness_delay)

        # مرحله 3: کral کردن صفحه
        self.crawl_page(current_url)

    print("\n✅ کralینگ به پایان رسید.")
    print(f"📊 آمار نهایی:")
    print(f"تعداد کل صفحات کral شده - {self.crawled_count}")
    print(f"تعداد کل لینک‌های کشف شده - {self.total_links_found}")
    print(f"های باقی‌مانده در مرزURL تعداد - {len(self.url_frontier)}")

# --- مثال اجرا ---
if __name__ == "__main__":
    # های شروعURL لیست (seed set)
    SEED_URLS = [
        "http://example.com/",
        "http://university.edu/departments",
        "http://blog-site.org/index"
    ]

    # ساخت و اجرای کralر
    crawler = WebCrawler(seed_urls=SEED_URLS, max_pages=10, politeness_delay=2)
    crawler.start_crawling()
```

...کراالر وب شروع به کار کرد

```
--- [CRAWL] شروع کراال صفحه http://example.com/ ---
🔍 [DNS] example.com برای IP در حال پیدا کردن آدرس ...
🌐 [HTTP] http://example.com/... در حال دریافت صفحه از
🤖 [Robots] example.com برای robots.txt در حال دریافت
✅ به مرز اضافه شد http://example.com/page1.html (از http://example.com/) لینک جدید
🤖 [Robots] example.com برای robots.txt استفاده از کش
✅ به مرز اضافه شد http://example.com/page2.html (از http://example.com/) لینک جدید
🤖 [Robots] external-site.com برای robots.txt در حال دریافت
✅ به مرز اضافه شد http://external-site.com/about (از http://example.com/) لینک جدید
--- [CRAWL] http://example.com/ کراال صفحه ---

--- [CRAWL] شروع کراال صفحه http://university.edu/departments ---
🔍 [DNS] university.edu برای IP در حال پیدا کردن آدرس ...
🌐 [HTTP] http://university.edu/departments... در حال دریافت صفحه از
🤖 [Robots] university.edu برای robots.txt در حال دریافت
✅ به مرز اضافه شد http://university.edu/page2.html (از http://university.edu/departments) لینک جدید
🤖 [Robots] university.edu برای robots.txt استفاده از کش
✅ به مرز اضافه شد http://university.edu/page1.html (از http://university.edu/departments) لینک جدید
--- [CRAWL] http://university.edu/departments کراال صفحه ---

--- [CRAWL] شروع کراال صفحه http://blog-site.org/index ---
🔍 [DNS] blog-site.org برای IP در حال پیدا کردن آدرس ...
🌐 [HTTP] http://blog-site.org/index... در حال دریافت صفحه از
🤖 [Robots] blog-site.org برای robots.txt در حال دریافت
✅ به مرز اضافه شد http://blog-site.org/page2.html (از http://blog-site.org/index) لینک جدید
🤖 [Robots] blog-site.org برای robots.txt استفاده از کش
✅ به مرز اضافه شد http://blog-site.org/page1.html (از http://blog-site.org/index) لینک جدید
--- [CRAWL] http://blog-site.org/index کراال صفحه ---

--- [CRAWL] شروع کراال صفحه http://example.com/page1.html ---
🔍 [DNS] example.com برای IP در حال پیدا کردن آدرس ...
🌐 [HTTP] http://example.com/page1.html... در حال دریافت صفحه از
🤖 [Robots] example.com برای robots.txt استفاده از کش
✅ به مرز اضافه شد http://example.com/page1.html (از http://example.com/page1.html) لینک جدید
--- [CRAWL] http://example.com/page1.html کراال صفحه ---
⌚ example.com... منتظر ماندن برای ادب به میزبان

--- [CRAWL] شروع کراال صفحه http://example.com/page2.html ---
🔍 [DNS] example.com برای IP در حال پیدا کردن آدرس ...
🌐 [HTTP] http://example.com/page2.html... در حال دریافت صفحه از
🤖 [Robots] example.com برای robots.txt استفاده از کش
✅ به مرز اضافه شد http://example.com/page2.html (از http://example.com/page2.html) لینک جدید
--- [CRAWL] http://example.com/page2.html کراال صفحه ---

--- [CRAWL] شروع کراال صفحه http://external-site.com/about ---
🔍 [DNS] external-site.com برای IP در حال پیدا کردن آدرس ...
🌐 [HTTP] http://external-site.com/about... در حال دریافت صفحه از
🤖 [Robots] external-site.com برای robots.txt استفاده از کش
✅ به مرز اضافه شد http://external-site.com/about (از http://external-site.com/about) لینک جدید
🤖 [Robots] external-site.com برای robots.txt استفاده از کش
✅ به مرز اضافه شد http://external-site.com/page2.html (از http://external-site.com/about) لینک جدید
🤖 [Robots] external-site.com برای robots.txt استفاده از کش
✅ به مرز اضافه شد http://external-site.com/page1.html (از http://external-site.com/about) لینک جدید
--- [CRAWL] http://external-site.com/about کراال صفحه ---

--- [CRAWL] شروع کراال صفحه http://university.edu/page2.html ---
🔍 [DNS] university.edu برای IP در حال پیدا کردن آدرس ...
🌐 [HTTP] http://university.edu/page2.html... در حال دریافت صفحه از
🤖 [Robots] university.edu برای robots.txt استفاده از کش
✅ به مرز اضافه شد http://university.edu/page2.html (از http://university.edu/page2.html) لینک جدید
--- [CRAWL] http://university.edu/page2.html کراال صفحه ---
⌚ university.edu... منتظر ماندن برای ادب به میزبان

--- [CRAWL] شروع کراال صفحه http://university.edu/page1.html ---
🔍 [DNS] university.edu برای IP در حال پیدا کردن آدرس ...
🌐 [HTTP] http://university.edu/page1.html... در حال دریافت صفحه از
🤖 [Robots] university.edu برای robots.txt استفاده از کش
✅ به مرز اضافه شد http://university.edu/page1.html (از http://university.edu/page1.html) لینک جدید
--- [CRAWL] http://university.edu/page1.html کراال صفحه ---

--- [CRAWL] شروع کراال صفحه http://blog-site.org/page2.html ---
🔍 [DNS] blog-site.org برای IP در حال پیدا کردن آدرس ...
🌐 [HTTP] http://blog-site.org/page2.html... در حال دریافت صفحه از
🤖 [Robots] blog-site.org برای robots.txt استفاده از کش
✅ به مرز اضافه شد http://blog-site.org/page2.html (از http://blog-site.org/page2.html) لینک جدید
--- [CRAWL] http://blog-site.org/page2.html کراال صفحه ---
⌚ blog-site.org... منتظر ماندن برای ادب به میزبان

--- [CRAWL] شروع کراال صفحه http://blog-site.org/page1.html ---
🔍 [DNS] blog-site.org برای IP در حال پیدا کردن آدرس ...
🌐 [HTTP] http://blog-site.org/page1.html... در حال دریافت صفحه از
🤖 [Robots] blog-site.org برای robots.txt استفاده از کش
✅ به مرز اضافه شد http://blog-site.org/page1.html (از http://blog-site.org/page1.html) لینک جدید
--- [CRAWL] http://blog-site.org/page1.html کراال صفحه ---

✅ کراالینگ به پایان رسید.
📊 آمار نهایی:
- تعداد کل صفحات کراال شده: 10
- تعداد کل لینک‌های کشف شده: 30
- های باقی‌مانده در مرز: URL9 تعداد
```

```
In [17]: from requests import Session
from requests.adapters import HTTPAdapter
try:
    # urllib3 v2+
    from urllib3.util.retry import Retry
except Exception:
    # fallback for some environments
    from requests.packages.urllib3.util.retry import Retry

from bs4 import BeautifulSoup
```

```

from collections import deque
from concurrent.futures import ThreadPoolExecutor, wait
from threading import Lock
from typing import List, Optional, Dict, Any
import json
import time
import random

class IMDbCrawler:
    """
    IMDb crawler (Top 250-seeded) that only extracts 4 fields:
    - title : str
    - genres : List[str]
    - stars : List[str]
    - summaries : List[str]

    Networking:
    - Session with retries/backoff for 429/5xx
    - Global rate limiter shared by threads
    """

    BASE_URL = "https://www.imdb.com"
    TOP_250_URL = f"{BASE_URL}/chart/top/"

    headers = {
        "User-Agent": (
            "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
            "AppleWebKit/537.36 (KHTML, like Gecko) "
            "Chrome/122.0.0.0 Safari/537.36"
        )
    }

    def __init__(
        self,
        crawling_threshold: int = 1000,
        *,
        max_workers: int = 3,
        min_interval: float = 1.0,
        timeout: int = 30,
        retries: int = 6
    ) -> None:
        self.crawling_threshold = crawling_threshold
        self.not_crawled: deque[str] = deque()
        self.crawled: List[Dict[str, Any]] = []
        self.added_ids: set[str] = set()

        self.add_list_lock = Lock()
        self.add_queue_lock = Lock()

        self.max_workers = max_workers
        self.min_interval = float(min_interval)
        self.timeout = timeout
        self.session = self._build_session(retries)

        self._last_request_ts = 0.0
        self._rate_lock = Lock()

    # ----- Networking -----

    def _build_session(self, retries: int) -> Session:
        s = Session()
        try:
            retry = Retry(
                total=retries,
                connect=retries,
                read=retries,
                status=retries,
                backoff_factor=1.0,
                status_forcelist=[429, 500, 502, 503, 504],
                allowed_methods=frozenset({"GET"}),
                raise_on_status=False,
            )
        except TypeError:
            retry = Retry(
                total=retries,
                connect=retries,
                read=retries,
                status=retries,
                backoff_factor=1.0,
                status_forcelist=[429, 500, 502, 503, 504],
                method_whitelist=frozenset({"GET"}),
                raise_on_status=False,
            )
        adapter = HTTPAdapter(max_retries=retry, pool_connections=100, pool_maxsize=100)
        s.mount("https://", adapter)
        s.mount("http://", adapter)
        s.headers.update({
            "User-Agent": self.headers.get("User-Agent", ""),
            "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8",
            "Accept-Language": "en-US,en;q=0.9",
            "Accept-Encoding": "gzip, deflate, br",
            "Connection": "keep-alive",
            "Referer": "https://www.imdb.com/",
        })
        return s

    def crawl(self, url: str):
        """GET with retries/timeout + GLOBAL rate limit."""
        with self._rate_lock:
            now = time.time()
            wait_time = self.min_interval - (now - self._last_request_ts)
            if wait_time > 0:

```

```

        time.sleep(wait_time)
        time.sleep(random.uniform(0.05, 0.25)) # jitter
        self._last_request_ts = time.time()

    try:
        resp = self.session.get(url, timeout=self.timeout)
        if resp.status_code == 200:
            return resp
        print(f"Failed to crawl {url}: Status code {resp.status_code}")
    except Exception as e:
        print(f"An error occurred while crawling {url}: {e}")
    return None

# ----- Seeds -----

def extract_top_250(self) -> None:
    """Seed the queue from IMDb Top 250 page."""
    try:
        response = self.crawl(self.TOP_250_URL)
        if response is None:
            print("Failed to retrieve Top 250 movies list.")
            return

        soup = BeautifulSoup(response.content, "html.parser")
        movies = soup.find_all("li", class_="ipc-metadata-list-summary-item")

        self.not_crawled.clear()
        self.added_ids.clear()

        for movie in movies:
            link = movie.find("a", href=True)
            if not link:
                continue
            href = link.get("href", "")
            parts = href.split("/")
            if len(parts) > 2:
                movie_id = parts[2]
                if movie_id and movie_id not in self.added_ids:
                    self.not_crawled.append(f"{self.BASE_URL}/title/{movie_id}/")
                    self.added_ids.add(movie_id)
    except Exception as e:
        print(f"An error occurred while extracting Top 250 movies: {e}")

# ----- Public -----

def start_crawling(self) -> None:
    """Crawl titles from Top 250 (and related) until threshold/queue empty."""
    self.extract_top_250()

    futures = []
    crawled_counter = 0

    with ThreadPoolExecutor(max_workers=self.max_workers) as executor:
        while crawled_counter < self.crawling_threshold and self.not_crawled:
            url = self.not_crawled.popleft()
            futures.append(executor.submit(self.crawl_page_info, url, crawled_counter))
            crawled_counter += 1

            if not self.not_crawled:
                wait(futures)
                futures.clear()

        if futures:
            wait(futures)

def crawl_page_info(self, url: str, crawler_counter: int) -> Optional[Dict[str, Any]]:
    """
    Crawl a single title page and extract ONLY:
    - title
    - genres
    - stars
    - summaries
    Related links may be appended to the queue.
    """
    response = self.crawl(url)
    if response is None:
        print(f"Failed to crawl page: {url}")
        return None

    soup = BeautifulSoup(response.content, "html.parser")

    movie = self.get_imdb_instance()
    movie["id"] = self.get_id_from_URL(url)
    movie["title"] = self.get_title(soup)
    movie["genres"] = self.get_genres(soup)
    movie["stars"] = self.get_stars(soup)

    # Prefer full plot summaries page; fallback to first page short plot if empty
    summaries = self.get_summaries(url)
    if not summaries:
        short = self.get_first_page_summary(soup)
        if short and short != "Summary not found.":
            summaries = [short]
    movie["summaries"] = summaries

    with self.add_list_lock:
        self.crawled.append(movie)

    # Optional queue expansion via "More like this"
    related_links = self.get_related_links(soup)
    if related_links:
        with self.add_queue_lock:
            for link in related_links:

```

```

        movie_id = self.get_id_from_URL(link)
        if movie_id and movie_id not in self.added_ids:
            self.not_crawled.append(link)
            self.added_ids.add(movie_id)

    print(crawler_counter, f"Successfully crawled and processed: {url}")
    return movie

# ----- Serialization -----

def save_to_json(self) -> None:
    """Write crawled results to a JSON file."""
    try:
        with open("IMDB_crawled.json", "w", encoding="utf-8") as f:
            json.dump(self.crawled, f, ensure_ascii=False, indent=4)
        print(f"✅ اطلاعات {len(self.crawled)} فیلم با موفقیت در فایل IMDB_crawled.json ذخیره شد.")
    except Exception as e:
        print(f"An error occurred while writing to JSON file: {e}")

# ----- Extraction helpers -----

@staticmethod
def get_imdb_instance() -> Dict[str, Any]:
    return {
        "id": None,
        "title": None,
        "genres": None,
        "stars": None,
        "summaries": None,
    }

@staticmethod
def get_id_from_URL(url: str) -> Optional[str]:
    """https://www.imdb.com/title/tt0111161/ -> tt0111161"""
    try:
        parts = url.split("/")
        title_index = parts.index("title")
        if title_index + 1 < len(parts):
            return parts[title_index + 1]
        print("Error: 'title' segment found, but no ID follows in the URL.")
    except ValueError:
        print("Error: The URL does not contain a 'title' segment.")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")
    return None

def get_summary_link(self, url: str) -> Optional[str]:
    """ /title/<id>/ -> /title/<id>/plotsummary """
    try:
        movie_id = self.get_id_from_URL(url)
        if movie_id:
            return f"{self.BASE_URL}/title/{movie_id}/plotsummary"
        print("Failed to extract movie ID from URL.")
    except Exception as e:
        print(f"Failed to get summary link: {e}")
    return None

def get_related_links(self, soup: BeautifulSoup) -> List[str]:
    """Collect 'More like this' title links to expand crawling (optional)."""
    try:
        related_links: List[str] = []
        section = soup.find("section", {"data-testid": "MoreLikeThis"})
        if not section:
            return related_links
        links = section.find_all("a", class_="ipc-lockup-overlay ipc-focusable")
        for link in links:
            href = link.get("href")
            if href:
                related_links.append(f"https://www.imdb.com{href}")
        return related_links
    except Exception as e:
        print(f"Failed to get related links: {e}")
    return []

@staticmethod
def get_title(soup: BeautifulSoup) -> Optional[str]:
    try:
        tag = soup.find("h1")
        return tag.text.strip() if tag else None
    except Exception as e:
        print(f"Failed to get title: {e}")
    return None

@staticmethod
def get_first_page_summary(soup: BeautifulSoup) -> str:
    try:
        tag = soup.find("span", {"data-testid": "plot-x1"})
        return tag.text.strip() if tag else "Summary not found."
    except Exception as e:
        print(f"Failed to get summary: {e}")
    return "Summary not found."

@staticmethod
def _get_principal_names(soup: BeautifulSoup, label_text: str) -> List[str]:
    """Extract names (e.g., Stars) from principal credit blocks."""
    try:
        names: List[str] = []
        blocks = soup.find_all("li", {"data-testid": "title-pc-principal-credit"})
        for block in blocks:
            for tag_name, cls in [
                ("span", "ipc-metadata-list-item__label"),
                ("a", "ipc-metadata-list-item__label"),
            ]:

```

```
        label_tag = block.find(tag_name, class_=cls)
        if label_tag and label_tag.text.strip() == label_text:
            for a in block.find_all("a", class_="ipc-metadata-list-item__list-content-item"):
                txt = a.text.strip()
                if txt:
                    names.append(txt)
            return list(set(names)) if names else []
    except Exception as e:
        print(f"Failed to get {label_text.lower()}: {e}")
    return []

def get_stars(self, soup: BeautifulSoup) -> List[str]:
    return self._get_principal_names(soup, "Stars")

@staticmethod
def get_genres(soup: BeautifulSoup) -> List[str]:
    genres: List[str] = []
    try:
        scroller = soup.find("div", class_="ipc-chip-list__scroller")
        if not scroller:
            return genres
        for span in scroller.find_all("span", class_="ipc-chip__text"):
            txt = span.text.strip()
            if txt:
                genres.append(txt)
    except Exception as e:
        print(f"Failed to get genres: {e}")
    return genres

def get_summaries(self, url: str) -> List[str]:
    summaries: List[str] = []
    try:
        summaries_url = self.get_summary_link(url)
        if not summaries_url:
            return summaries
        resp = self.crawl(summaries_url)
        if not resp:
            return summaries
        soup = BeautifulSoup(resp.content, "html.parser")
        section = soup.find("div", {"data-testid": "sub-section-summaries"})
        if not section:
            return summaries
        for li in section.find_all("li"):
            div = li.find("div", class_="ipc-html-content-inner-div")
            if div and div.text:
                summaries.append(div.text.strip())
    except Exception as e:
        print(f"Failed to get summaries: {e}")
    return summaries

def main() -> None:
    imdb_crawler = IMDBCrawler(
        crawling_threshold=10, # فقط ۱۰ فیلم صرفاً جهت تمرین
        max_workers=3,
        min_interval=1.0,
        timeout=30,
        retries=6,
    )
    imdb_crawler.start_crawling()

    # ذخیره اطلاعات در فایل پس از اتمام کراپینگ
    imdb_crawler.save_to_json()

if __name__ == "__main__":
    main()
```

0 Successfully crawled and processed: https://www.imdb.com/title/tt0111161/
1 Successfully crawled and processed: https://www.imdb.com/title/tt0068646/
2 Successfully crawled and processed: https://www.imdb.com/title/tt0468569/
3 Successfully crawled and processed: https://www.imdb.com/title/tt0071562/
4 Successfully crawled and processed: https://www.imdb.com/title/tt0050083/
5 Successfully crawled and processed: https://www.imdb.com/title/tt0167260/
6 Successfully crawled and processed: https://www.imdb.com/title/tt0108052/
8 Successfully crawled and processed: https://www.imdb.com/title/tt0110912/
7 Successfully crawled and processed: https://www.imdb.com/title/tt0120737/
9 Successfully crawled and processed: https://www.imdb.com/title/tt0060196/
✅ ذخیره شد IMDB_crawled.json 10 اطلاعات فیلم با موفقیت در فایل.

20.3 توزیع ایندکس‌ها: معماری پشت موتورهای جستجوی بزرگ

🌐 چالش گوگل: مدیریت میلیاردها صفحه در هزاران سرور

تصور کنید مهندسی در گوگل هستید و باید سیستمی طراحی کنید که بتواند به میلیاردها صفحه وب پاسخ دهد. هر روز حدود ۳.۵ میلیارد جستجو در گوگل انجام می‌شود و هر جستجو باید در کسری از ثانیه پاسخ داده شود.

برای حل این چالش، گوگل و سایر شرکت‌های بزرگ جستجو از **توزیع ایندکس‌ها** در خوشه‌های بزرگ کامپیوتری استفاده می‌کنند. دو استراتژی اصلی برای این کار وجود دارد:

در این روش، واژگان ایندکس به زیرمجموعه‌هایی تقسیم می‌شوند و هر سرور مسئول واژه‌های خاصی است. برای مثال، یک سرور ممکن است واژه‌هایی که با "آ" شروع می‌شوند را نگهداری کند، در حالی که سرور دیگر مسئول واژه‌هایی با "ب" است.

در عمل، این روش با چالش‌های جدی روبرو است:

- وقتی کاربر عبارت "خرید لپ‌تاپ" را جستجو می‌کند، سیستم باید لیست‌های بزرگ را بین سرورهای مختلف منتقل کند.
- توزیع بار به صورت یکنواخت دشوار است، زیرا برخی واژه‌ها (مانند "news") بسیار محبوب‌تر از سایر واژه‌ها هستند.

در این روش که در شرکت‌هایی مانند گوگل و مایکروسافت رایج‌تر است، هر سرور ایندکس کامل برای زیرمجموعه‌ای از اسناد را نگهداری می‌کند. برای مثال، سرور شماره ۱ ممکن است مسئول ایندکس تمام صفحات وبسایت‌های خبری باشد، در حالی که سرور شماره ۲ مسئول وبلاگ‌های شخصی است.

وقتی کاربر عبارتی را جستجو می‌کند، پرس‌وجو به همه سرورها ارسال می‌شود و نتایج از هر سرور جمع‌آوری و ادغام می‌شوند.

چگونه اسناد را بین سرورها توزیع کنیم؟

در سیستم تقسیم‌بندی بر اساس اسناد، دو روش اصلی برای توزیع اسناد بین سرورها وجود دارد:

- توزیع بر اساس میزبان:** تمام صفحات یک دامنه (مانند wikipedia.org) به یک سرور اختصاص می‌یابند. این روش ساده است، اما مشکلی جدی دارد: بسیاری از جستجوها نتایجشان از تعداد کمی دامنه‌های محبوب (مانند آمازون، یوتیوب و فیسبوک) می‌آید، بنابراین بار محاسباتی به صورت یکنواخت توزیع نمی‌شود.
- توزیع بر اساس هش URL:** هر URL با استفاده از یک تابع هش به یک سرور نگاشت می‌شود. این روش تضمین می‌کند که بار محاسباتی به صورت یکنواخت بین سرورها توزیع شود. برای مثال، URLهای وبسایت‌های مختلف به صورت تصادفی بین سرورها پخش می‌شوند.

تکنیک پیشرفته: تقسیم‌بندی سلسله‌مراتبی

برای بهینه‌سازی بیشتر، شرکت‌هایی مانند گوگل از تکنیک **تقسیم‌بندی سلسله‌مراتبی** استفاده می‌کنند:

- اسناد بر اساس احتمال رتبه‌بندی بالا به دو دسته تقسیم می‌شوند: اسناد باکیفیت تر و اسناد باکیفیت پایین.
 - اسناد باکیفیت تر در **ایندکس‌های اولیه** قرار می‌گیرند که معمولاً در حافظه‌های سریع‌تر (مانند SSD) ذخیره می‌شوند.
 - اسناد باکیفیت در **ایندکس‌های ثانویه** ذخیره می‌شوند که ممکن است در حافظه‌های کندتر (مانند HDD) قرار داشته باشند.
 - هنگام جستجو، ابتدا فقط ایندکس‌های اولیه جستجو می‌شوند. اگر نتایج کافی نبود، سیستم به جستجو در ایندکس‌های ثانویه می‌پردازد.
- این روش مانند جستجو در یک کتابخانه بزرگ است که ابتدا در بخش مراجع معتبر جستجو می‌کنید و فقط در صورت نیاز، به بخش‌های کمتر مرجع مراجعه می‌کنید.

یکی از چالش‌های مهم در سیستم توزیع شده، محاسبه آمارهای سراسری مانند (Inverse Document Frequency) **idf** است که برای رتبه‌بندی نتایج جستجو ضروری است.

برای حل این مشکل، شرکت‌ها از فرآیندهای پس‌زمینه‌ای استفاده می‌کنند که به صورت دوره‌ای آمارهای سراسری را محاسبه کرده و به سرورهای مختلف ارسال می‌کنند. این فرآیندها معمولاً در ساعات کم‌ترافیک شبکه اجرا می‌شوند تا تأثیری بر عملکرد سیستم نداشته باشند.

20.4 سرورهای همبستگی: نقشه راهنمای وب

چرا گوگل به نقشه وب نیاز دارد؟

تصور کنید در تیم مهندسی گوگل کار می‌کنید و می‌خواهید الگوریتم PageRank را بهبود دهید. برای این کار، باید بدانید کدام وب‌سایت‌های معتبر به صفحه اصلی دانشگاه استنفورد لینک داده‌اند. یا شاید بخواهید بدانید کدام صفحات از سایت **bbc.com** به یک خبر خاص اشاره دارند.

این پرس‌وجوها که به آنها **پرس‌وجوهای همبستگی** (Connectivity Queries) می‌گویند، برای موتورهای جستجو حیاتی هستند. اما پاسخ به آنها چالش‌برانگیز است، زیرا نیاز به ذخیره‌سازی کل گراف وب (که میلیاردها صفحه و تریلیون‌ها لینک دارد) در حافظه دارد.

چالش ذخیره‌سازی: وقتی وب بزرگتر از حافظه است

بیا بید ابعاد این چالش را درک کنیم. اگر وب شامل ۴ میلیارد صفحه باشد و هر صفحه به طور متوسط ۱۰ لینک داشته باشد، برای ذخیره‌سازی هر لینک (با شناسه مبدأ و مقصد ۳۲ بیتی)، به **۳۲۰ گیگابایت** حافظه نیاز داریم! این مانند تلاش برای بارگذاری کل کتابخانه کنگره در RAM یک لپ‌تاپ است.

راه‌حل، فشردن هوشمندانه‌ای است که هم فضا را کاهش دهد و هم پرس‌وجوهای سریع را پشتیبانی کند. این تفاوت اصلی با فشردن معمولی است؛ ما نه تنها داده‌ها را فشرده می‌کنیم، بلکه باید بتوانیم به سرعت به آنها دسترسی پیدا کنیم.

سه تکنیک کلیدی برای فشردن گراف وب

مهندسان شرکت‌هایی مانند گوگل و آمازون از ترکیبی از سه تکنیک هوشمندانه برای حل این مشکل استفاده می‌کنند:

1. فریب هوشمندانه: کدگذاری فاصله‌ها (Gap Encoding)

به جای ذخیره شماره مطلق هر صفحه مقصد، **تفاضل آن با صفحه قبلی** را ذخیره می‌کنیم. چون لیست‌ها مرتب هستند، این فاصله‌ها (گپ‌ها) معمولاً اعداد کوچکی هستند و فضای بسیار کمتری اشغال می‌کنند. این تکنیکی است که در فشردن‌سازی ایندکس‌های معکوس نیز استفاده می‌شود.

2. پیدا کردن الگوها: شباهت بین ردیف‌ها

بسیاری از وب‌سایت‌ها از الگوی یکسانی برای ساختار خود استفاده می‌کنند. برای مثال، تقریباً تمام صفحات دانشگاه

استنفورد لینک‌هایی به صفحه اصلی، نقشه سایت، و صفحه کپی‌رایت دارند. به جای ذخیره این لیست برای هر صفحه، می‌توانیم یک **ردیف نمونه** (prototype) ایجاد کرده و تفاوت‌های هر صفحه را با آن نمونه ذخیره کنیم.

3. **هم‌جواری: لینک‌های محلی**

بسیاری از لینک‌ها به صفحات **هم‌میزبان** اشاره می‌کنند. در لیست مرتب‌شده URLها، این یعنی شماره‌های مقصد نزدیک به هم هستند و گپ‌های کوچکتری تولید می‌شود، که فشرده‌سازی را مؤثرتر می‌کند.

🏗️ معماری عملی: از تئوری تا پیاده‌سازی

این تکنیک‌ها چگونه در عمل ترکیب می‌شوند؟

- مرتب‌سازی و شماره‌گذاری:** ابتدا تمام URLها را به‌صورت لکسیکوگرافیک مرتب کرده و به هر کدام یک شماره منحصر‌به‌فرد (ID عددی) می‌دهیم. این کار پایه و اساس سیستم ماست.
- ایجاد جدول مجاورت:** جدولی ایجاد می‌کنیم که هر ردیف آن شامل لیست مرتب‌شده‌ای از ID صفحاتی است که به آن صفحه لینک داده‌اند (in-links) یا از آن لینک گرفته‌اند (out-links).
- کدگذاری هوشمند:** از بالا به پایین جدول را پیمایش می‌کنیم و هر ردیف را بر اساس ۷ ردیف قبلی فشرده می‌کنیم. چرا ۷؟ چون صفحات یک وب‌سایت در این مرتب‌سازی کنار هم قرار می‌گیرند و ۷ ردیف قبلی به احتمال زیاد به همان سایت تعلق دارند. این محدودیت باعث می‌شود جستجو برای ردیف نمونه سریع باشد (فقط ۳ بیت برای ذخیره Offset نیاز است).
- بازسازی در زمان پرس‌وجو:** هنگام پاسخ به یک پرس‌وجو، ابتدا ID صفحه مورد نظر را پیدا کرده، سپس ردیف فشرده‌شده آن را بازسازی می‌کنیم. اگر ردیف به یک ردیف نمونه اشاره کند، آن ردیف نمونه نیز بازسازی می‌شود.

این تکنیک‌ها به‌طور ترکیبی، مصرف حافظه را از **۶۴ بیت به ازای هر لینک** در روش ساده، به تنها **۳ بیت به ازای هر لینک** کاهش می‌دهند - یک پیشرفت چشمگیر!

⚖️ تعادل بین فشرده‌سازی و سرعت

یک چالش مهم در این سیستم، پیدا کردن تعادل مناسب است. اگر آستانه شباهت برای انتخاب ردیف نمونه خیلی بالا باشد، از این تکنیک به ندرت استفاده می‌شود و فشرده‌سازی ضعیف می‌شود. اگر آستانه خیلی پایین باشد، ردیف‌های زیادی به ردیف‌های نمونه اشاره می‌کنند و بازسازی یک ردیف ساده ممکن است به چندین سطح از ارجاعات نیاز داشته باشد که سرعت را کاهش می‌دهد.

این تعادل دقیق یکی از رازهای موفقیت موتورهای جستجوی مدرن است که به آنها اجازه می‌دهد کل وب را در حافظه مدیریت کرده و به پرس‌و‌جوهای پیچیده در کسری از ثانیه پاسخ دهند.

In [2]: شبیه‌سازی فشرده‌سازی گراف وب #

```
import random
from typing import List, Tuple, Optional, Dict

class WebGraphCompressor:
    """
    شبیه‌سازی فشرده‌سازی گراف وب با سه تکنیک:
    1. کدگذاری گپ (Gap Encoding)
    2. کدگذاری نسبت به ردیف نمونه (Prototype-based Encoding)
    """

    def __init__(self, similarity_threshold=0.6):
        self.similarity_threshold = similarity_threshold
        self.compressed_rows = [] # لیست ردیف‌های فشرده‌شده
        self.url_to_id = {} # عددی ID به URL نگاشت
        self.id_to_url = [] # ID به URL نگاشت
        self.next_id = 0
```

```
def register_url(self, url: str) -> int:
    """عددی منحصریبهفرد آن ID و بازگرداندن URL ثبت یک"""
    if url not in self.url_to_id:
        self.url_to_id[url] = self.next_id
        self.id_to_url.append(url)
        self.next_id += 1
    return self.url_to_id[url]

def encode_gap(self, sorted_ids: List[int]) -> List[int]:
    """کدگذاری گپ: تبدیل لیست شماره‌های مرتب به تفاضل‌های متوالی"""
    if not sorted_ids:
        return []
    gaps = [sorted_ids[0]]
    for i in range(1, len(sorted_ids)):
        gaps.append(sorted_ids[i] - sorted_ids[i-1])
    return gaps

def decode_gap(self, gaps: List[int]) -> List[int]:
    """بازگردانی لیست اصلی از روی گپ‌ها"""
    if not gaps:
        return []
    ids = [gaps[0]]
    for i in range(1, len(gaps)):
        ids.append(ids[-1] + gaps[i])
    return ids

def calculate_similarity(self, row1: List[int], row2: List[int]) -> float:
    """محاسبه شباهت بین دو ردیف بر اساس اشتراک مجموعه‌ای"""
    if not row1 or not row2:
        return 1.0
    set1, set2 = set(row1), set(row2)
    intersection = len(set1 & set2)
    union = len(set1 | set2)
    return intersection / union if union > 0 else 0.0

def find_best_prototype(self, current_row: List[int], window_size: int = 7) -> Optional[Tuple[int, Dict[str, List[int]]]]:
    """
    ردیف آخر 'window_size' یافتن بهترین ردیف نمونه از بین
    .اگر شباهت کافی وجود نداشته باشد None یا (offset, differences) بازگشت
    """
    if not self.compressed_rows:
        return None

    num_rows = len(self.compressed_rows)
    start_idx = max(0, num_rows - window_size)

    best_similarity = 0.0
    best_offset = 0
    best_diff = {"remove": [], "add": []}

    for i in range(num_rows - 1, start_idx - 1, -1):
        # بازسازی ردیف نمونه (همیشه ذخیره‌شده به صورت گپ)
        prototype_gaps = self.compressed_rows[i]["data"]
        prototype_row = self.decode_gap(prototype_gaps)

        similarity = self.calculate_similarity(current_row, prototype_row)

        if similarity > best_similarity:
            best_similarity = similarity
            best_offset = num_rows - i # فاصله از ردیف فعلی

            # محاسبه تفاوت‌ها: عناصری که باید حذف/افزوده شوند
            remove = list(set(prototype_row) - set(current_row))
            add = list(set(current_row) - set(prototype_row))
            best_diff = {"remove": sorted(remove), "add": sorted(add)}

    if best_similarity >= self.similarity_threshold:
        return (best_offset, best_diff)
    return None

def add_adjacency_row(self, out_links: List[str]) -> None:
    """
    افزودن یک ردیف مجاورت (لیست لینک‌های خروجی یک صفحه) به گراف فشرده‌شده
    """
    # های عددی ID ها به URL تبدیل
    link_ids = [self.register_url(url) for url in out_links]
    link_ids.sort() # لیست باید مرتب باشد

    # تلاش برای یافتن یک ردیف نمونه مناسب
    prototype_info = self.find_best_prototype(link_ids)

    # اگر ردیف نمونه مناسبی پیدا شد، آن را فشرده می‌کنیم
    if prototype_info is not None:
        offset, diff = prototype_info
        self.compressed_rows.append({
            "type": "prototype",
            "offset": offset,
            "diff": diff
        })
    else:
        # اگر ردیف نمونه مناسبی نبود، از کدگذاری گپ استفاده می‌کنیم
        gaps = self.encode_gap(link_ids)
        self.compressed_rows.append({
            "type": "gap",
            "data": gaps
        })

def reconstruct_row(self, row_index: int) -> List[int]:
    """
    بازسازی لیست لینک‌های یک ردیف از فرم فشرده‌شده
    """
    if row_index >= len(self.compressed_rows) or row_index < 0:
```

```
        return []

    row = self.compressed_rows[row_index]
    if row["type"] == "gap":
        return self.decode_gap(row["data"])
    elif row["type"] == "prototype":
        # بازسازی ردیف نمونه اول
        prototype_index = row_index - row["offset"]
        prototype_row = self.reconstruct_row(prototype_index)

        # اعمال تغییرات
        result_set = set(prototype_row)
        for item in row["diff"]["remove"]:
            result_set.discard(item)
        for item in row["diff"]["add"]:
            result_set.add(item)
        return sorted(list(result_set))
    return []

def get_original_links(self, row_index: int) -> List[str]:
    """برای یک ردیف URL بازگرداندن لینک‌های اصلی به صورت"""
    ids = self.reconstruct_row(row_index)
    return [self.id_to_url[i] for i in ids if i < len(self.id_to_url)]

# --- شبیه‌سازی سرور همبستگی ---
class ConnectivityServer:
    """
    شبیه‌سازی یک سرور همبستگی که پرس‌وجوهای را مدیریت می‌کند.
    """

    def __init__(self, compressor: WebGraphCompressor):
        self.compressor = compressor
        self.inverted_index = {} # نگاشت لینک -> لیست اسناد

    def add_document(self, doc_id: int, links: List[str]):
        """افزودن یک سند جدید به ایندکس معکوس"""
        for link in links:
            # --- اصلاح: استفاده از متدهای دیکشنری ---
            if link not in self.inverted_index:
                self.inverted_index[link] = set()
            self.inverted_index[link].add(doc_id)

    def handle_query(self, query_links: List[str]) -> List[int]:
        """
        پاسخ به یک پرس‌وجو: لیست اسنادهایی که شامل تمام لینک‌های پرس‌وجو هستند.
        """
        result_sets = [set(self.inverted_index.get(link, set())) for link in query_links]

        if not result_sets:
            return []

        # پیدا کردن اشتراک بین تمام مجموعه‌ها
        common_docs = result_sets[0]
        for i in range(1, len(result_sets)):
            common_docs.intersection_update(result_sets[i])

        # تبدیل مجموعه اسناد به لیست مرتب
        return sorted(list(common_docs))

# --- سناریوی اصلی ---
def main():
    """
    سناریوی اصلی: ایجاد گراف، فشردسازی، و پاسخ به پرس‌وجوها.
    """

    # تعریف صفحات و لینک‌ها (مطابق شکل ۲۰.۵)
    pages = {
        1: {"id": 1, "links": [2, 3, 4, 8, 16, 32, 64]},
        2: {"id": 2, "links": [1, 3, 5, 6, 7, 8, 9]},
        3: {"id": 3, "links": [1, 4, 5, 6, 7, 8]},
        4: {"id": 4, "links": [1, 2, 3, 4, 5, 6]},
        5: {"id": 5, "links": [1, 2, 3, 4, 5]},
        6: {"id": 6, "links": [1, 2, 3, 4]}
    }

    # ایجاد فشرده‌ساز و سرور
    compressor = WebGraphCompressor()
    server = ConnectivityServer(compressor)

    # افزودن اسناد به سرور
    for doc_id, page_data in pages.items():
        server.add_document(doc_id, page_data["links"])

    # سناریوی اصلی: پاسخ به پرس‌وجوها
    queries = [
        [2, 4], # پرس‌وجو: صفحاتی که به صفحه ۲ لینک دارند
        [1, 5, 6], # پرس‌وجو: صفحاتی که به صفحات ۱، ۳ و ۵ لینک دارند
        [3, 4, 5], # پرس‌وجو: صفحاتی که به صفحات ۲، ۳ و ۴ لینک دارند
    ]

    print("شبیه‌سازی سرور همبستگی 🔄")
    print("-" * 40)

    for i, query in enumerate(queries):
        print(f"پرس‌وجو {i+1}: {query} -> {server.handle_query(query)}")
        result = server.handle_query(query)
        print(f"پاسخ: {result}")

if __name__ == "__main__":
    main()
```

("بسیار ناکارآمد است و در مقیاس‌های بزرگ کارایی ندارد (Query Processing) نکته مهم: تکنیک فشردسازی گراف، هرچقدر قدرتمند، برای کاهش حجم داده‌ها عالی است، اما در عملکرد جستجو")

پرس‌وجو 1: [4, 2] < - [6, 5, 4, 1]
پاسخ: [6, 5, 4, 1]

پرس‌وجو 2: [6, 5, 1] < - [4, 3, 2]
پاسخ: [4, 3, 2]

پرس‌وجو 3: [5, 4, 3] < - [5, 4]
پاسخ: [5, 4]

بسیار ناکارآمد است و در مقیاس‌های بزرگ کارایی ندارد (Query Processing) نکته مهم: تکنیک فشردسازی گراف، هرچقدر قدرتمند، برای کاهش حجم داده‌ها عالی است، اما در عملکرد جستجو

جمع‌بندی فصل ۲۰: خزنده‌ها و ایندکس‌های وب

در این فصل:

- فرآیند **خزنده‌برداری وب** یا **به اختصار خزش وب** (Web crawling) را به‌عنوان گامی حیاتی برای جمع‌آوری صفحات وب و ساخت ایندکس معرفی کردیم.
- ویژگی‌های ضروری خزنده‌ها را بررسی کردیم: – **ربات‌گیر** (Robustness): مقاومت در برابر دام‌های خزنده (spider traps)، – **ادب** (Politeness): رعایت محدودیت‌های سرورها در سرعت درخواست.
- معماری یک خزنده واقعی را تحلیل کردیم: – **مرز** (URL frontier) برای مدیریت صف درخواست‌ها، – **ماژول‌های DNS، دریافت** (fetch)، **تجزیه** (parse) و **حذف تکرار** (duplicate elimination).
- چالش‌های خزنده‌های توزیع‌شده را پوشش دادیم: – پارتیشن‌بندی میزبان‌ها بین گره‌ها، – مدیریت سراسری مجموعه اثرانگشت‌ها برای شناسایی صفحات تکراری.
- راهکارهای هوشمند برای **مرز URL** را معرفی کردیم: – صف‌های جلو برای **اولویت‌بندی** (کیفیت و تازگی)، – صف‌های عقب برای رعایت **ادب** (فاصله زمانی بین درخواست‌ها به یک میزبان).
- دو روش اصلی **توزیع ایندکس** را مقایسه کردیم: – پارتیشن‌بندی بر اساس **اصطلاحات** (term partitioning) – چالش‌برانگیز برای پرسمان‌های چندکلمه‌ای، – پارتیشن‌بندی بر اساس **اسناد** (document partitioning) – رایج‌تر و ساده‌تر برای پیاده‌سازی.
- نیاز به **سرورهای اتصال** (Connectivity servers) را تبیین کردیم: ذخیره ساختار گراف وب برای پشتیبانی از تحلیل پیوندها (فصل ۲۱) با استفاده از فشردسازی هوشمند لیست‌های مجاورت.

این فصل پایه‌های فنی ساخت یک موتور جست‌وجوی وب را فراهم می‌کند. در فصل بعدی، با مفاهیمی مانند:

- تحلیل ساختار گراف وب برای رتبه‌بندی (PageRank و HITS)
- الگوریتم‌های تشخیص و مبارزه با اسپم پیوندی (Link spam)
- استفاده از متن لنگر (Anchor text) به‌عنوان سیگنال رتبه‌بندی

آشنا خواهیم شد – که همگی بر همین پایه داده‌های جمع‌آوری‌شده توسط خزنده‌ها و ایندکس‌های توزیع‌شده بنا شده‌اند.

End of Chapter Exercises

مسئله 20.1: ویژگی‌های الزامی خزشگر (Mandatory Crawler Features)

در معماری یک خزشگر وب، دو ویژگی **Robustness** (مقاومت) و **Politeness** (ادب) به عنوان ویژگی‌های الزامی **Must provide** معرفی شده‌اند. توضیح دهید چرا عدم رعایت هر یک از این دو، می‌تواند کل عملیات خزش را با شکست مواجه کند؟

مسئله 20.2: ویژگی‌های ترجیحی خزشگر (Preferred Crawler Features)

یک خزشگر **Distributed** و **Scalable** چه مزیت‌هایی نسبت به یک خزشگر متمرکز دارد؟ آیا یک خزشگر می‌تواند بدون این ویژگی‌ها نیز در مقیاس وب عمل کند؟

مسئله 20.3: اجزای اصلی معماری (Core Architecture Components)

وظیفه ماژول **Duplicate Detection** (تشخیص محتوای تکراری) در معماری پایه خزشگر چیست و چگونه از اتلاف منابع سیستم جلوگیری می‌کند؟

مسئله 20.4: گلوگاه وضوح نام دامنه (DNS Resolution Bottleneck)

چرا عملیات ترجمه نام دامنه به **IP** (**DNS Resolution**) معمولاً به عنوان یکی از کندترین بخش‌های فرآیند خزش شناخته می‌شود؟

مسئله 20.5: ساختار صف آدرس‌ها (URL Frontier Structure)

صف آدرس‌ها (**URL Frontier**) در یک خزشگر چه تفاوتی با یک صف ساده **FIFO** (**First-In-First-Out**) دارد و چرا نمی‌توان از یک صف ساده استفاده کرد؟

مسئله 20.6: مفهوم توزیع شاخص (Index Distribution Concept)

در سیستم‌های مقیاس‌پذیر، توزیع شاخص‌ها به دو روش اصلی **Term-partitioned** و **Document-partitioned** انجام می‌شود. تفاوت بنیادین این دو روش در نحوه ذخیره‌سازی داده‌ها چیست؟

مسئله 20.7: هدف سرورهای اتصال (Purpose of Connectivity Servers)

سرورهای اتصال (**Connectivity Servers**) چه نقشی در موتورهای جستجو ایفا می‌کنند و چرا داده‌های آن‌ها برای الگوریتم‌هایی مانند **PageRank** حیاتی است؟

مسئله 20.8: نرمال‌سازی آدرس‌های وب (URL Normalization)

چرا پیش از قرار دادن یک آدرس جدید در **URL Frontier**، عملیات نرمال‌سازی (مانند حذف حروف بزرگ، حذف پورت‌های پیش‌فرض و پارامترهای اضافی) انجام می‌شود؟

مسئله 20.9: استخراج پیوندها (Link Extraction)

هنگامی که خزشگر یک صفحه **HTML** را واکشی می‌کند، ماژول استخراج پیوند چگونه لینک‌های نسبی (**Relative URLs**) را به لینک‌های مطلق (**Absolute URLs**) تبدیل می‌کند؟

مسئله 20.10: فایل روبات‌ها (Robots.txt Protocol)

نقش فایل `robots.txt` در مکانیزم خزش چیست؟ اگر سایتی این فایل را نداشته باشد، خزشگر چه فرضی در مورد آن سایت می‌کند؟

مسئله 20.11: سیاست‌های واکشی مجدد در سایت‌های خبری (Re-fetch Policies in News Sites)

شما در حال طراحی خزشگری برای ایندکس کردن خبرگزاری‌ها هستید. با توجه به اینکه صفحه اصلی خبرگزاری هر دقیقه به‌روز می‌شود اما آرشیو اخبار سال گذشته تغییری نمی‌کند، مکانیزم `URL Frontier` چگونه باید نرخ تغییرات (`Change Rate`) را برای زمان‌بندی واکشی مجدد (`Re-fetch`) این دو نوع صفحه مدیریت کند؟

مسئله 20.12: مدیریت کش و همزمانی در DNS (DNS Cache & Concurrency Management)

اگر خزشگر شما دارای 500 نخ کارگر (`Worker Thread`) باشد و همه آن‌ها همزمان به مازول `DNS` درخواست دهند، استفاده از یک کش استاندارد (`Standard Cache`) باعث قفل شدن نخ‌ها (`Thread Contention`) می‌شود. چه معماری برای مازول کش `DNS` پیشنهاد می‌دهید که همزمان سرعت بالا و سازگاری با نخ‌ها را تضمین کند؟

مسئله 20.13: فیلتر کردن محتوای تکراری متنی (Textual Duplicate Filtering)

دو صفحه وب دارای محتوای متنی دقیقاً یکسان هستند اما ساختار `HTML`، تبلیغات و هدرهای آن‌ها متفاوت است. اگر صرفاً از تابع هش (مانند `MD5`) روی کل فایل `HTML` استفاده کنیم، این دو صفحه متفاوت شناخته می‌شوند. الگوریتم `Document Fingerprinting` (مانند `Simhash` یا `Shingling`) چگونه این چالش را حل می‌کند؟

مسئله 20.14: مقایسه پارتیشن‌بندی در کوئری‌های پیچیده (Partitioning in Complex Queries)

کاربری کوئری "دانشگاه صنعتی" را در موتوری که 100 میلیون سند دارد جستجو می‌کند. فرآیند مسیریابی این کوئری و تجمیع نتایج (`Result Aggregation`) را یک‌بار در معماری `Document-partitioned` و یک‌بار در معماری `Term-partitioned` تحلیل کنید. کدام معماری ترافیک شبکه (`Network Traffic`) کمتری ایجاد می‌کند؟

مسئله 20.15: استراتژی‌های صف جلو و عقب در مرز آدرس‌ها (Front and Back Queue Strategies)

صف‌های `Front` برای اولویت‌بندی و صف‌های `Back` برای رعایت `Politeness` طراحی شده‌اند. نشان دهید اگر یک وب‌سایت خاص تعداد بسیار زیادی لینک با اولویت بالا در صف `Front` داشته باشد، چگونه صف‌های `Back` از بمباران شدن سرور آن وب‌سایت جلوگیری می‌کنند؟

مسئله 20.16: ماتریس مجاورت در مقیاس وب (Adjacency Matrix at Web Scale)

فرض کنید وب دارای 10^{10} صفحه است. اگر بخواهیم گراف وب را با یک ماتریس مجاورت (`Adjacency Matrix`) در حافظه ذخیره کنیم، به چه میزان حافظه نیاز داریم؟ چرا در `Connectivity Servers` از لیست مجاورت فشرده‌شده به جای ماتریس استفاده می‌شود؟

مسئله 20.17: تشخیص حلقه‌ها در گراف وب (Cycle Detection in Web Graph)

سیستم در حال خزش دایرکتوری‌هایی است که به صورت چرخشی به یکدیگر لینک داده‌اند (مانند $A \rightarrow B \rightarrow C \rightarrow A$).

خزشگر چگونه با استفاده از تاریخچه **URL Seen** ماژول از افتادن در این حلقه بی‌نهایت جلوگیری می‌کند؟

مسئله 20.18: توزیع بار در سرورهای شاخص (Load Balancing in Index Servers)

در سیستم **Term-partitioned**، کلماتی مانند "است" یا "در" (**Stop words**) بسیار بیشتر از کلمات تخصصی جستجو می‌شوند. این امر باعث می‌شود گره‌هایی که مسئول این کلمات هستند دچار سربار پردازشی (**Overload**) شوند. چه راه‌حلی برای توازن بار در این معماری پیشنهاد می‌دهید؟

مسئله 20.19: برخورد با صفحات پویا و پارامترهای تکراری (Handling Dynamic Pages)

یک وب‌سایت فروشگاهی دارای قابلیت مرتب‌سازی (**Sort**) کالاها بر اساس قیمت، نام و تاریخ است که هر کدام پارامتر خاصی به **URL** اضافه می‌کنند، اما محتوای اصلی صفحه تغییری نمی‌کند. قانونی بنویسید که ماژول نرمال‌سازی **URL** بتواند این پارامترهای بی‌اثر (مانند **sort=price**) را شناسایی و حذف کند.

مسئله 20.20: تخصیص زمان در Min-Heap (Time Allocation in Min-Heap)

در مکانیزم **Politeness**، از یک **Min-Heap** برای تعیین زمان بیدار شدن صف‌های **Back** استفاده می‌شود. اگر سرور A محدودیت 2 ثانیه تأخیر بین درخواست‌ها و سرور B محدودیت 5 ثانیه تأخیر داشته باشد، گره‌های مربوط به این دو سرور پس از هر واکنشی دقیقاً با چه مقداری (طبق فرمول $T_{next} = T_{current} + \Delta T$) در **Min-Heap** به‌روزرسانی و درج مجدد می‌شوند؟

مسئله 20.21: پیاده‌سازی تشخیص تله‌های عنکبوتی (Implementing Spider Trap Detection)

دانشجویان کدی توسعه داده‌اند که در یک تله عنکبوتی (**Spider Trap**) ناشی از تقویم‌های بی‌نهایت پویا گرفتار شده است. الگوریتمی نظارتی (**Watchdog Algorithm**) طراحی کنید که با تحلیل طول آدرس (**URL Length**) و تراکم تکرار الگوهای مسیر، این تله‌ها را شناسایی کند. الگوریتم باید پارامتر کنترلی **Threshold** را بررسی کرده و در صورت نقض شرط $Length(URL) > L_{max}$ یا مشاهده الگوهای تکراری غیرمتعارف، عملیات خزش را متوقف کند. این منطق را جهت اتصال به سیستم ارزیاب خودکار (**Autograder**) پیاده‌سازی نمایید.

مسئله 20.22: طراحی الگوریتم صف‌های اولویت‌دار دوگانه (Dual Priority Queues Algorithm Design)

ساختمان داده‌ای طراحی کنید که همزمان از صف‌های **Front** (مبتنی بر امتیاز اولویت صفحه) و صف‌های **Back** (مبتنی بر ساختمان داده‌ای طراحی کنید که همزمان از صف‌های **Front** و **Min-Heap** زمان‌بندی پشتیبانی کند. توابع **insert(url, priority)** و **get_next()** را با شبه‌کد بنویسید. منطق تخصیص لینک‌ها از **Front** به **Back** را جهت رعایت شرط $T_{next} \geq T_{current} + \Delta T$ به طور دقیق پیاده‌سازی کنید.

مسئله 20.23: سناریوی خرابی گره در پارتیشن‌بندی ترم (Node Failure Scenario in Term Partitioning)

فرض کنید در زمان اجرای عملیات جستجو، گره مسئول کلمه "الگوریتم" در معماری **Term-partitioned** از کار می‌افتد. بدون استفاده از تکنیک تکثیر (**Replication**)، کاربر چه پاسخی دریافت می‌کند؟ یک مکانیزم **Fallback** طراحی کنید که در صورت بروز این خطا، سیستم به جای نمایش خطا، نتایج جزئی (**Partial Results**) را با دقت و سرعت مناسب بازگرداند.

مسئله 20.24: برنامه‌نویسی رمزگذاری دلتا در لیست مجاورت (Delta Encoding Programming) ★

در **Connectivity Servers**، لیست ارجاعات یک صفحه به صورت $[1020, 1025, 1035, 1050, 1055]$ است. الف) برنامه‌ای به زبان **Python** بنویسید که یک لیست مرتب‌شده از شناسه‌ها را دریافت کرده و آن را به فرمت رمزگذاری دلتا (**Delta Encoded**) تبدیل کند. خروجی باید برای آرایه بالا به شکل $[1020, 5, 10, 15, 5]$ باشد. ب) با استفاده از رابطه $\Delta_i = ID_i - ID_{i-1}$ ، توضیح دهید چرا اعداد دلتا برای فشرده‌سازی با کدهای طول متغیر (**Variable-length**) مناسب‌تر از شناسه‌های اصلی هستند.

مسئله 20.25: رمزگذاری ارجاعی با بلوک‌های همسایه (Reference Encoding with Neighbor Blocks) ★

صفحات وب با آدرس‌های مشابه معمولاً لینک‌های خروجی مشترک زیادی دارند. الگوریتم **Reference Encoding** را به گونه‌ای گسترش دهید که لیست مجاورت صفحه 100 را با استفاده از لیست مجاورت صفحه 99 فشرده کند. با فرض:

$$L_{99} = [8, 15, 22, 35, 40]$$
$$L_{100} = [8, 15, 28, 35, 45]$$

خروجی فشرده‌سازی دقیقاً شامل چه بیت‌های کنترلی (Flags) و اعدادی خواهد بود؟ توضیح دهید که چگونه پرچم **Copy Flag** در این متد، عملیات بازیابی (Decoding) را تسهیل می‌کند.

مسئله 20.26: مدیریت بن‌بست در استخراج موازی (Deadlock Management in Parallel Extraction) ★

در یک خزنگر کاملاً توزیع‌شده (**Distributed Crawler**)، دو نخ کارگر همزمان در حال پردازش صفحاتی هستند که به یکدیگر لینک داده‌اند. هر دو نخ تلاش می‌کنند جدول **URL Seen** (که دارای قفل‌های سطح رکورد است) را آپدیت کنند. چگونه با استفاده از تکنیک‌های هشینگ و صف‌بندی پیام‌ها (**Message Queuing**)، معماری را به گونه‌ای تغییر می‌دهید که نیاز به قفل‌گذاری (**Locking**) روی جدول مرکزی حذف شده و از وقوع بن‌بست (**Deadlock**) جلوگیری شود؟

مسئله 20.27: بهینه‌سازی کوئری‌های چندکلمه‌ای توزیع‌شده (Distributed Multi-term Query Optimization) ★

در معماری **Document-partitioned**، کوئری "پردازش سیگنال پیشرفته" به 50 گره ارسال می‌شود. هر گره 1000 نتیجه برتر خود را برمی‌گرداند (مجموعاً 50,000 سند برای تجمیع). الگوریتمی دو مرحله‌ای (مشابه تکنیک‌های **MapReduce**) طراحی کنید که حجم داده‌های انتقال‌یافته روی شبکه را در فاز تجمیع (**Aggregation Phase**) حداقل 50% کاهش دهد، بدون آنکه دقت 10 نتیجه برتر (**Top 10**) افت کند. نحوه استفاده از آستانه‌گذاری (**Thresholding**) را در طراحی خود شرح دهید.

مسئله 20.28: سناریوی محاسبه پیج‌رنک در حافظه محدود (PageRank Scenario in Limited Memory) ★

شما باید الگوریتم **PageRank** را روی گرافی با 10^9 گره محاسبه کنید. سرور شما تنها 32 گیگابایت رم دارد که برای بارگذاری کل گراف وب و بردارهای رتبه کافی نیست. با استفاده از **Connectivity Servers** و تکنیک **Block-Stripe Update**، فرآیند محاسبه روی دیسک را به گونه‌ای طراحی کنید که عملیات خواندن/نوشتن کاملاً به صورت ترتیبی (**Sequential I/O**) انجام شود و گلوگاه دیسک برطرف گردد. الگوریتم خود را با تمرکز بر چگونگی قطعه‌بندی ماتریس انتقال (**Transition Matrix**) شرح دهید.

مسئله 20.29: شبیه‌سازی صف‌های ادب با لاگ‌های خروجی (Simulating Politeness Queues) ★

برای بررسی صحت عملکرد الگوریتم زمان‌بندی خزشگر، قطعه کدی طراحی کنید که رویدادهای خروجی `URL` از `URL Frontier` را شبیه‌سازی کند. خروجی باید یک فایل لاگ شامل `[Timestamp, Host_IP, URL]` باشد. سپس اسکریپتی برای ارزیابی (Autograder) طراحی کنید که شرط $T_{req_i} - T_{req_i-1} \geq \Delta T_{politeness}$ را برای هر هاست بررسی نماید.

مسئله 20.30: توزیع مقیاس‌پذیر در برابر حملات تزریق لینک (Scalable Distribution vs. Link Injection Attacks) ★

شبکه‌ای از سایت‌های اسپم با تولید میلیون‌ها لینک مصنوعی به یکدیگر در تلاشند تا اولویت خزش خود را در `URL Frontier` بالا ببرند و منابع خزشگر را هدر دهند. چگونه با ادغام مفهوم `TrustRank` در زمان محاسبه اولویت درون `URL Frontier`، این حملات را در همان مرحله ابتدایی خنثی می‌کنید؟ فرمول محاسبه اولویت (`Priority Score`) را با دخالت دادن امتیاز اعتماد (اعتبار) بازنویسی کنید.

مسئله 20.31: معماری تکثیر متقاطع با همگام‌سازی ناهمگام (Cross-Replication with Asynchronous Sync) ★

برای اطمینان از دسترسی‌پذیری بالا (`High Availability`) در کلاستر موتور جستجو، نیاز به تکثیر داده‌ها است. حال شما یک معماری تکثیر `Leader-Follower` برای سرورهای شاخص طراحی کنید که همگام‌سازی (`Synchronization`) در آن به صورت `Asynchronous` انجام شود. در این معماری، اگر گره `Leader` در میان فرآیند ادغام شاخص (`Index Merge`) بسوزد، داده‌های در حال پردازش چگونه بازیابی می‌شوند؟

مسئله 20.32: طراحی جامع خزشگر متمرکز با ارزیابی خودکار (Comprehensive Focused Crawler Design) ★

یک خزشگر متمرکز (`Focused Crawler`) برای استخراج مستندات و مقالات پژوهشی با فرمت `PDF` طراحی کنید. سیستم شما باید دارای سه ماژول اصلی باشد:

- طبقه‌بند موضوعی (`Topic Classifier`) پیش از واکنش برای تخمین ارتباط `URL` با مقالات پژوهشی.
 - استخراج‌کننده متادیتا از هدر `HTTP` بدون دانلود کامل فایل در صورت نامرتب بودن.
 - تولیدکننده فایل گزارش `JSON` که شامل نرخ موفقیت (`Hit Rate`) و حجم پهنای باند مصرفی باشد.
- معماری دقیق جریان داده (`Data Flow`) بین این سه ماژول را برای ارائه در جلسه دفاعیه ترسیم و تحلیل کنید.

پاسخنامه مسئله 20.1: ویژگی‌های الزامی خزشگر (Mandatory Crawler Features)

مقدمه و مرور مفاهیم پایه‌ای

در معماری موتورهای جستجو، خزشگر وب (`Web Crawler`) نرم‌افزاری است که وظیفه دارد گراف عظیم وب را پیمایش کرده و محتوای صفحات را برای پردازش و نمایه‌سازی (`Indexing`) جمع‌آوری کند. محیط وب، 1 محیط توزیع‌شده، به‌شدت پویا، بدون مدیریت مرکزی و پر از خطاهای ساختاری است. به همین دلیل، طراحی خزشگر با طراحی 1 نرم‌افزار معمولی شبکه تفاوت اساسی دارد.

در ادبیات بازیابی اطلاعات، **2** ویژگی **مقاومت (Robustness)** و **ادب (Politeness)** صرفاً ویژگی‌های «بهبوددهنده» نیستند، بلکه پیش‌شرط‌های حیاتی **(Mandatory)** برای آغاز عملیات خزش محسوب می‌شوند. در ادامه، مکانیزم اثر و چرایی شکست سیستم در صورت عدم رعایت هر **1** را به صورت گام‌به‌گام تحلیل می‌کنیم.

بخش 1) مقاومت (Robustness)

تعریف: مقاومت به توانایی خزشگر در مواجهه با صفحات مخرب، خطاهای ساختاری، سرورهای مختل‌شده و تله‌های عنکبوتی (Spider Traps) بدون از کار افتادن سیستم گفته می‌شود.

چرا عدم وجود مقاومت منجر به شکست می‌شود؟

وب پر از ساختارهای گمراه‌کننده است که برخی به صورت عمدی (برای فریب موتورهای جستجو) و بسیاری به صورت غیرعمدی (به دلیل معماری سیستم‌های مدیریت محتوا) ایجاد شده‌اند. فرآیند شکست خزشگر فاقد مقاومت، معمولاً طی مراحل زیر رخ می‌دهد:

گام اجرایی	شرح رویداد در سیستم فاقد مقاومت
گام 1 مواجهه با تله (Spider Trap)	خزشگر وارد وب‌سایتی می‌شود که دارای ساختار پیوندهای بی‌نهایت است (مثلاً 1 تقویم پویای آنلاین که پیوند «ماه بعد» تا بی‌نهایت در آن تولید می‌شود، یا دایرکتوری‌هایی که پیوندهای چرخشی به یکدیگر دارند).
گام 2 تولید آدرس‌های کاذب (Fake URLs)	سیستم مدیریت محتوای آن سایت، در پاسخ به هر درخواست خزشگر، 1 URL جدید اما بی‌ارزش تولید می‌کند (مانند <code>example.com/calendar?year=3045</code>).
گام 3 انفجار صف خزش (Frontier Explosion)	خزشگر این پیوندهای جدید را استخراج کرده و به صف آدرس‌های در انتظار خزش (Crawl Frontier) اضافه می‌کند. از آنجایی که این روند پایانی ندارد، صف خزش به سرعت از میلیون‌ها URL بی‌ارزش پر می‌شود.
گام 4 شکست نهایی (System Crash)	2 اتفاق رخ می‌دهد؛ یا حافظه (Memory) و فضای ذخیره‌سازی سرور خزشگر به دلیل حجم عظیم URL ها پر شده و سیستم اصطلاحاً Crash می‌کند، و یا خزشگر تمام پهنای باند و منابع پردازشی خود را در 1 سایت بی‌نهایت هدر داده و از خزش سایر بخش‌های وب باز می‌ماند (Starvation).

نتیجه‌گیری بخش اول:

ویژگی **Robustness** خزشگر را در برابر محیط غیرقابل پیش‌بینی وب **محافظت** می‌کند. بدون آن، خزشگر در اولین تله‌های ساختاری به دام افتاده و فلج می‌شود.

بخش 2) ادب (Politeness)

تعریف: ادب به مجموعه‌ای از سیاست‌ها گفته می‌شود که تضمین می‌کند خزشگر به حقوق سرورهای میزبان احترام گذاشته و بار ترافیکی نامعقولی به آن‌ها تحمیل نکند. این اصل از طریق رعایت فایل **robots.txt** و اعمال تاخیر زمانی (**Crawl-delay**) بین درخواست‌ها پیاده‌سازی می‌شود.

چرا عدم رعایت ادب منجر به شکست می‌شود؟

برخلاف کاربران انسانی که پس از باز کردن 1 صفحه، مدتی را صرف خواندن آن می‌کنند، 1 خزشگر می‌تواند در هر ثانیه هزاران درخواست HTTP ارسال کند. اگر خزشگر فاقد مکانیزم‌های ادب باشد، فرآیند شکست به این شکل رخ می‌دهد:

گام اجرایی	شرح رویداد در سیستم فاقد مکانیزم ادب
گام 1 کشف خوشه پیوندها (Cluster Discovery)	خزشگر صدها هزار پیوند مربوط به 1 دامنه خاص (مثلاً target-server.com) را در صف خود پیدا می‌کند.
گام 2 درخواست‌های همزمان (Parallel Requests)	از آنجا که خزشگر برای سرعت بالا طراحی شده است، بدون در نظر گرفتن تاخیر زمانی، هزاران درخواست موازی برای واکنشی این صفحات به سرور مقصد ارسال می‌کند.
گام 3 حمله منع سرویس (DoS Effect)	سرور میزبان توانایی پردازش این حجم از درخواست همزمان را ندارد. منابع پردازشی (CPU) و حافظه (RAM) سرور به اشغال کامل درآمده و سرور از دسترس کاربران واقعی خارج می‌شود (شبیه به 1 حمله سایبری Denial of Service).
گام 4 شکست نهایی (IP Ban)	فایروال‌ها و مدیران شبکه سرور مقصد، بلافاصله متوجه این ترافیک غیرعادی شده و IP سرورهای خزشگر شما را در لیست سیاه (Blacklist) قرار می‌دهند. با مسدود شدن IP، خزشگر دیگر قادر به واکنشی هیچ داده‌ای از آن سایت (و احتمالاً سایت‌های هم‌شبکه آن) نخواهد بود. همچنین این موضوع می‌تواند تبعات قانونی و حقوقی برای تیم توسعه‌دهنده به همراه داشته باشد.

نتیجه‌گیری بخش دوم:

ویژگی **Politeness** سرورهای وب را در برابر قدرت پردازشی بالای خزشگر **محافظت** می‌کند. بدون آن، خزشگر به‌عنوان 1 عامل مخرب شبکه شناخته شده و توسط سرورها مسدود می‌شود که نتیجه آن جمع‌آوری ناقص و ناموفق داده‌هاست.

جمع‌بندی:

برای به خاطر سپردن این 2 مفهوم، می‌توانید از این قاعده ساده استفاده کنید:

Robustness (مقاومت)

سپری است که خزشگر در دست می‌گیرد تا از خودش در برابر وب محافظت کند (جلوگیری از افتادن در تله‌ها و کرش کردن).

Politeness (ادب)

ترمزی است که خزشگر اعمال می‌کند تا سرورهای وب را در برابر سرعت خودش محافظت کند (جلوگیری از مسدود شدن شبکه).

بدون هر 1 از این 2، چرخه حیات سیستم خزش (**Crawl Lifecycle**) پیش از رسیدن به مرحله نمایه‌سازی، متوقف خواهد شد.

پاسخنامه مسئله 20.2: ویژگی‌های ترجیحی خزشگر (Preferred Crawler Features)

مقدمه و مرور مفاهیم

در معماری موتورهای جستجو، اگرچه مقاومت (**Robustness**) و ادب (**Politeness**) شروط لازم و حیاتی برای شروع به کار 1 خزشگر هستند، اما برای مواجهه با حجم عظیم داده‌ها در دنیای واقعی، خزشگر باید دارای 2 ویژگی ترجیحی اما به شدت کلیدی باشد:

1. **توزیع‌شدگی (Distributed):** اجرای همزمان فرآیند خزش توسط چندین ماشین (گره یا Node) مستقل که به‌صورت هماهنگ در شبکه‌ای وسیع کار می‌کنند.

2. **مقیاس‌پذیری (Scalability):** قابلیت سیستم برای افزایش توان عملیاتی (**Throughput**) صرفاً با اضافه کردن سخت‌افزار جدید، بدون نیاز به تغییر در معماری یا توقف نرم‌افزار.

بخش 1) مزیت‌های خزشگر توزیع‌شده نسبت به متمرکز

1 خزشگر متمرکز (**Centralized**) که تنها روی 1 ماشین یا 1 خوشه سرور محلی اجرا می‌شود، در برابر 1 معماری توزیع‌شده از نظر چندین شاخص مهندسی دچار ضعف است. جدول زیر این تفاوت‌ها را برجسته می‌کند:

شاخص ارزیابی	خزشگر متمرکز (Centralized)	خزشگر توزیع‌شده (Distributed)
گلوگاه شبکه (Bottleneck)	تمام ترافیک ورودی و خروجی باید از 1 روتر و رابط شبکه عبور کند.	ترافیک در نقاط جغرافیایی مختلف و پهنای باندهای مستقل پخش می‌شود.
تحمل‌پذیری خطا (Fault Tolerance)	خرابی سرور مرکزی (Single Point of Failure) مساوی با توقف کامل خزش است.	با خرابی 1 گره، سایر گره‌ها وظایف آن را بر عهده می‌گیرند.
تاخیر ارتباطی (Latency)	به دلیل فاصله جغرافیایی سرور با برخی میزبان‌ها، تاخیر شبکه بالا است.	هر گره می‌تواند صفحات نزدیک به منطقه جغرافیایی خود را واکشی کند.
منابع پردازشی (CPU/RAM)	محدود به ظرفیت سخت‌افزاری و حافظه 1 سرور واحد است.	بار پردازشی (مثل تحلیل گراف وب) بین هزاران سرور تقسیم می‌شود.

تحلیل ساختاری:

در عملیات خزش، وظایف سنگینی مانند حل‌وفصل آدرس‌های دامنه (**DNS Resolution**)، استخراج پیوندها از بدنه HTML، و بررسی تکراری بودن صفحات (**Duplicate Detection**) نیازمند پردازش بالایی است. توزیع‌شدگی این امکان را فراهم می‌کند که هر گره به‌طور مستقل بخشی از گراف وب را پردازش کند و نتایج را در 1 مخزن مشترک بهینه‌سازی کند.

بخش 2) آیا خزش وب بدون این ویژگی‌ها در مقیاس بالا ممکن است؟

پاسخ تحلیلی: خیر.

در مقیاس واقعی وب (**Web-Scale**)، استفاده از 1 سیستم متمرکز از نظر فیزیکی و منطقی غیرممکن است. این محدودیت ناشی از ضعف الگوریتم‌ها نیست، بلکه ریشه در محدودیت‌های فیزیکی سخت‌افزار و شبکه دارد.

اثبات ریاضی (محاسبات گلوگاه)

برای درک این محدودیت، 1 سناریوی فرضی و ساده‌شده از وب را بررسی می‌کنیم:

• فرض کنید قصد داریم تنها 10 میلیارد صفحه وب را در طول 1 ماه (حدود 2.59×10^6 ثانیه) واکشی کنیم.

• فرض کنید میانگین حجم هر صفحه (به همراه کدهای HTML و هدرهای پروتکل HTTP) برابر با 100 کیلوبایت باشد.

برای انجام این عملیات در زمان مقرر، نرخ واکشی (Fetch Rate) مورد نیاز برابر است با:

$$\text{Fetch Rate} = \frac{10,000,000,000 \text{ Pages}}{2.59 \times 10^6 \text{ sec}} \approx 3,861 \text{ Pages/sec}$$

پهنای باند مداوم و خالصی که تنها برای دانلود این صفحات (بدون احتساب درخواست‌های DNS و ترافیک سربرار شبکه) نیاز است، به شرح زیر محاسبه می‌شود:

$$\text{Bandwidth} = 3,861 \times 100 \text{ KB} \approx 386 \text{ MB/sec} \approx 3 \text{ Gbps}$$

دلایل شکست سیستم متمرکز تحت این بار کاری:

عامل 1 گلوگاه I/O دیسک	ذخیره‌سازی مداوم نزدیک به 4000 فایل در هر ثانیه، سرعت نوشتن (Write Speed) هر سیستم فایل استاندارد و دیسک فیزیکی متمرکزی را به اشباع می‌کشاند.
عامل 2 انسداد در DNS	هیچ سرور DNS محلی توانایی پاسخگویی پایدار به 4000 درخواست جستجوی مستقل در هر ثانیه را به مدت 1 ماه مداوم ندارد.
عامل 3 قفل‌های همگام‌سازی (Sync Locks)	مدیریت 1 «صف خزش» (Crawl Frontier) شامل میلیاردها آدرس و بررسی لحظه‌ای اینکه «آیا این URL قبلاً دیده شده است یا خیر؟» در 1 سیستم متمرکز نیازمند قفل‌گذاری روی ساختارهای داده است. در این نرخ از درخواست، سربرار مدیریت این قفل‌ها باعث قفل‌شدگی منابع (Deadlock) یا توقف پردازش می‌شود.

نتیجه‌گیری پایانی:

در معماری موتورهای جستجو، مقیاس‌پذیری و توزیع‌شدگی صرفاً «ویژگی‌های لوکس» برای بهبود عملکرد نیستند؛ بلکه تنها راهکار فیزیکی و مهندسی موجود برای غلبه بر محدودیت‌های پهنای باند، پردازش مرکزی و ذخیره‌سازی در مواجهه با گراف بی‌نهایت وب به شمار می‌آیند.

پاسخنامه مسئله 20.3: تحلیل ماژول تشخیص محتوای تکراری (Duplicate Detection)

بخش 1) مقدمه: پدیده محتوای تکراری در وب

پیش از بررسی ساختار ماژول، باید بدانیم که گراف وب مملو از صفحات تکراری (Duplicate) یا شبه‌تکراری (Near-duplicate) است. این افزونگی داده‌ها به دلایل متعددی رخ می‌دهد؛ از جمله:

تنوع در آدرس‌دهی (URL Aliasing): 1 محتوای واحد ممکن است با چندین آدرس در دسترس باشد (مثلاً http در

برابر https، با www و بدون آن، یا وجود پارامترهای نشست کاربری در URL مانند `?session=123`).

نسخه‌برداری و آینه‌سازی (Mirroring): کپی شدن محتوای 1 سایت در سایت‌های دیگر برای توزیع بار یا سرقت محتوا.

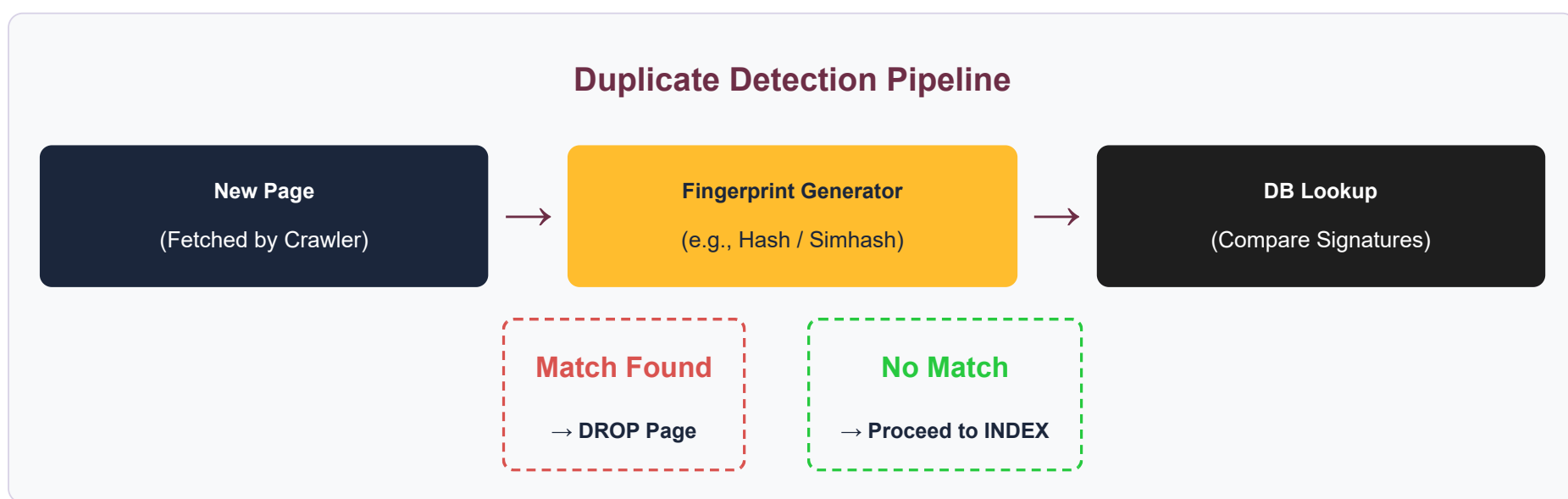
صفحات قالب‌بندی‌شده (Templated Pages): صفحاتی که متن اصلی آن‌ها یکسان است، اما صرفاً در هدر، فوتر یا

تبلیغات تفاوت‌های جزئی دارند.

بخش 2) وظیفه ماژول Duplicate Detection

در معماری موتورهای جستجو، ماژول تشخیص محتوای تکراری به‌عنوان 1 فیلتر کنترلی عمل می‌کند. وظیفه اصلی این ماژول این است که با مقایسه محتوای صفحه واکنشی‌شده با پایگاه داده صفحات قبلی، تشخیص دهد که آیا این سند، اطلاعات جدیدی برای نمایه‌سازی (Indexing) دارد یا صرفاً تکرار داده‌های پیشین است.

این فرآیند عموماً از طریق تولید 1 شناسه یکتا یا «اثر انگشت» (Fingerprint) برای هر صفحه و مقایسه آن با آثار انگشت ذخیره‌شده انجام می‌شود.



بخش 3) مکانیزم‌های جلوگیری از اتلاف منابع (تحلیل هزینه-فایده)

اگر این ماژول از معماری خزشگر حذف شود، سیستم به سرعت با 1 بحران منابع (Resource Exhaustion) مواجه خواهد شد. این ماژول از 4 جنبه حیاتی از اتلاف منابع جلوگیری می‌کند:

1. آزادسازی فضای ذخیره‌سازی دیسک (Storage):

ذخیره کردن متن کامل (Full-text) میلیاردها صفحه وب نیازمند آرایه‌های ذخیره‌سازی عظیمی است. اگر خزشگر 10 نسخه از 1 مقاله ویکی‌پدیا را که با URL‌های مختلف واکنشی شده‌اند در مخزن اسناد (Document Repository) ذخیره کند، حجم عظیمی از فضای دیسک برای نگهداری داده‌های بی‌ارزش هدر می‌رود. این ماژول تضمین می‌کند که از هر محتوای یکتا، تنها 1 نماینده ذخیره شود.

2. کاهش بار پردازشی (CPU & Processing):

نمایه‌سازی 1 صفحه شامل مراحل سنگینی مانند تجزیه ساختار (HTML Parsing)، استخراج کلمات، حذف کلمات توقف (Stop-words)، ریشه‌یابی (Stemming) و بروزرسانی نمایه وارونه (Inverted Index) است. با دور انداختن صفحات تکراری در همان مراحل اولیه، ماژول Duplicate Detection از تحمیل این بار پردازشی به سیستم جلوگیری می‌کند.

3. مدیریت بهینه پهنای باند و صف خزش:

هنگامی که خزشگر 1 صفحه تکراری را پردازش می‌کند، تمام پیوندهای (Links) خروجی آن صفحه را مجدداً استخراج کرده و به صف خزش (Crawl Frontier) اضافه می‌کند. این امر باعث بزرگ شدن تصاعدی و بی‌دلیل صف خزش

می‌شود. با تشخیص زودهنگام تکراری بودن صفحه، خزشگر پیوندهای آن را نادیده گرفته و پهنای باند شبکه را صرف واکشی صفحات واقعاً جدید می‌کند.

4. ارتقای کیفیت نتایج (جلوگیری از اتلاف زمان کاربر):

علاوه بر منابع سخت‌افزاری، این مازول از اتلاف منبع ارزشمندتری به نام «زمان کاربر» جلوگیری می‌کند. اگر محتوای تکراری فیلتر نشود، صفحه نتایج جستجو (SERP) پر از لینک‌های متفاوتی می‌شود که همگی به 1 محتوای یکسان اشاره دارند، که این موضوع تجربه کاربری را به شدت کاهش می‌دهد.

بخش 4) نگاهی به الگوریتم‌های پایه

مقایسه کلمه‌به‌کلمه 2 سند برای تشخیص تکراری بودن، از نظر پردازشی دارای پیچیدگی زمانی $O(N^2)$ است و در مقیاس وب غیرممکن است. به همین دلیل، در این مازول از تکنیک‌های درهم‌سازی (Hashing) با پیچیدگی بسیار پایین و نزدیک به $O(1)$ استفاده می‌شود.

نوع تکرار	الگوریتم‌های مرجع	نحوه عملکرد و ویژگی‌ها
تکرارهای دقیق (Exact Duplicates)	MD5, SHA-1, SHA-256	سیستم از توابع هش استاندارد روی کل محتوای صفحه استفاده می‌کند. اگر هش صفحه جدید با هشی در پایگاه داده برابر باشد، صفحه دقیقاً 100% تکراری است. (تغییر حتی 1 فاصله، هش را کاملاً دگرگون می‌کند).
تکرارهای تقریبی (Near-Duplicates)	MinHash, Simhash	از آنجایی که هدرها و فوترها ممکن است متغیر باشند، این تکنیک‌ها با استفاده از محاسبه شباهت ژاکارد (Jaccard Similarity) به خزشگر اجازه می‌دهند تا صفحاتی که مثلاً 95% تشابه متنی دارند را شناسایی کرده و از نمایه‌سازی مجدد آن‌ها جلوگیری کند.

جمع‌بندی: معماری قدرتمند 1 موتور جستجو، کیفیت و سرعت خود را تا حد زیادی وام‌دار مازول تشخیص محتوای تکراری است. این سیستم با پیاده‌سازی ریاضیات گسسته (مانند درهم‌سازی و محاسبات احتمالاتی)، از ورود زباله‌های دیجیتال به هسته اصلی موتور جستجو جلوگیری کرده و پایداری کل زیرساخت را تضمین می‌نماید.

پاسخنامه مسئله 20.4: گلوگاه وضوح نام دامنه (DNS Resolution Bottleneck)

مقدمه: نقش DNS در عملیات خزش

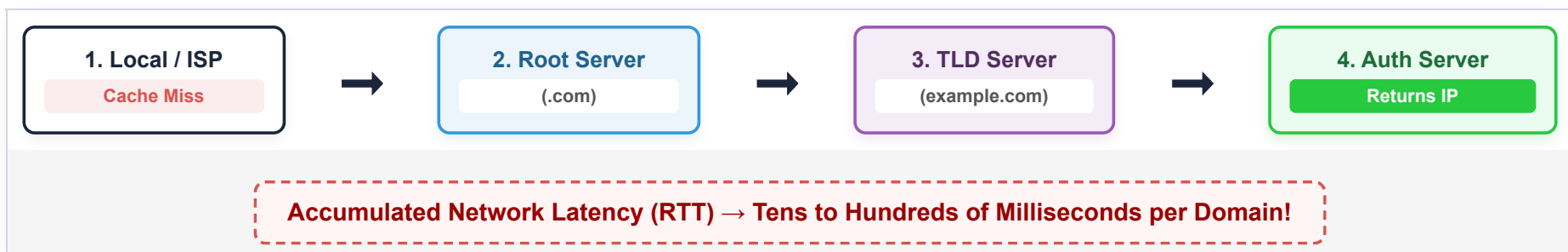
پیش از آنکه خزشگر بتواند 1 پروتکل HTTP را برای دریافت محتوای صفحه آغاز کند، ابتدا باید نام دامنه (مانند `www.example.com`) را به 1 آدرس فیزیکی در شبکه (IP Address) ترجمه کند. این وظیفه بر عهده سیستم نام دامنه یا DNS (Domain Name System) است.

در حالی که واکشی خود صفحه وب با استفاده از پروتکل‌های بهینه‌شده و اتصالات پایدار (Keep-Alive) انجام می‌شود، عملیات DNS ماهیتی متفاوت دارد و در مقیاس موتورهای جستجو، به یکی از جدی‌ترین گلوگاه‌های (Bottlenecks) سیستم تبدیل می‌شود.

کندی فرآیند DNS در خزش وب ناشی از ترکیب 4 عامل ساختاری و نرم‌افزاری زیر است:

عامل ساختاری	شرح مکانیزم و اثر آن بر سیستم
عامل 1 ماهیت مسدودکننده (Synchronous APIs)	در اکثر سیستم‌عامل‌ها، توابع استاندارد ترجمه نام دامنه (مانند <code>gethostbyname</code>) در زبان‌های برنامه‌نویسی C/C++ به صورت همگام (Synchronous) پیاده‌سازی شده‌اند. این بدان معناست که وقتی 1 رشته پردازشی (Thread) درخواست ترجمه 1 دامنه را می‌دهد، تا زمان دریافت پاسخ نهایی از شبکه متوقف (Block) می‌شود. از آنجا که 1 خزشگر برای رسیدن به توان عملیاتی بالا نیازمند ارسال هزاران درخواست موازی است، مسدود شدن رشته‌های پردازشی باعث اتلاف شدید منابع CPU و کاهش نرخ خزش (Crawl Rate) می‌شود.
عامل 2 تاخیر انباشته (Network Latency)	ترجمه 1 آدرس اینترنتی 1 عملیات تک‌مرحله‌ای نیست. اگر آدرس در حافظه پنهان (Cache) موجود نباشد، سرور DNS باید 1 مسیر سلسله‌مراتبی را طی کند (جستجو از سرور ریشه، سپس TLD و در نهایت سرور معتبر). هر 1 از این پرش‌های شبکه (Network Hops) دارای تاخیر زمانی (RTT) است. مجموع این تاخیرها برای دامنه‌های ناشناخته می‌تواند از ده‌ها تا صدها میلی‌ثانیه زمان ببرد.
عامل 3 رفتار متفاوت خزشگر (Low Cache Hit)	کاربران انسانی معمولاً در طول روز به تعداد محدودی از سایت‌های آشنا سر می‌زنند. بنابراین، سرورهای DNS محلی ارائه‌دهندگان اینترنت (ISP)، آدرس این سایت‌ها را در حافظه پنهان خود دارند و پاسخ را در کمتر از 1 میلی‌ثانیه برمی‌گردانند. اما 1 خزشگر دائماً در حال کشف دامنه‌های جدید، حاشیه‌ای و کمتر شناخته‌شده است. در نتیجه، نرخ برخورد با حافظه پنهان (Cache Hit Ratio) در سیستم خزش بسیار پایین است و خزشگر مجبور است برای درصد بالایی از دامنه‌ها، کل مسیر سلسله‌مراتبی شبکه را از ابتدا طی کند.
عامل 4 مسدودسازی ترافیک (Throttling)	سرورهای عمومی DNS (مانند سرورهای گوگل 8.8.8.8 یا سرورهای ISP) برای پاسخگویی به بار ترافیکی عادی طراحی شده‌اند. هنگامی که 1 خزشگر هزاران درخواست ترجمه را در هر ثانیه به این سرورها ارسال می‌کند، مکانیزم‌های دفاعی سرور (Rate Limiting) این حجم از درخواست را به عنوان 1 حمله (DDoS) شناسایی کرده و بسته‌های داده را دور می‌ریزند (Packet Drop) یا اتصال را موقتاً مسدود می‌کنند.

مدل‌سازی شماتیک پیشرفته: سلسله‌مراتب شبکه و تاخیر انباشته DNS



جمع‌بندی: راهکار مهندسی چیست؟

به دلیل گلوگاه‌های فوق، خزشگرهای صنعتی و موتورهای جستجوی بزرگ هرگز از توابع استاندارد سیستم‌عامل یا سرورهای عمومی DNS استفاده نمی‌کنند.

برای حل این مشکل، معماران موتور جستجو باید 1 سرور DNS اختصاصی و غیرمسدودکننده (Asynchronous Custom DNS Resolver) در کنار خزشگر توسعه دهند که درخواست‌ها را به صورت موازی (بر پایه UDP) مدیریت کرده

و دارای 1 سیستم **Caching** بسیار تهاجمی (**Aggressive**) باشد تا تاخیر شبکه را به حداقل رسانده و از قفل‌شدگی منابع جلوگیری نماید.

پاسخنامه مسئله 20.5: ساختار صف آدرس‌ها (URL Frontier Architecture)

مقدمه: چرا صف FIFO شکست می‌خورد؟

در نگاه اول، ساده‌ترین ساختار داده برای نگهداری آدرس‌های استخراج‌شده (**URL**) استفاده از 1 صف استاندارد **FIFO** (اولین ورودی، اولین خروجی) است. در این مدل، هر پیوندی که کشف می‌شود به انتهای صف می‌رود و رشته‌های خزشگر (**Crawl Threads**) از ابتدای صف آدرس‌ها را برمی‌دارند.

اما در محیط واقعی وب، استفاده از 1 صف ساده **FIFO** به سرعت منجر به نقض 2 اصل حیاتی «ادب» (**Politeness**) و «کیفیت/اولویت» (**Priority**) می‌شود که در مسائل قبلی به آن‌ها اشاره کردیم.

سناریوی شکست صف FIFO

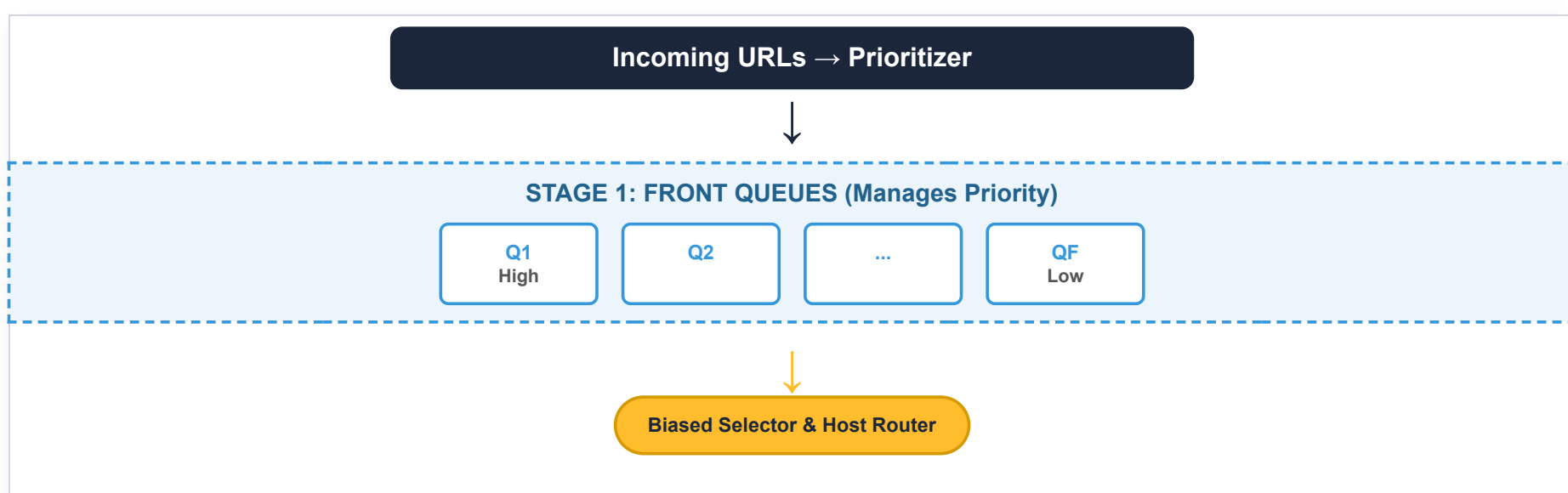
فرض کنید خزشگر وارد سایت **wikipedia.org** می‌شود و در 1 لحظه هزاران پیوند داخلی ویکی‌پدیا را استخراج کرده و به صف **FIFO** اضافه می‌کند. چه اتفاقی می‌افتد؟

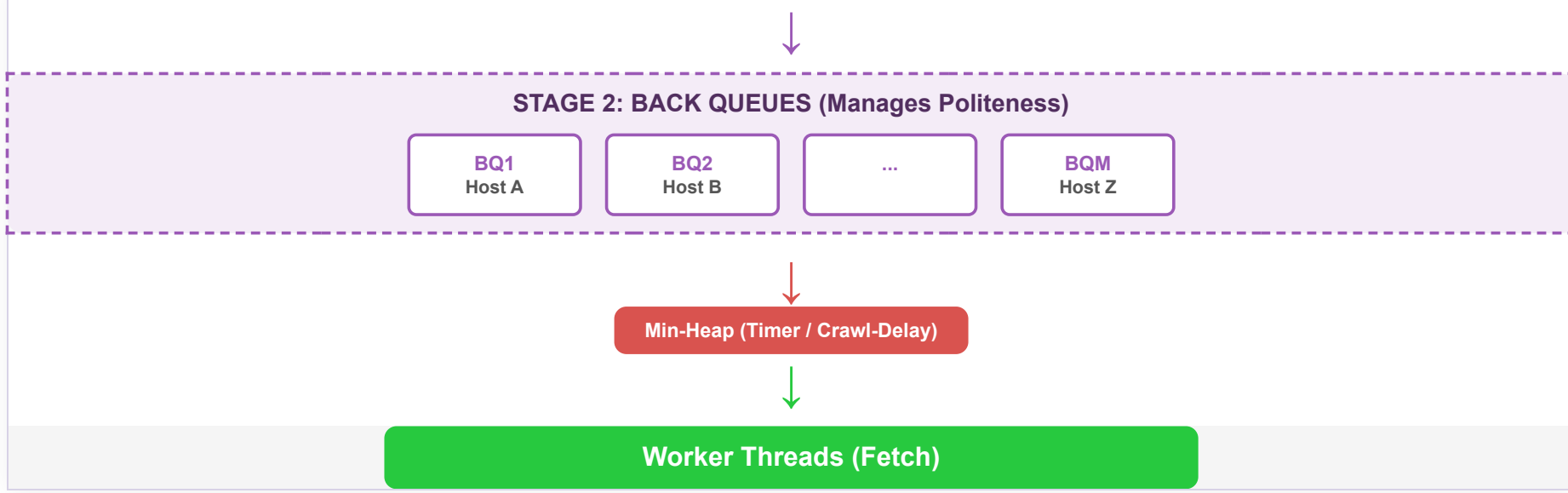
هزاران آدرس اول صف همگی متعلق به 1 دامنه (ویکی‌پدیا) هستند. خزشگر تمام این آدرس‌ها را پشت سر هم واکنشی می‌کند و باعث حمله منع سرویس (DoS) به سرور میزبان می‌شود.	بحران 1 نقض ادب (Politeness Violation)
آدرس‌های بسیار مهم‌تری از سایت‌های خبری که دقایقی بعد کشف می‌شوند، باید ساعت‌ها در انتهای این صف عظیم منتظر بمانند تا نوبتشان برسد که به این پدیده گرسنگی (Starvation) می‌گویند.	بحران 2 افت کیفیت خزش (Priority Starvation)

معماری صف خزش پیشرفته (URL Frontier / Mercator)

برای حل این مشکلات، معماری استاندارد **URL Frontier** (معروف به معماری مرکاتور - **Mercator**) به جای 1 صف یکپارچه، از 1 سیستم دو مرحله‌ای (**Two-Stage**) استفاده می‌کند. این سیستم شامل **صف‌های جلویی (Front Queues)** و **صف‌های پشتی (Back Queues)** است.

مدل‌سازی شماتیک: ساختار دو مرحله‌ای مرکاتور (Mercator Architecture)





نکته کلیدی:

جداسازی منطق اولویت از منطق ادب، راز موفقیت این معماری است. بخش بالایی تصمیم می‌گیرد «چه چیزی مهم‌تر است»، و بخش پایینی تضمین می‌کند که «سرور میزبان تحت فشار قرار نگیرد».

تشریح مراحل از ورودی تا خروجی

برای درک دقیق این معماری، مسیر حرکت **URL 1** را از لحظه ورود تا لحظه خروج به سمت شبکه ردیابی می‌کنیم:

مرحله اول: صف‌های جلویی (Front Queues) - مدیریت اولویت

هدف این مرحله، پیاده‌سازی مکانیزم **Priority** است.

هر URL جدیدی که وارد می‌شود، بر اساس فاکتورهایی مانند PageRank قبلی، نرخ تغییرات سایت (Update Frequency) و عمق صفحه، 1 امتیاز اولویت دریافت می‌کند.	1. اولویت‌بند (Prioritizer)
بر اساس این امتیاز، URL در یکی از F صف‌های جلویی قرار می‌گیرد (مثلاً صف شماره 1 برای بالاترین اولویت و صف شماره F برای پایین‌ترین اولویت).	2. مسیریابی (Routing)
1 الگوریتم انتخاب تصادفی سوگیری‌شده (Biased Random Selector) برای برداشتن آدرس‌ها از صف‌های جلویی استفاده می‌شود که به صف‌های دارای اولویت بالاتر، شانس بیشتری برای انتخاب شدن می‌دهد (بدون اینکه صف‌های پایین‌تر را کاملاً محروم کند).	3. انتخاب‌گر (Selector)

مرحله دوم: صف‌های پشتی (Back Queues) - مدیریت ادب

هدف این مرحله، پیاده‌سازی مکانیزم **Politeness** است. قانون طلایی در اینجا این است: «هر صف پشتی، در هر لحظه فقط و

فقط مجاز به نگهداری آدرس‌های مربوط به **1 دامنه (Host)** مشخص است.»

آدرسی که از صف‌های جلویی خارج شده، بررسی می‌شود. سیستم به دنبال صف پشتی‌ای می‌گردد که هم‌اکنون به آن دامنه (مثلاً wikipedia.org) اختصاص دارد.	1. مسیریاب دامنه (Router)
اگر صفی برای آن دامنه وجود داشت، آدرس به انتهای آن اضافه می‌شود. اگر وجود نداشت، سیستم یکی از صف‌های پشتی که در حال حاضر خالی است را به آن دامنه اختصاص می‌دهد.	2. تخصیص پویا (Allocation)
هر صف پشتی 1 زمان مشخص دارد که نشان می‌دهد چه زمانی مجاز است آدرس بعدی خود را برای واکنشی ارسال کند (بر اساس Crawl-Delay سرور میزبان). 1 ساختار داده هرم (Heap) این زمان‌ها را مدیریت می‌کند.	3. زمان‌بند (Heap/Timer)

رشته‌های خزشگر (Crawl Threads) فقط از هرم، صف‌هایی را انتخاب می‌کنند که زمان تاخیر مجاز آن‌ها سپری شده باشد. پس از واکنشی 1 آدرس، زمان مجاز بعدی آن صف به‌روزرسانی شده و مجدداً در هرم قرار می‌گیرد.

پاسخنامه مسئله 20.6: معماری توزیع نمایه وارونه (Distributed Indexing Architecture)

مقدمه: نیاز قطعی به توزیع شاخص

در سیستم‌های مقیاس‌پذیر موتورهای جستجو، حجم نمایه وارونه (Inverted Index) به قدری بزرگ می‌شود که امکان ذخیره‌سازی و پردازش آن روی 1 سرور واحد (Single Machine) از نظر فیزیکی وجود ندارد. برای درک این مقیاس، در نظر بگیرید که گراف وب شامل ده‌ها میلیارد سند است و نمایه استخراج‌شده از آن به پتابایت‌ها داده خام و پردازش‌شده می‌رسد. بنابراین، معماران سیستم باید این ماتریس عظیم داده را روی 1 خوشه (Cluster) از سرورها (گره‌ها یا Nodes) توزیع کنند. برای قطعه‌قطعه کردن (Partitioning) این نمایه، 2 رویکرد ریاضی و ساختاری کاملاً متفاوت وجود دارد: توزیع سند-محور (طولی) و توزیع واژه-محور (عرضی). در ادامه مکانیزم دقیق، روند پردازش و مزایا و معایب هر یک را موشکافی می‌کنیم.

بخش 1) معماری توزیع سند-محور (Document-Partitioned / Local Indexing)

در این روش، مجموعه کل اسناد وب بین N گره مختلف تقسیم می‌شود. تمرکز این معماری بر روی بخش‌بندی پایگاه داده اسناد است.

مکانیزم ذخیره‌سازی:

اگر کل اسناد را 1,000,000,000 (1 میلیارد) فرض کنیم و 10 سرور داشته باشیم، به هر سرور 100,000,000 (100 میلیون) سند اختصاص می‌یابد. هر سرور به طور کاملاً مستقل، 1 نمایه وارونه کامل برای اسناد خودش می‌سازد. در این حالت، کلمه "دانشگاه" در دیکشنری هر 10 سرور وجود دارد، اما لیست پستینگ (Posting List) هر سرور، فقط و فقط شامل اسنادی از همان سرور است که کلمه "دانشگاه" در آن‌ها آمده است.

نحوه پردازش پرسمان (Query Routing):

- کاربر پرسمان را ارسال می‌کند و درخواست به گره مرکزی (Broker) می‌رسد.
- گره مرکزی، پرسمان را به تمام سرورها (Broadcast) ارسال می‌کند.
- هر سرور به صورت محلی جستجو را انجام داده و k نتیجه برتر (Top-k) خود را به گره مرکزی برمی‌گرداند.
- گره مرکزی این نتایج را با هم ادغام (Merge) کرده و رتبه‌بندی نهایی را به کاربر نمایش می‌دهد.

بخش 2) معماری توزیع واژه-محور (Term-Partitioned / Global Indexing)

در این روش، برخلاف رویکرد قبلی، اسناد تقسیم نمی‌شوند، بلکه دامنه لغات (Vocabulary) بین سرورها توزیع و تقسیم می‌شود.

مکانیزم ذخیره‌سازی:

دیکشنری لغات بر اساس حروف الفبا یا توابع درهم‌سازی (Hash) بین N سرور توزیع می‌شود. به عنوان مثال، سرور

A مسئول نگهداری کلماتی است که با حروف "الف" تا "خ" شروع می‌شوند و سرور **B** مسئول حروف "د" تا "ی" است. در این معماری، اگر سرور **A** کلمه "دانشگاه" را ذخیره کرده باشد، لیست پستینگ آن شامل تمام اسناد موجود در کل وب است که این کلمه را دارند.

نحوه پردازش پرسمان (Query Routing):

- کاربر پرسمان را به گره مرکزی ارسال می‌کند.
- گره مرکزی بررسی می‌کند که کلمات پرسمان در دامنه وظایف کدام سرورها قرار دارند و درخواست را فقط به همان سرورها ارسال می‌کند.
- اگر پرسمان شامل چند کلمه باشد (مثلاً **A AND B**) و کلمه **A** در سرور 1 و کلمه **B** در سرور 2 باشد، 1 فاجعه پردازشی رخ می‌دهد: سرور 1 باید کل لیست پستینگ کلمه **A** را از طریق شبکه برای سرور 2 ارسال کند تا عمل اشتراک‌گیری (**Intersection**) انجام شود.

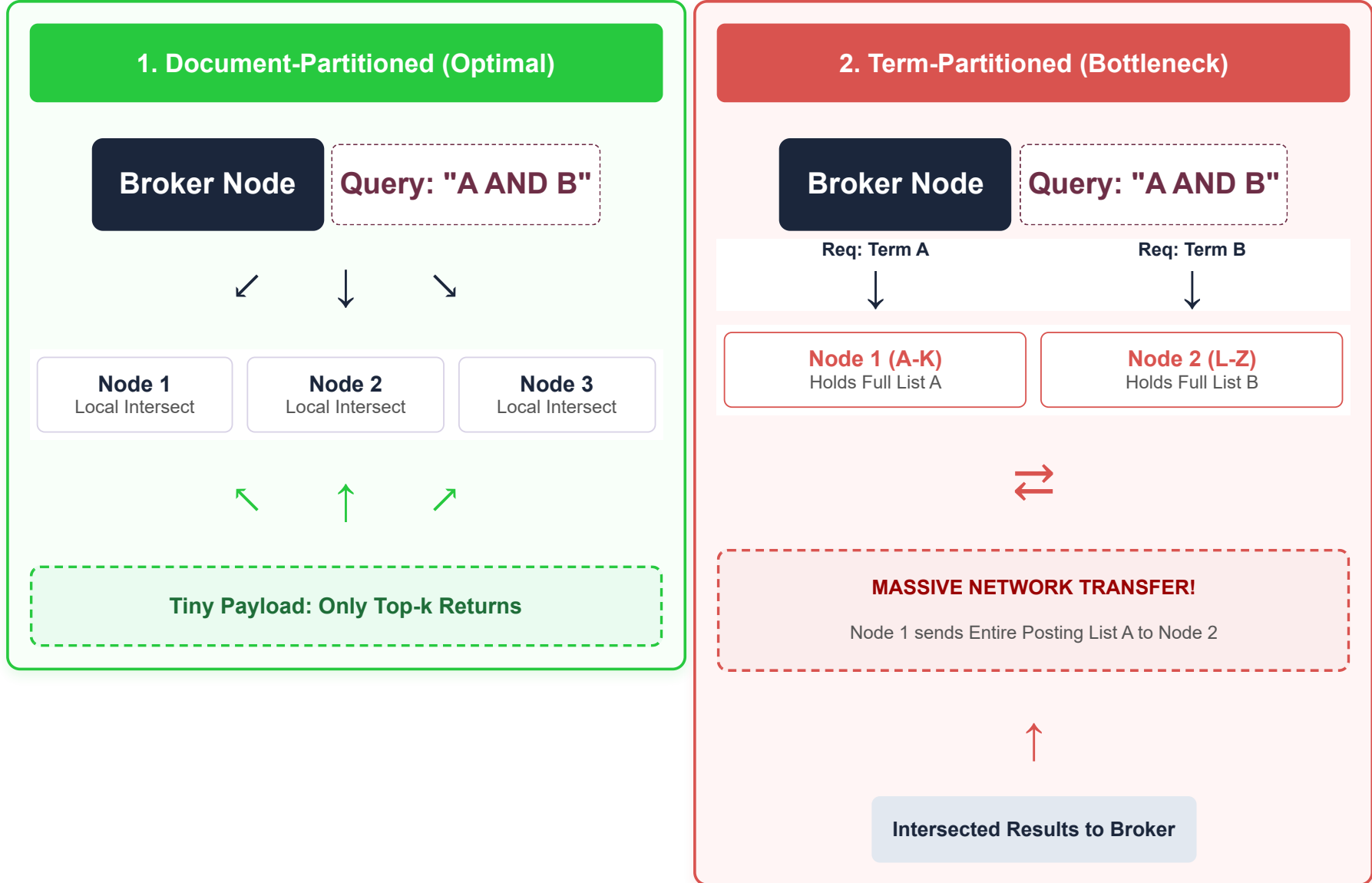
بخش 3) مقایسه تحلیلی و مهندسی 2 معماری

تفاوت بنیادین این 2 معماری از منظر مهندسی سیستم‌های توزیع‌شده، در جدول پیشرفته زیر مقایسه شده است. (رنگ سبز نشانگر برتری و رنگ قرمز نشانگر نقطه ضعف معماری است).

شاخص ارزیابی	توزیع سند-محور (Document-Partitioned)	توزیع واژه-محور (Term-Partitioned)
ترافیک شبکه (Network Overhead)	بسیار پایین (بهینه): فقط k نتیجه برتر از هر گره در شبکه منتقل می‌شود.	بسیار بالا (گلوگاه): در پرسمان‌های چندکلمه‌ای، لیست‌های طولانی پستینگ باید بین سرورها منتقل شوند.
موازی‌سازی (Concurrency)	پردازش توزیع‌شده کامل: تمام سرورها همزمان روی 1 پرسمان کار می‌کنند. زمان پاسخگویی به شدت کاهش می‌یابد.	عدم تعادل بار: فقط سرورهای حاوی کلمات پرسمان درگیر می‌شوند؛ سایر سرورها در آن لحظه بیکار می‌مانند.
تحمل‌پذیری خطا (Fault Tolerance)	بالا (تخریب تدریجی): با خرابی 1 سرور، فقط 10% از اسناد از نتایج حذف می‌شوند، اما سیستم همچنان کار می‌کند.	پایین (شکست فاجعه‌بار): با خرابی 1 سرور، دسترسی به بخشی از دایره لغات کاملاً قطع می‌شود و پرسمان‌ها متوقف می‌شوند.
پراکندگی داده‌ها (Data Skew)	توزیع یکنواخت: بار ذخیره‌سازی اسناد به طور مساوی و بالانس شده بین تمام سرورها تقسیم می‌شود.	فشار متمرکز: کلمات پرکاربرد باعث پر شدن ظرفیت و فشار پردازشی مضاعف روی 1 سرور خاص می‌شوند.

بخش 4) شبیه‌ساز پایدار جریان پرسمان (Query Flow Simulator)

برای درک بهتر تفاوت بار شبکه در هنگام اجرای پرسمان‌های چندکلمه‌ای (مانند **A AND B**)، مسیر حرکت داده‌ها با استفاده از ساختار جدول‌بندی پایدار (سازگار با محیط **Jupyter**) ترسیم شده است:



نتیجه‌گیری پایانی:

به دلیل سربار وحشتناک و فلج‌کننده شبکه در انتقال لیست‌های حجیم پستینگ (همان‌طور که در شبیه‌ساز مشاهده شد) و همچنین چالش‌های فاجعه‌بار مربوط به تحمل خطای سیستم، در تمامی موتورهای جستجوی مدرن و مقیاس‌وب (نظیر معماری زیرساخت **Google** و **Bing**)، به صورت 1 استاندارد صنعتی غیرقابل تغییر، از **معماری توزیع سند-محور (Document-partitioned)** استفاده می‌شود. رویکرد واژه-محور صرفاً دارای ارزش تئوریک و دانشگاهی است، اما از لحاظ مهندسی قابلیت اجرای پایدار را ندارد.

پاسخنامه مسئله 20.7: هدف و معماری سرورهای اتصال (Connectivity Servers)

مقدمه: گراف وب و نیاز به تفکیک داده‌ها

در معماری 1 موتور جستجو، داده‌های جمع‌آوری‌شده توسط خزشگر به 2 دسته کاملاً مجزا تقسیم می‌شوند:

1. **داده‌های محتوایی (متن):** کلمات داخل صفحات که پردازش شده و در ساختاری به نام «نمایه وارونه»

(**Inverted Index**) ذخیره می‌شوند تا به پرسمان‌های متنی (**Keywords**) پاسخ دهند.

2. **داده‌های ساختاری (توپولوژی):** پیوندهایی (**Hyperlinks**) که صفحات را به یکدیگر متصل می‌کنند و شکل گراف

وب را می‌سازند.

نمایه وارونه برای جستجوی متنی بسیار بهینه‌شده است، اما برای پیمایش گراف (مثلاً یافتن پاسخ این سوال که "چه صفحاتی به صفحه A لینک داده‌اند؟") به شدت ناکارآمد است. در این نقطه، قطعه‌ای ساختاری و حیاتی به نام **سرور اتصال (Connectivity Server)** وارد معماری موتور جستجو می‌شود.

بخش 1) سرور اتصال چیست؟

سرور اتصال 1 سیستم ذخیره‌سازی و پردازش بهینه‌شده است که وظیفه دارد **ماتریس مجاورت (Adjacency Matrix)** گراف عظیم وب را نگهداری کند. این سرور به موتور جستجو اجازه می‌دهد تا با سرعت بسیار بالا (در مرتبه زمانی **O(1)** یا نزدیک به آن) به 2 سوال ساختاری حیاتی پاسخ دهد:

پیوندهای خروجی (Out-links): صفحه **X** به چه صفحاتی لینک داده است؟

پیوندهای ورودی (In-links): چه صفحاتی به صفحه **X** لینک داده‌اند؟

بخش 2) چرا PageRank به سرور اتصال وابسته است؟

الگوریتم‌های تحلیل پیوند مانند **PageRank** منحصراً بر اساس ساختار گراف وب کار می‌کنند و متن صفحات برای آن‌ها بی‌اهمیت است. برای درک این وابستگی مطلق، به فرمول پایه الگوریتم **PageRank** دقت کنید:

$$PR(u) = (1 - d) + d \sum_{v \in B(u)} \frac{PR(v)}{L(v)}$$

در این فرمول ریاضی:

مجموعه B(u): نشان‌دهنده تمام صفحاتی است که به صفحه **u** لینک داده‌اند (مجموعه **In-links**).

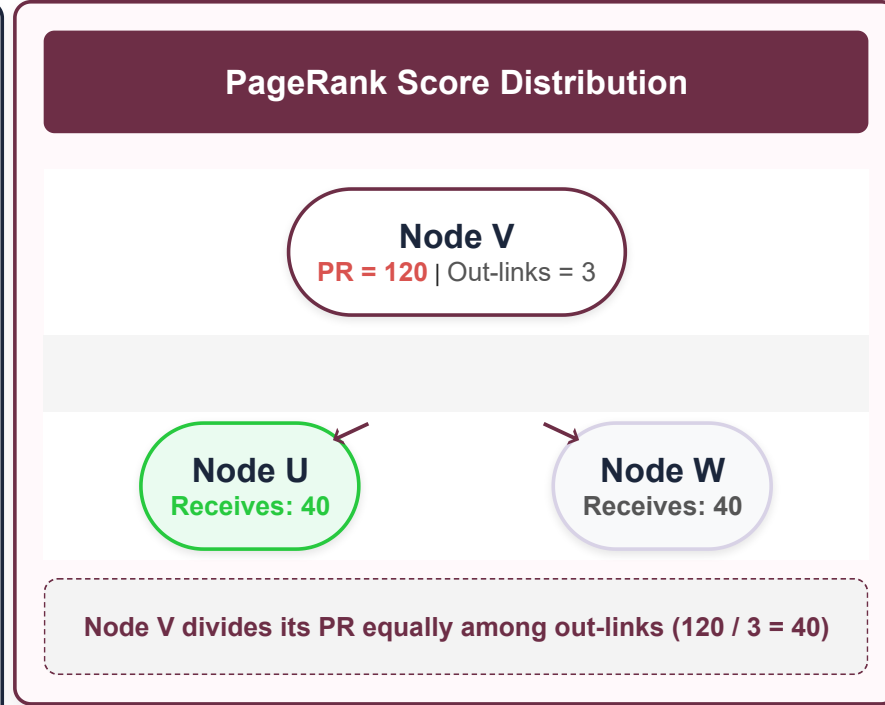
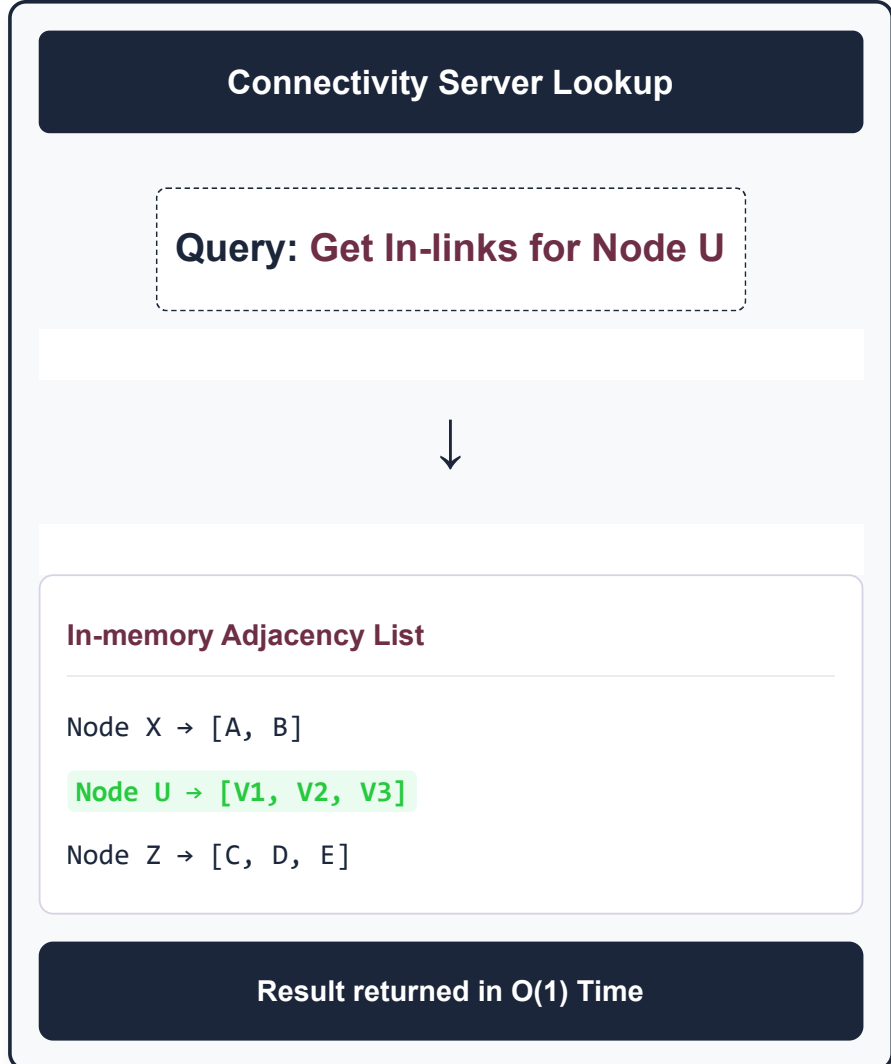
متغیر L(v): نشان‌دهنده تعداد کل لینک‌های خروجی از صفحه **v** است (**Out-degree**).

تحلیل گلوگاه محاسباتی

برای محاسبه رتبه صفحه **u**، سیستم باید بداند دقیقاً چه صفحاتی به آن لینک داده‌اند. در گراف وب با ده‌ها میلیارد گره، استخراج این اطلاعات از پایگاه‌های داده رابطه‌ای (**SQL**) یا فایل‌های خام برای هر صفحه، ماه‌ها طول می‌کشد.

از سوی دیگر، الگوریتم **PageRank** 1 الگوریتم تکرارشونده (**Iterative**) است؛ یعنی برای همگرایی اعداد، باید این فرمول ده‌ها بار روی **کل گراف وب** محاسبه شود. سرور اتصال این امکان را فراهم می‌کند که الگوریتم بتواند گراف را میلیون‌ها بار در ثانیه در حافظه اصلی (**RAM**) پیمایش کند.

برای درک بهتر نحوه توزیع امتیازها بر اساس پیوندها و نقش سرور اتصال، ساختار مدل‌سازی‌شده زیر را بررسی کنید:



بخش 3) چالش مهندسی و فشرده‌سازی (Compression)

نگهداری گراف وب در حافظه اصلی (RAM) 1 چالش مهندسی سنگین است. اگر هر پیوند را صرفاً با 2 عدد صحیح 32 بیتی (مبدأ و مقصد) ذخیره کنیم، گراف وب با 1 تریلیون پیوند به چندین ترابایت حافظه RAM سریع نیاز خواهد داشت که توجیه‌پذیر نیست.

برای دور زدن محدودیت‌های سخت‌افزاری دیسک (Disk I/O)، سرورهای اتصال از تکنیک‌های پیچیده فشرده‌سازی گراف وب (مانند الگوریتم Boldi-Vigna) استفاده می‌کنند. این تکنیک‌ها بر اساس 2 ویژگی گراف وب عمل می‌کنند:

1. مکان‌مندی (Locality): اکثر لینک‌های 1 صفحه به صفحاتی در همان دامنه یا ساختار درختی اشاره دارند.

2. تشابه (Similarity): صفحاتی که آدرس‌های مشابهی دارند، معمولاً لینک‌های خروجی بسیار مشابهی نیز دارند.

جمع‌بندی پایانی: با استفاده از رمزگذاری ارجاعی (Reference Encoding) و فاصله‌ای (Gap Encoding)، سرور اتصال می‌تواند حجم مصرفی برای ذخیره هر پیوند را از 64 بیت به کمتر از 3 بیت کاهش دهد و کل توپولوژی وب را درون کلاستر فشرده‌ای از سرورهای پردازشی جا دهد.

پاسخنامه مسئله 20.8: عملیات نرمال‌سازی آدرس‌های وب (URL Normalization)

مقدمه: آناتومی 1 آدرس وب و پدیده چندنامی (Aliasing)

در ساختار وب، 1 آدرس اینترنتی یا URL (Uniform Resource Locator) صرفاً 1 رشته متنی است که برای شناسایی و مکان‌یابی 1 منبع استفاده می‌شود. استاندارد تعریف‌کننده URL ها (RFC 3986) دارای انعطاف‌پذیری بالایی است. این انعطاف‌پذیری به مدیران سرور اجازه می‌دهد تا پیکربندی‌های متنوعی داشته باشند، اما همین ویژگی باعث بروز 1 مشکل مهندسی بزرگ در موتورهای جستجو به نام پدیده چندنامی (URL Aliasing) می‌شود.

پدیده چندنامی یعنی ده‌ها رشته متنی ظاهراً متفاوت، همگی به 1 محتوای فیزیکی واحد در 1 سرور اشاره کنند. برای مثال، تمام 4 آدرس زیر دقیقاً 1 صفحه واحد را بارگذاری می‌کنند:

HTTP://www.Example.com:80/index.html

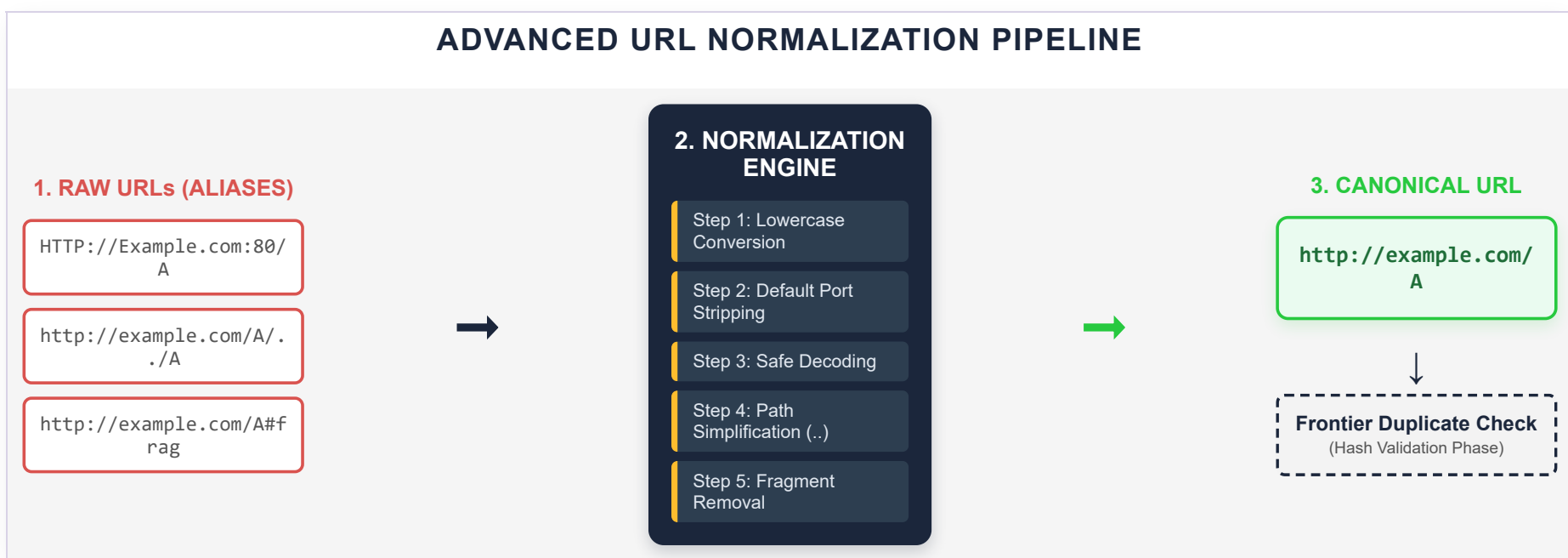
http://www.example.com/

http://example.com

http://www.example.com/page?session_id=123xyz

عملیات **نرمال‌سازی آدرس (URL Normalization)** 1 پیش‌پردازش متنی و استانداردسازی است که پیش از ورود هر آدرس به صف خزش (**URL Frontier**) روی آن اعمال می‌شود تا این تنوع ظاهری را از بین برده و تمام آدرس‌های هم‌ارز را به 1 رشته استاندارد و یکتا (**Canonical Form**) تبدیل کند.

نمودار شماتیک پیشرفته: معماری و جریان داده موتور نرمال‌سازی



چرا نرمال‌سازی پیش از ورود به صف خزش ضروری است؟

اگر عملیات نرمال‌سازی انجام نشود یا به مراحل بعد از خزش موکول گردد، سیستم خزشگر با چالش‌های مخربی در 3 سطح مواجه می‌شود:

1. شکست مکانیزم بررسی تکرار در صف (Frontier Duplicate Check):

پیش از آنکه خزشگر 1 URL جدید را به صف اضافه کند، باید بررسی کند که "آیا این آدرس قبلاً در صف قرار گرفته یا خزش شده است؟". این بررسی معمولاً از طریق مقایسه هاش (**Hash**) آدرس جدید با دیتابیس آدرس‌های دیده‌شده انجام می‌شود.

اگر آدرس `http://example.com/A` قبلاً خزش شده باشد و خزشگر با آدرس جدید `HTTP://Example.com:80/A` روبرو شود، به دلیل تفاوت در رشته متنی، تابع هاش آن‌ها کاملاً متفاوت خواهد بود. در نتیجه، خزشگر گمان می‌کند 1 صفحه جدید کشف کرده و آن را به صف اضافه می‌کند. نرمال‌سازی با یکسان‌سازی رشته‌ها، مکانیزم بررسی تکرار را نجات می‌دهد.

2. اتلاف شدید منابع (Crawling Waste):

بدون نرمال‌سازی، همان‌طور که در بند قبل اشاره شد، صف خزش پر از آدرس‌های تکراری پنهان می‌شود. خزشگر مجبور است پهنای باند گران‌قیمت خود را صرف واکنشی صفحاتی کند که از قبل آن‌ها را دارد. این موضوع، نرخ خزش

مفید (Useful Crawl Rate) را به شدت کاهش داده و بار غیرضروری و غیرمؤدبانه‌ای (نقض اصل Politeness) به سرور میزبان وارد می‌کند.

3. تضعیف الگوریتم‌های تحلیل پیوند (مانند PageRank):

الگوریتم‌هایی مانند رتبه صفحه (PageRank) بر اساس پیوندهای ورودی (In-links) به 1 صفحه کار می‌کنند. اگر 3 سایت مختلف با 3 فرمت متنی متفاوت (مثلاً با **www**، بدون **www**، و با پورت **80**) به سایت شما لینک بدهند، در صورت عدم نرمال‌سازی، موتور جستجو این لینک‌ها را بین 3 گره مجزا در گراف وب تقسیم می‌کند. در نتیجه، اعتبار و رتبه صفحه شما به جای تمرکز در 1 گره قدرتمند، پراکنده و تضعیف می‌شود.

گام‌های استاندارد در الگوریتم نرمال‌سازی

عملیات نرمال‌سازی معمولاً در 2 سطح «ایمن» (تضمین‌شده توسط پروتکل) و «اکتشافی» (وابسته به رفتار وب) انجام می‌شود. مراحل زیر در 1 خزشگر استاندارد به‌صورت متوالی پیاده‌سازی می‌گردند:

گام نرمال‌سازی	مثال قبل از اصلاح	مثال بعد از اصلاح (نرمال‌شده)	دلیل فنی و استانداردسازی
کوچک کردن حروف دامنه و پروتکل	HTTP://WWW.TEST.COM/Page	http://www.test.com/Page	طبق استاندارد DNS، نام دامنه به حروف بزرگ و کوچک حساس نیست. (بخش مسیر فایل ممکن است حساس باشد، لذا تغییر نمی‌کند).
حذف پورت‌های پیش‌فرض	http://example.com:80/	http://example.com/	پورت 80 برای HTTP و پورت 443 برای HTTPS پیش‌فرض هستند و ذکر آن‌ها زائد است.
تبدیل کاراکترهای رمزگذاری‌شده	http://test.com/%7Euser	http://test.com/~user	کاراکترهای ایمن نباید در URL به‌صورت رمزگذاری‌شده (URL Encoding) رها شوند.
حذف پوشه‌های نقطه‌دار (Dot-segments)	http://test.com/a/b/./c	http://test.com/a/c	استانداردسازی مسیر فایل با رفع ارجاعات نسبی (..) یعنی بازگشت به پوشه قبلی).
حذف شناسه قطعه (Fragment)	http://test.com/page#section	http://test.com/page	مرورگرها بخش بعد از شناسه قطعه را به سرور ارسال نمی‌کنند؛ این بخش صرفاً برای پرش کاربر در صفحه است و محتوای جدیدی به همراه ندارد.

نتیجه‌گیری پایانی:

نرمال‌سازی URL اولین سد دفاعی خزشگر در برابر هرج‌ومرج ساختاری وب است. این عملیات سبک و سریع در پردازش متن، از ساعت‌ها اتلاف وقت در منابع محاسباتی، هدررفت ترابایت‌ها پهنای باند و ایجاد نویز مخرب در ساختار گراف وب جلوگیری می‌کند.

پاسخنامه مسئله 20.9: استخراج پیوندها و تبدیل آدرس نسبی به مطلق (URL Resolution)

مقدمه: ضرورت تبدیل آدرس‌ها در خزشگر

هنگامی که ماژول استخراج پیوند (**Link Extractor**) در خزشگر، کدهای **HTML** 1 صفحه را برای یافتن تگ‌های لنگر (مانند **<a>**)، تصاویر (****) یا منابع دیگر تجزیه می‌کند، اغلب با آدرس‌های نسبی (**Relative URLs**) مواجه می‌شود. توسعه‌دهندگان وب از آدرس‌های نسبی برای کوتاه شدن کدها و سهولت جابه‌جایی سایت بین دامنه‌های مختلف استفاده می‌کنند.

با این حال، 1 آدرس نسبی (مانند **../images/photo.jpg**) به تنهایی در شبکه اینترنت قابل مسیریابی و واکشی نیست. صف خزش (**URL Frontier**) برای برقراری اتصال **HTTP** تنها آدرس‌های مطلق (**Absolute URLs**) را می‌پذیرد که دارای پروتکل و نام میزبان مشخص باشند. بنابراین، خزشگر موظف است هر آدرس نسبی را با توجه به «موقعیت فعلی سند» به 1 آدرس مطلق و کامل ترجمه کند.

نمودار شماتیک پیشرفته: خط‌لوله پردازش و تبدیل آدرس (Resolution Pipeline)

فرآیند پردازش 1 آدرس در موتور خزشگر از لحظه استخراج متن خام تا تولید 1 رشته اینترنتی قابل واکشی، در مدل پایدار زیر ترسیم شده است:



الگوریتم تبدیل آدرس نسبی به مطلق

این عملیات بر اساس استانداردهای شبکه (**RFC 3986**) در 4 گام متوالی و دقیق انجام می‌شود. ترتیب اجرای این مراحل برای جلوگیری از خطاهای مسیره‌دهی بسیار حیاتی است:

گام اجرایی	مکانیزم پردازش و جزئیات فنی
1 تعیین آدرس پایه (Base URL)	یافتن نقطه مرجع: خزشگر ابتدا باید بداند که صفحه فعلی در چه آدرسی قرار دارد. این آدرس پایه به صورت پیش‌فرض همان آدرسی است که صفحه از طریق آن واکشی شده است. با این حال، خزشگر موظف است تگ <head> سند را بررسی کند. اگر تگ <base href= "..."> در کدها وجود داشته باشد، مقدار این تگ، جایگزین آدرس پیش‌فرض شده و به عنوان آدرس پایه جدید برای تمام پیوندهای آن صفحه در نظر گرفته می‌شود.
2 تحلیل نوع پیوند (Type Analysis)	دسته‌بندی ساختاری: خزشگر پیوند پیدا شده را می‌خواند. اگر پیوند با پروتکل (مانند http:// یا https://) آغاز شود، 1 آدرس مطلق است و مستقیماً به صف خزش می‌رود. در غیر این صورت، پیوند نسبی است و یکی از حالت‌های زیر را دارد: ریشه-نسبی (Root-relative): با کاراکتر اسلش / آغاز می‌شود (مثلاً about.html/). مسیر-نسبی (Path-relative): بدون اسلش آغاز می‌شود (مثلاً files/doc.pdf).
3	چسباندن آدرس‌ها: در این مرحله، خزشگر بخش‌هایی از آدرس پایه را به پیوند نسبی متصل می‌کند:

گام اجرایی	مکانیزم پردازش و جزئیات فنی
ادغام مسیرها (Path Merging)	- برای پیوندهای ریشه-نسبی: خزشگر فقط نام پروتکل و میزبانِ آدرس پایه (مانند <code>http://example.com</code>) را نگه می‌دارد و مابقی را حذف کرده، سپس پیوند استخراج‌شده را به آن می‌چسباند. - برای پیوندهای مسیر-نسبی: خزشگر مسیر آدرس پایه را تا آخرین پوشه (Directory) نگه می‌دارد (یعنی نام فایل صفحه فعلی را حذف می‌کند) و سپس پیوند استخراج‌شده را به انتهای آن متصل می‌کند.
4 نرمال‌سازی مسیر (Normalization)	رفع ارجاعات دایرکتوری: پس از ادغام، ممکن است در آدرس نهایی نشانگرهای دایرکتوری ایجاد شده باشند. خزشگر در گام آخر: - کاراکترهای <code>./</code> (اشاره به پوشه فعلی) را بدون هیچ اثری حذف می‌کند. - در مواجهه با <code>../</code> (اشاره به پوشه والد)، پوشه قبلی را در مسیر ساخته‌شده حذف کرده و خودش را نیز پاک می‌کند تا مسیر نهایی کاملاً صاف و استاندارد شود.

بررسی نمونه‌های محاسباتی (خروجی الگوریتم)

برای درک عمیق‌تر الگوریتم، جدول تحلیلی زیر خروجی ماژول استخراج پیوند را با فرض اینکه آدرس پایه صفحه فعلی دقیقاً `http://www.university.edu/cs/courses/index.html` باشد، نشان می‌دهد:

پیوند استخراج‌شده در بدنه HTML	دسته‌بندی پیوند	آدرس مطلق محاسبه‌شده (خروجی نهایی)
<code>syllabus.pdf</code>	مسیر-نسبی (همان پوشه)	<code>http://www.university.edu/cs/courses/syllabus.pdf</code>
<code>/login.php</code>	ریشه-نسبی (از ریشه سایت)	<code>http://www.university.edu/login.php</code>
<code>../faculty/john.html</code>	مسیر-نسبی (1 پوشه به عقب)	<code>http://www.university.edu/cs/faculty/john.html</code>
<code>//other-domain.com/img.jpg</code>	مستقل از پروتکل (Protocol-relative)	<code>http://other-domain.com/img.jpg</code>

نکته کلیدی:

عدم پیاده‌سازی صحیح گام اول (نادیده گرفتن تگ `<base>`) یکی از رایج‌ترین و مخرب‌ترین باگ‌ها در توسعه خزشگرهای دانشجویی است که منجر به تولید هزاران آدرس اشتباه و در نهایت دریافت انبوهی از خطای **404 (Not Found)** از سمت سرورها می‌شود.

پاسخنامه مسئله 20.10: پروتکل فایل ربات‌ها (robots.txt)

مقدمه: پروتکل حذف روبات‌ها (REP)

در معماری وب، پیش از آنکه خزشگر واکنشی صفحات 1 دامنه جدید را آغاز کند، قانوناً و اخلاقاً موظف است به عنوان اولین درخواست شبکه، فایلی به نام `robots.txt` را از پوشه ریشه (**Root Directory**) آن سایت درخواست کند (مثلاً

این فایل متنی ساده، پیاده‌سازی عملی استاندارد به نام «پروتکل حذف روبات‌ها» (**Robots Exclusion Protocol**) است. این فایل به عنوان 1 قرارداد ارتباطی بین مدیران وب‌سایت‌ها و موتورهای جستجو عمل می‌کند تا مرزهای مجاز خزش را مشخص کند.

نقش فایل robots.txt در مکانیزم خزش

این فایل 4 نقش بنیادین در هدایت و کنترل رفتار خزشگرها ایفا می‌کند:

نقش عملکردی	شرح وظایف و کاربرد در معماری خزشگر
1 کنترل دسترسی (Access Control)	اصلی‌ترین وظیفه این فایل، ایجاد حریم خصوصی و جلوگیری از نمایه‌سازی صفحات غیرضروری است. با استفاده از دستور Disallow ، مدیر سایت می‌تواند بخش‌هایی مانند پوشه‌های مدیریتی (Admin Panels)، سبد خرید کاربران، یا صفحات جستجوی داخلی را از دید خزشگر پنهان کند. این کار مستقیماً از هدررفت منابع خزشگر در صفحات بی‌ارزش جلوگیری می‌کند.
2 هدف‌گیری اختصاصی (User-Agent)	فایل robots.txt می‌تواند سیاست‌های متفاوتی را برای خزشگرهای مختلف اعمال کند. به عنوان مثال، مدیر سایت با متغیر User-agent می‌تواند به خزشگر متنی گوگل (Googlebot) اجازه دسترسی کامل بدهد، اما خزشگر تصاویر گوگل (Googlebot-Image) یا خزشگرهای ابزارهای هوش مصنوعی را مسدود کند.
3 مدیریت ترافیک (Crawl-Delay)	در راستای ویژگی الزامی «ادب» (Politeness)، مدیر سایت می‌تواند با استفاده از دستور Crawl-delay از موتور جستجو بخواهد که بین هر 2 درخواست متوالی به سرور، مدت زمان مشخصی (مثلاً 10 ثانیه) مکث کند تا سرور دچار افت عملکرد یا اختلال (DoS) نشود.
4 معرفی نقشه سایت (Sitemap)	این فایل می‌تواند حاوی مسیر دقیق فایل XML نقشه سایت باشد. این کار به خزشگر کمک می‌کند تا به جای اتکا به کشف تصادفی پیوندها، 1 لیست ساختاریافته از تمام صفحات مهم سایت را به سرعت دریافت کند.

فرض خزشگر در صورت عدم وجود فایل (Missing robots.txt)

این بخش به یکی از مهم‌ترین پروتکل‌های تصمیم‌گیری در سیستم خزش می‌پردازد: اگر سایت این فایل را نداشته باشد چه می‌شود؟

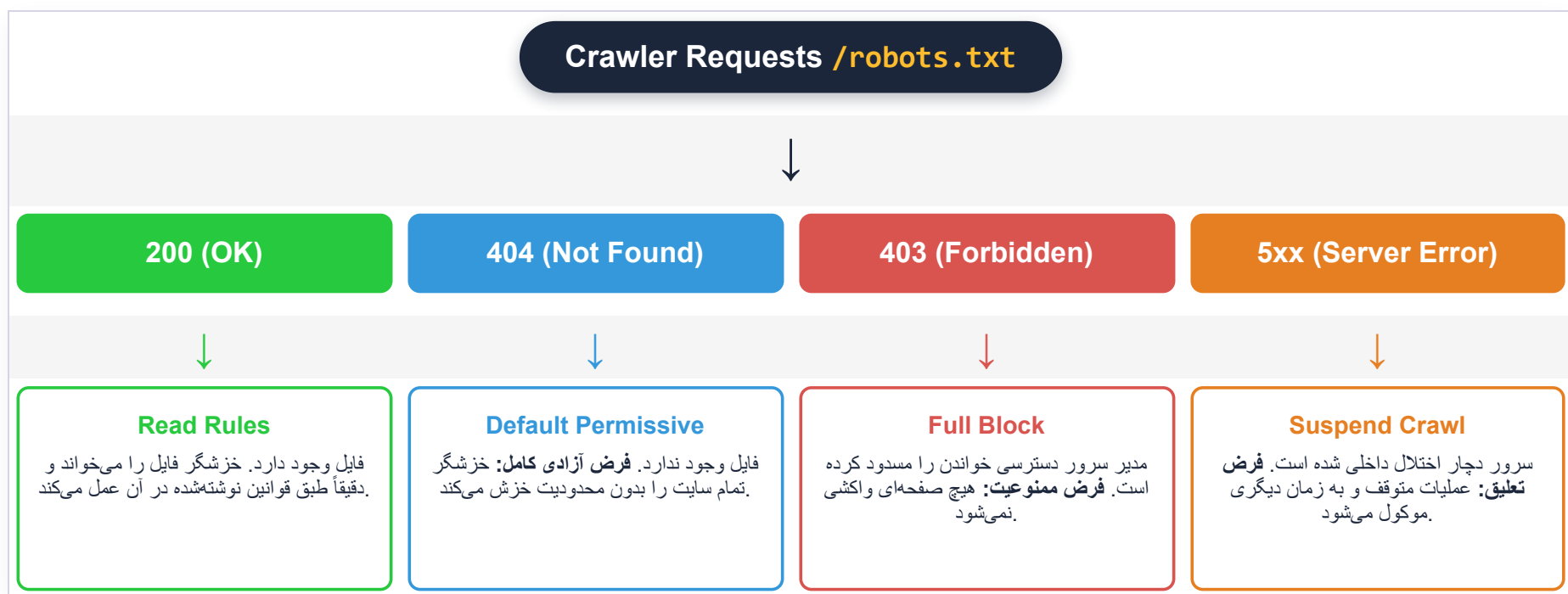
هنگامی که خزشگر درخواست دریافت این فایل را ارسال می‌کند، سرور میزبان کد وضعیت (**HTTP Status Code**) متفاوتی برمی‌گرداند که رفتار خزشگر را تعیین می‌کند. اگر فایل وجود نداشته باشد، سرور کد 404 (**Not Found**) را برمی‌گرداند. در این حالت، معماری خزشگر بر اساس سیاست «آزادی کامل» (**Default Permissive Policy**) عمل می‌کند.

فرض پیش‌فرض خزشگر:

"مدیر این سایت هیچ محدودیتی برای نمایه‌سازی محتوای خود قائل نشده است. بنابراین من مجاز هستم تمام صفحات، پوشه‌ها و پیوندهای این دامنه را واکشی و نمایه‌سازی کنم."

تحلیل آکادمیک: تفاوت کدهای وضعیت در رفتار خزشگر

برای تکمیل درک دانشجویان از این مکانیزم، توجه به تفاوت خطای 404 با سایر خطاها ضروری است. نمودار تصمیم‌گیری پایدار زیر، رفتار قطعی سیستم خزش را در مواجهه با پاسخ‌های مختلف سرور مدل‌سازی می‌کند:



نتیجه‌گیری پایانی:

فایل **robots.txt** ابزار اصلی مدیران شبکه برای کنترل خزشگرهاست. با این حال، در غیاب این فایل، معماری وب به‌گونه‌ای طراحی شده است که جریان آزاد اطلاعات متوقف نشود و خزشگرها سایت را به‌صورت پیش‌فرض، کاملاً عمومی و در دسترس تلقی کنند.

پاسخنامه مسئله 20.11: سیاست‌های واکنشی مجدد در سایت‌های خبری (Re-visiting Policies)

مقدمه: چالش خزش مستمر (Continuous Crawling)

در موتورهای جستجوی مدرن، واکنشی صفحات وب 1 فرآیند یک‌باره نیست. از آنجا که گراف وب دائماً در حال تغییر، حذف و به‌روزرسانی است، خزشگر باید به صورت پیوسته صفحات قبلی را بازبینی کند تا پایگاه داده یا نمایه (**Index**) موتور جستجو «تازگی» (**Freshness**) خود را حفظ کند.

وب‌سایت‌های خبری بارزترین مثال برای چالش نامتقارن بودن نرخ تغییرات هستند. خزشگر در مواجهه با خبرگزاری‌ها باید تعادلی ظریف ایجاد کند: اگر منابع خود را صرف واکنشی مداوم آرشیو کند، اخبار فوری (**Breaking News**) صفحه اصلی را از دست می‌دهد؛ و اگر تمام منابع را روی صفحه اصلی متمرکز کند، تغییرات احتمالی و اصلاحیه‌های مقالات قدیمی را نخواهد دید.

بخش 1) مدل‌سازی ریاضی نرخ تغییرات (فرآیند پواسون)

در ادبیات بازیابی اطلاعات (الگوریتم‌های **Cho and Garcia-Molina**)، تغییرات 1 صفحه وب معمولاً با استفاده از 1 فرآیند پواسون (**Poisson Process**) مدل‌سازی می‌شود. هر صفحه دارای 1 نرخ تغییر تخمینی به نام λ است:

صفحه اصلی خبرگزاری: دارای λ بسیار بالا (مثلاً تغییر در هر 5 دقیقه).

صفحه آرشیو سال گذشته: دارای λ بسیار پایین و نزدیک به صفر.

خزشگر این نرخ را به صورت پویا (**Dynamic**) و از طریق مشاهده رفتار صفحه در طول زمان یاد می‌گیرد. احتمال اینکه 1 صفحه پس از گذشت زمان t بدون تغییر باقی مانده باشد (معیار تازگی یا **Freshness**) از فرمول زیر محاسبه می‌گردد:

$$F(t) = P(\text{Unmodified in time } t) = e^{-\lambda t}$$

تحلیل نمودار و سن (Age):

هنگامی که صفحه وب در سرور تغییر می‌کند (**Modify**)، مقدار تازگی در پایگاه داده خزشگر به صفر سقوط می‌کند و سن صفحه (**Age**) به صورت خطی بالا می‌رود تا زمانی که خزشگر مجدداً آن صفحه را واکنشی و همگام‌سازی (**Sync**) کند.

بخش 2) مکانیزم زمان‌بندی در صف آدرس‌ها (URL Frontier)

برای مدیریت این صفحات متفاوت، معماری صف خزش از 1 سیستم ارزیابی مجدد و زمان‌بندی تاخیری (**Min-Heap Timer**) استفاده می‌کند. مسیر اجرای این الگوریتم به شرح زیر است:

مکانیزم سیستم و منطق تصمیم‌گیری	گام اجرایی
هر بار که صفحه‌ای واکنشی می‌شود، خزشگر محتوای جدید را هش (Hash) کرده و با هش قبلی مقایسه می‌کند: • اگر محتوا تغییر کرده باشد، بازه زمانی واکنشی بعدی (Fetch Interval) کاهش می‌یابد. • اگر محتوا تغییری نکرده باشد، خزشگر با استفاده از الگوریتم عقب‌نشینی نمایی (Exponential Backoff)، بازه زمانی را 2 برابر می‌کند.	گام 1 مقایسه هش (Change Detection)
به جای اینکه آدرس‌های واکنشی‌شده دوباره به انتهای صف عادی برگردند، ماژول زمان‌بند 1 فیلد T_{next} (زمان واکنشی بعدی) برای آن‌ها محاسبه می‌کند: • برای صفحه اصلی، T_{next} برابر با 5 دقیقه دیگر تنظیم می‌شود. • برای صفحه آرشیو، پس از چند بار واکنشی بدون تغییر، T_{next} برای 6 ماه آینده تنظیم می‌شود.	گام 2 برچسب زمان (Timestamping)
این آدرس‌ها وارد 1 صف اولویت‌دار مبتنی بر زمان (Priority Queue) می‌شوند. رشته‌های خزشگر (Crawl Threads) طوری برنامه‌ریزی شده‌اند که تحت هیچ شرایطی 1 آدرس را پیش از رسیدن زمان T_{next} آن واکنشی نکنند. به این ترتیب، صفحه آرشیو تا 6 ماه آینده اصلاً وارد چرخه پردازش شبکه نمی‌شود و منابع سیستم کاملاً آزاد می‌ماند.	گام 3 مسدودسازی (Min-Heap Block)

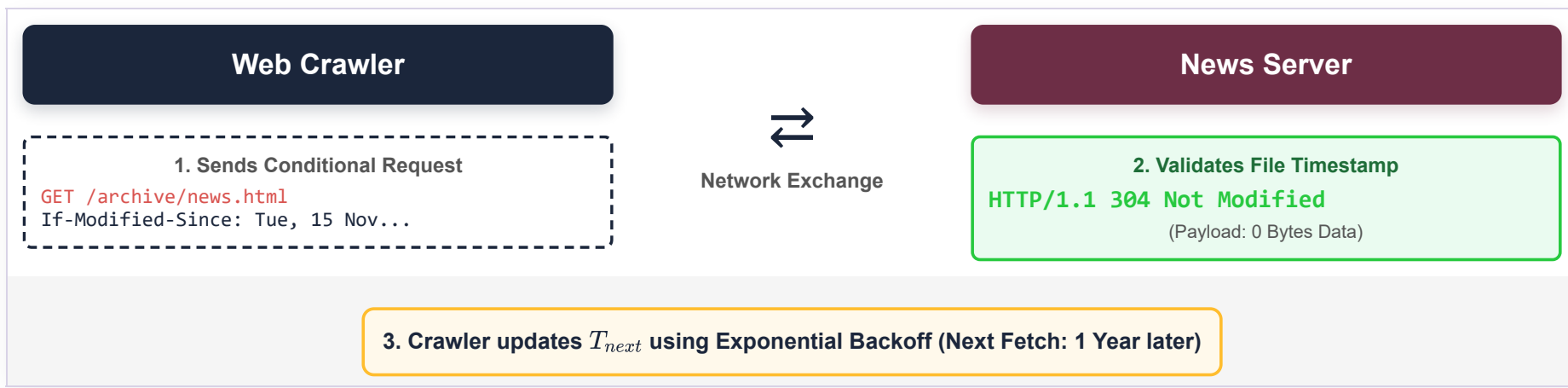
بخش 3) بهینه‌سازی ترافیک با درخواست‌های شرطی (Conditional HTTP)

حتی زمانی که نوبت واکنشی مجدد صفحه آرشیو پس از 6 ماه فرا می‌رسد، خزشگر هوشمند کل کد **HTML** را به صورت کورکورانه دانلود نمی‌کند. معماری خزشگر برای صفحات قدیمی از **هدرهای شرطی HTTP** استفاده می‌کند.

هنگام ارسال درخواست به سرور خبرگزاری، خزشگر اطلاعات زیر را در هدر (**Header**) قرار می‌دهد:

If-Modified-Since: آیا این صفحه از تاریخ [آخرین زمان خزش] تغییری کرده است؟

If-None-Match: آیا هش محتوای فعلی شما با این **ETag** (هش نسخه قبلی که من دارم) متفاوت است؟



خروجی این مکانیزم و نتیجه گیری:

اگر صفحه آرشیو واقعاً در این 6 ماه تغییری نکرده باشد، سرور خبرگزاری به جای ارسال چند مگابایت کدهای **HTML**، صرفاً 1 پاسخ خالی با کد وضعیت **304 Not Modified** برمی گرداند. این پاسخ چند بایتی، به خزشگر اطلاع می دهد که نسخه محلی اش هنوز کاملاً معتبر است و باید T_{next} را باز هم افزایش دهد. این تکنیک، صرفه جویی عظیمی در پهنای باند موتور جستجو و منابع پردازشی سرور میزبان ایجاد می کند.

پاسخنامه مسئله 20.12: مدیریت کش و همزمانی در DNS

مقدمه: ریشه مشکل در کش های استاندارد (Global Lock)

هنگامی که 500 نخ کارگر (**Thread**) در 1 خزشگر می خواهند نام های دامنه را به **IP** ترجمه کنند، استفاده از 1 کش استاندارد (مانند 1 **HashTable** یا **Dictionary** معمولی) باعث ایجاد گلوگاهی به نام **رقابت بر سر قفل (Lock Contention)** می شود.

در 1 کش استاندارد، ساختار داده با 1 «قفل سراسری» (**Global Mutex / Lock**) محافظت می شود تا از تداخل داده ها (**Race Condition**) هنگام نوشتن جلوگیری شود. این یعنی اگر نخ شماره 1 در حال نوشتن **IP** 1 دامنه در کش باشد، 499 نخ دیگر (حتی اگر بخواهند دامنه های کاملاً متفاوتی را بخوانند) باید متوقف (**Block**) شوند تا قفل آزاد شود. این اجرای ترتیبی، عملاً مزیت برنامه نویسی چندنخی (**Multi-threading**) را از بین برده و ماژول **DNS** را فلج می کند.

معماری پیشنهادی: کش قطعه بندی شده (Sharded / Partitioned Cache)

برای حل این بحران در سیستم های مقیاس پذیر، معماران موتور جستجو از الگوی کش **قطعه بندی شده (Sharded Cache)** یا ساختارهای داده همزمان (مانند **ConcurrentHashMap** در جاوا) استفاده می کنند.

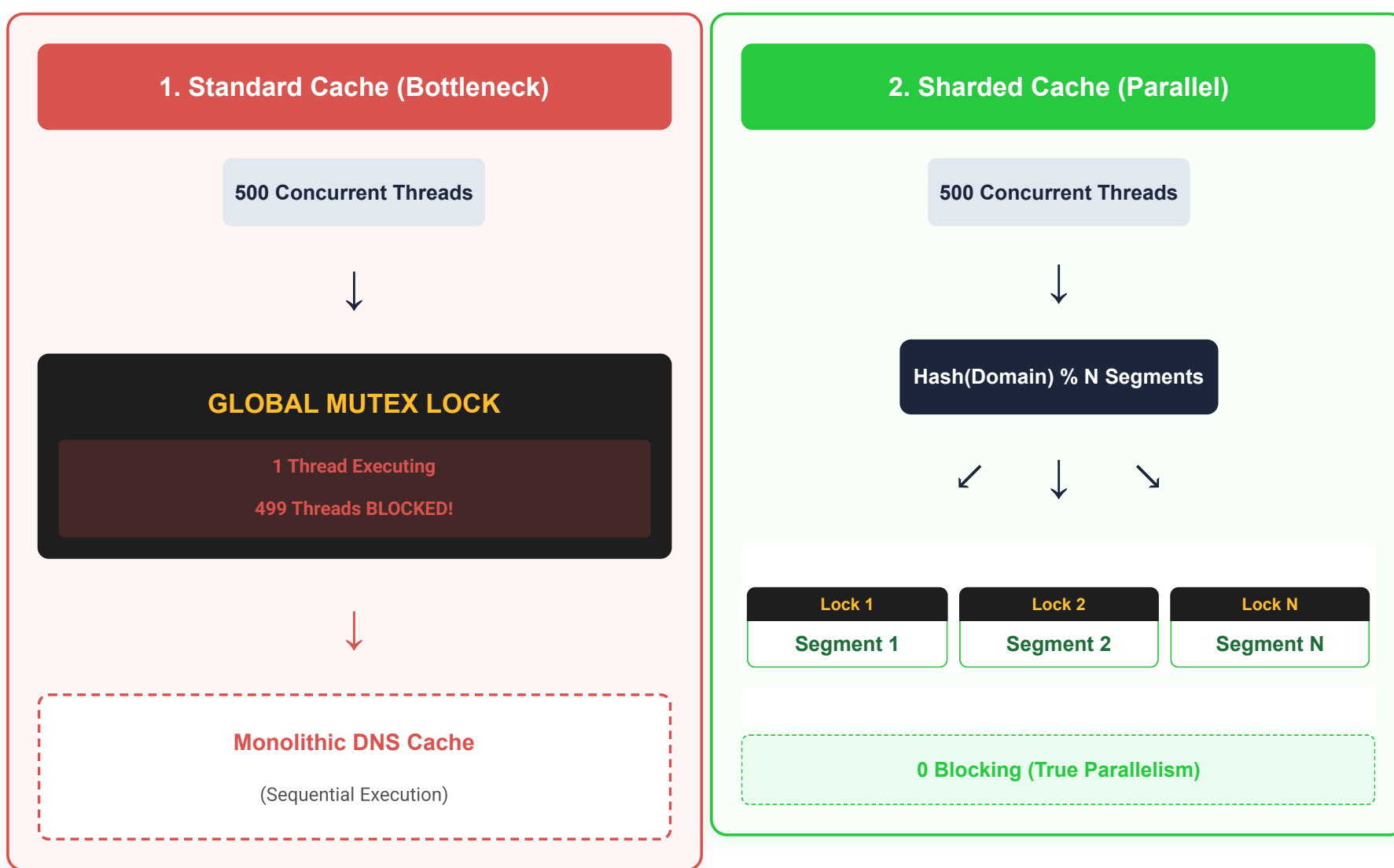
ایده اصلی این معماری، خرد کردن 1 کش یکپارچه به **N** بخش (**Segment**) کاملاً مستقل است که هر کدام قفل (**Lock**) اختصاصی خود را دارند. مسیر پردازش 1 درخواست **DNS** در این معماری طی مراحل زیر انجام می شود:

گام اجرایی	شرح عملکرد فنی و استراتژی مهندسی
گام 1 درهم سازی دامنه (Hashing)	توزیع بار ترافیکی: وقتی 1 نخ می خواهد IP دامنه <code>example.com</code> را پیدا کند، ابتدا رشته متنی دامنه را به 1 تابع هش (Hash Function) می دهد. خروجی این تابع، مشخص می کند که اطلاعات این دامنه دقیقاً در کدام قطعه (Segment) از کش قرار دارد (مثلاً قطعه شماره 7).

گام اجرایی	شرح عملکرد فنی و استراتژی مهندسی
گام 2 قفل‌گذاری دانه‌ریز (Fine-grained Locking)	حذف قفل سراسری: نخ کارگر مستقیماً به سراغ قطعه شماره 7 می‌رود. اگر نیاز به نوشتن داده جدیدی در کش باشد، فقط قفل همان قطعه شماره 7 را درگیر می‌کند. در همین زمان، 499 نخ دیگر می‌توانند بدون هیچ‌گونه توقفی، اطلاعات دامنه‌های دیگر را از سایر قطعات (مثلاً قطعه 2، 15 یا 30) بخوانند یا بنویسند.
گام 3 عملیات بدون قفل (Lock-free Reads)	معماری بدون توقف: در پیاده‌سازی‌های پیشرفته، حتی برای خواندن از 1 قطعه نیز قفلی ایجاد نمی‌شود. با استفاده از متغیرهای اتمیک (Atomic Variables) و عملیات مقایسه-و-جابه‌جایی (CAS - Compare-And-Swap) در سطح پردازنده، نخ‌ها می‌توانند رکوردهای DNS را به صورت کاملاً همزمان از حافظه بخوانند، مگر آنکه دقیقاً در همان کسر از ثانیه، ساختار داخلی آن قطعه در حال تغییر باشد.

شبیه‌ساز رقابت نخ‌ها (Thread Contention Simulator)

برای درک بهتر اینکه چگونه حذف قفل سراسری باعث افزایش چشمگیر توان عملیاتی (Throughput) می‌شود، ساختار شماتیک و پایدار زیر ترافیک 500 نخ همزمان را در هر 2 معماری مقایسه می‌کند:



جمع‌بندی:

در طراحی سیستم‌های توزیع‌شده مقیاس‌وب، قانون طلایی این است: "هیچ‌گاه نباید کل سیستم منتظر 1 عملیات محلی بماند". با تبدیل 1 کش واحد به آرایه‌ای از کش‌های کوچکتر و کاملاً مستقل (**Sharding**)، احتمال اینکه 2 نخ به طور همزمان به 1 قطعه یکسان برخورد کنند به شدت کاهش یافته و گلوگاه مهلك ماژول DNS به‌طور کامل برطرف می‌گردد.

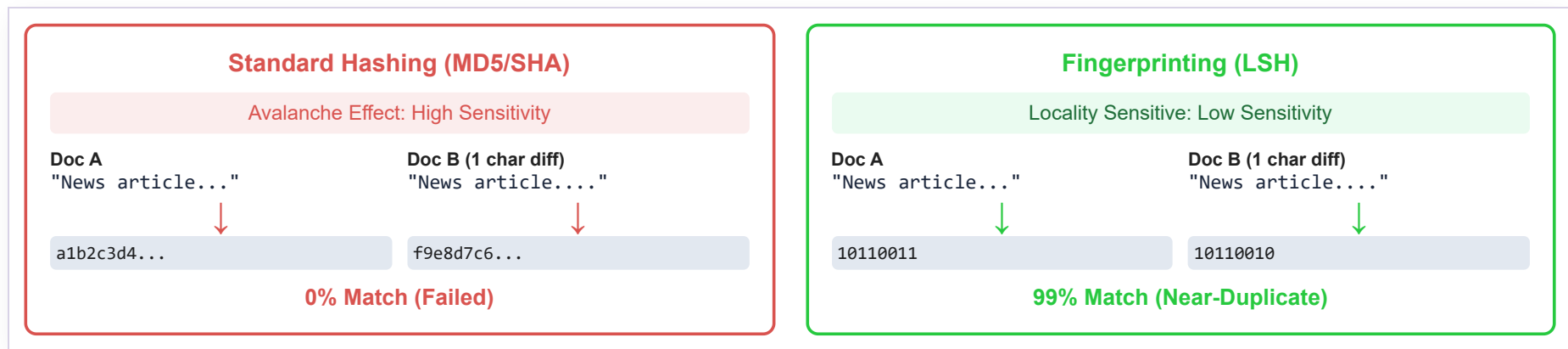
پاسخنامه مسئله 20.13: فیلتر کردن محتوای تکراری متنی (Near-Duplicate Detection)

مقدمه: ضعف توابع هش استاندارد در گراف وب

در محیط وب، پدیده «محتوای شبه تکراری» (**Near-Duplicate**) بسیار رایج است. برای مثال، 1 مقاله خبری ممکن است در چندین سایت منتشر شود، اما هر سایت قالب **HTML**، منوها، تبلیغات و هدرهای مخصوص به خود را دارد.

اگر خزشگر برای تشخیص تکراری بودن این صفحات از توابع هش رمزنگاری استاندارد مانند **MD5** یا **SHA-256** روی کل کد صفحه استفاده کند، با شکست قطعی مواجه می شود. دلیل این امر ویژگی «اثر بهمنی» (**Avalanche Effect**) در این توابع است؛ یعنی تغییر حتی 1 کاراکتر (مانند تغییر تاریخ در هدر سایت)، خروجی هش را کاملاً و به صورت تصادفی تغییر می دهد و 2 صفحه شبه تکراری را کاملاً متفاوت جلوه می دهد.

بنابراین، ما به الگوریتم هایی نیاز داریم که به جای «برابری دقیق»، «میزان شباهت متنی» را اندازه گیری کنند و به تغییرات جزئی (مانند تفاوت در کدهای **HTML** یا تبلیغات) حساسیت پایینی نشان دهند. الگوریتم های تولید اثر انگشت سند (**Document Fingerprinting**) دقیقاً برای همین هدف طراحی شده اند.



گام صفر: پیش پردازش و استخراج متن خالص

پیش از اجرای هرگونه الگوریتم اثر انگشت، موتور جستجو باید «نویزهای ساختاری» را حذف کند. در این مرحله، مازولی به نام **DOM Parser** تمام تگ های **HTML**، اسکریپت های جاوااسکریپت، منوهای ناوبری (**Boilerplates**) و تبلیغات را حذف می کند تا صرفاً **متن خالص (Plain Text)** مقاله باقی بماند. اما حتی پس از این کار، ممکن است تفاوت های جزئی در نگارش کلمات یا پاراگراف بندی وجود داشته باشد، که در مراحل بعدی حل می شود.

راهکار اول: رویکرد قطعه بندی (Shingling) و شباهت ژاکارد

این الگوریتم، متن را به قطعات کوچکی تقسیم می کند تا شباهت 2 سند را به صورت ریاضی مدل سازی کند. مراحل اجرای آن به شکل زیر است:

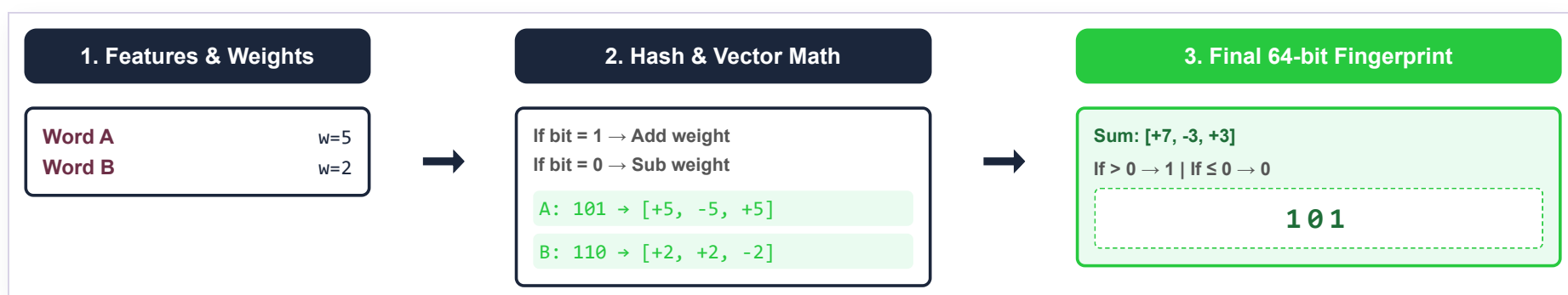
گام اجرایی	شرح عملکرد فنی و استراتژی مهندسی
گام 1 تولید قطعه (Shingling)	تبدیل متن به مجموعه ای از توکن ها: الگوریتم، متن خالص را به توالی های هم پوشان k -کلمه ای (یا k -کاراکتری) تقسیم می کند که به آن ها Shingle می گویند. مثال (برای $k = 3$ در جمله "این 1 متن آزمایشی است"): • قطعه 1: "این 1 متن" • قطعه 2: "1 متن آزمایشی" • قطعه 3: "متن آزمایشی است"
گام 2 شباهت ژاکارد (Jaccard Similarity)	مدل سازی ریاضی شباهت: اکنون هر سند به 1 مجموعه (Set) از این قطعات تبدیل شده است. شباهت 2 سند A و B از طریق فرمول ژاکارد محاسبه می شود: نسبت اشتراک قطعات به اجتماع آن ها.

شرح عملکرد فنی و استراتژی مهندسی	گام اجرایی
$J(A, B) = \frac{ A \cap B }{ A \cup B }$	
اگر این 2 سند، مقاله‌ای یکسان باشند که فقط 1 پاراگراف تبلیغاتی در یکی از آن‌ها اضافه شده است، اکثر Shingle ‌های آن‌ها یکسان خواهد بود و امتیاز ژاکارد عددی بالا (مثلاً 0.95) خواهد شد که از آستانه تکراری بودن عبور می‌کند.	
کاهش ابعاد برای مقیاس‌پذیری: محاسبه اشتراک مجموعه‌های بزرگ برای میلیاردها سند وب، از نظر پردازشی غیرممکن است. الگوریتم MinHash با استفاده از چندین تابع هش تصادفی، از مجموعه عظیم Shingle ‌های هر سند، 1 «امضای فشرده» (Signature) مثلاً با 100 عدد صحیح می‌سازد. از نظر اثبات ریاضی، شباهت ژاکارد بین 2 سند برابر است با احتمال برابر بودن اعداد در امضاهای MinHash آن‌ها. این تکنیک، پردازش را میلیون‌ها بار سریع‌تر می‌کند.	گام 3 فشرده‌سازی (MinHash)

راهکار دوم: الگوریتم Simhash (توسعه‌یافته توسط گوگل)

الگوریتم **Simhash** که به خانواده «هش‌های حساس به مکان» (**Locality Sensitive Hashing - LSH**) تعلق دارد، رویکرد متفاوتی را در پیش می‌گیرد. هدف **Simhash** این است که کل سند را به 1 عدد 64 بیتی (اثر انگشت) تبدیل کند؛ با این ویژگی جادویی که: اسناد شبیه به هم، اعداد 64 بیتی شبیه به هم تولید کنند.

معماری و نحوه عملکرد الگوریتم Simhash Engine



محاسبه شباهت (فاصله همینگ)

برای مقایسه 2 سند، سیستم صرفاً **فاصله همینگ (Hamming Distance)** بین 2 عدد 64 بیتی را محاسبه می‌کند (یعنی تعداد بیت‌هایی که با هم تفاوت دارند، که از طریق عملگر منطقی سریع **XOR** در سطح پردازنده انجام می‌شود).

استاندارد عملیاتی گوگل:

در موتور جستجوی گوگل، اگر فاصله همینگ 2 سند کمتر یا مساوی 3 بیت (از 64 بیت کل) باشد، آن 2 صفحه، شبه‌تکراری (**Near-Duplicate**) در نظر گرفته شده و فقط نسخه معتبرتر در نمایه باقی می‌ماند. این تغییرات جزئی 3 بیتی، دقیقاً همان تغییرات در تبلیغات، ساختار قالب یا هدرهای **HTML** هستند که اثر آن‌ها در بردار نهایی کاملاً خنثی شده است.

پاسخنامه مسئله 20.14: مقایسه پارتیشن‌بندی در کوئری‌های پیچیده (Query Routing & Partitioning)

مقدمه: کالبدشکافی پرسمان‌های چندکلمه‌ای

در موتورهای جستجو، کوئری «دانشگاه صنعتی» 1 پرسمان عطفی (**AND Query**) شامل 2 واژه مستقل است: t1 = دانشگاه ,

t2 = صنعتی

برای پاسخ به این پرسمان، موتور جستجو باید لیست اسناد حاوی کلمه اول (**Posting List 1**) را با لیست اسناد حاوی کلمه دوم (**Posting List 2**) اشتراک‌گیری (**Intersection**) کند تا اسنادی که هر 2 کلمه را دارند شناسایی شوند. در مقیاس 100 میلیون سند، طول این لیست‌های پستینگ می‌تواند به میلیون‌ها شناسه سند (**DocID**) برسد.

در ادامه، مسیر اجرای این پرسمان را در 2 معماری متفاوت تحلیل می‌کنیم. فرض می‌کنیم خوشه (**Cluster**) ما شامل N سرور (گره) است و کاربر k نتیجه برتر (مثلاً 10 نتیجه اول) را درخواست کرده است.

سناریوی 1: معماری توزیع سند-محور (Document-Partitioned)

در این معماری (که به آن نمایه‌سازی محلی یا **Local Indexing** نیز می‌گویند)، 100 میلیون سند به طور مساوی بین N سرور تقسیم شده‌اند. هر سرور 1 نمایه کامل اما فقط برای اسناد خودش (مثلاً $\frac{100}{N}$ میلیون سند) دارد. کلمات «دانشگاه» و «صنعتی» در دیکشنری تمام سرورها وجود دارند.

مسیر اجرای پرسمان و تجمیع نتایج:

مسیریابی همه‌پخشی (Broadcast Routing)	گره مرکزی (Broker) کوئری را دریافت کرده و آن را همزمان به تمام N سرور ارسال می‌کند.
پردازش محلی (Local Processing)	هر سرور به صورت کاملاً مستقل و روی حافظه RAM خود، لیست پستینگ محلی «دانشگاه» را با لیست محلی «صنعتی» تقاطع می‌دهد.
رتبه‌بندی محلی (Local Ranking)	هر سرور اسناد مشترک یافته‌شده را امتیازدهی (مثلاً با الگوریتم BM25) کرده و فقط k نتیجه برتر خود را استخراج می‌کند.
تجمیع نتایج (Result Aggregation)	هر سرور k نتیجه محلی خود (شامل DocID و امتیاز) را به گره مرکزی می‌فرستد. گره مرکزی اکنون $N \times k$ نتیجه دارد. آن‌ها را بر اساس امتیاز مرتب کرده و k نتیجه برتر نهایی (Global Top-k) را به کاربر نمایش می‌دهد.

سناریوی 2: معماری توزیع واژه-محور (Term-Partitioned)

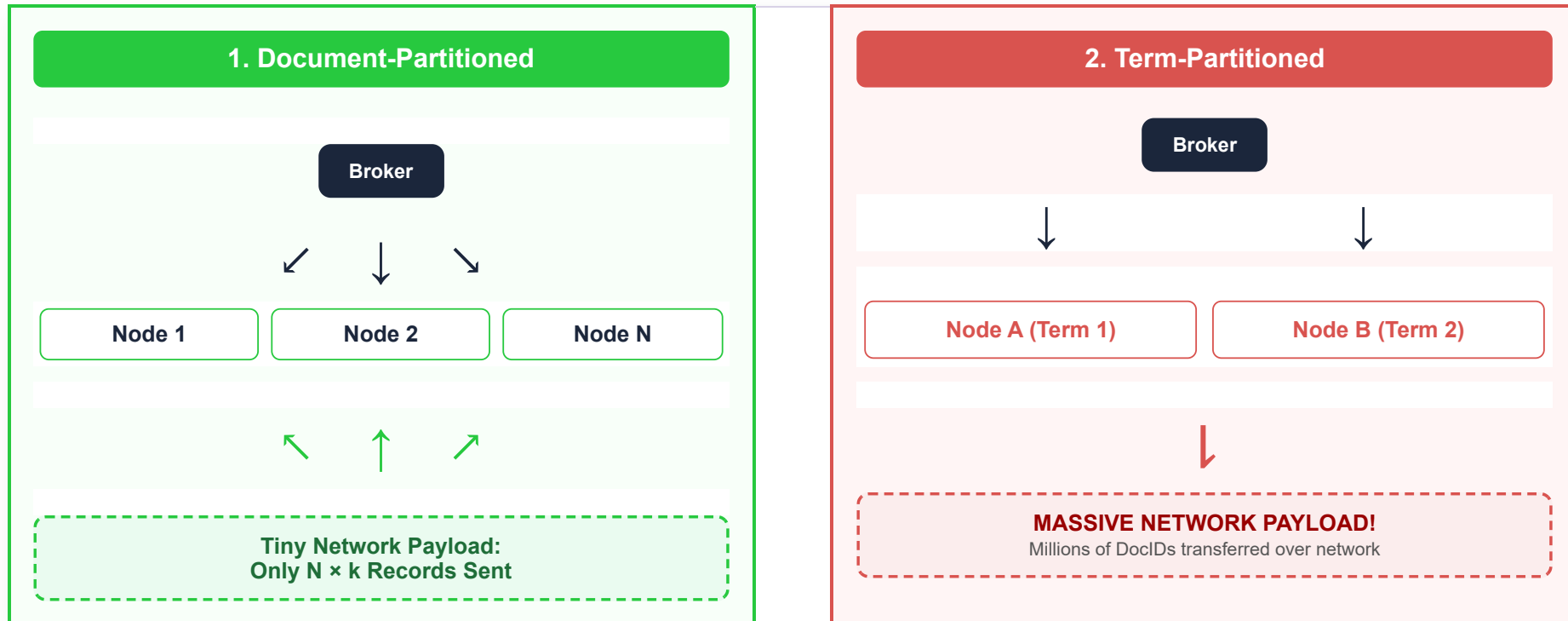
در این معماری (نمایه‌سازی سراسری یا **Global Indexing**)، 100 میلیون سند تقسیم نمی‌شوند، بلکه دامنه لغات (**Vocabulary**) توزیع می‌شود. فرض کنید بر اساس توابع درهم‌سازی (**Hash**)، کلمه «دانشگاه» (t_1) به همراه لیست پستینگ کاملش (در کل 100 میلیون سند) در سرور **A** قرار گرفته است و کلمه «صنعتی» (t_2) با لیست کاملش در سرور **B** ذخیره شده است.

مسیر اجرای پرسمان و تجمیع نتایج:

مسیریابی هدفمند (Directed Routing)	گره مرکزی بررسی می‌کند که این 2 کلمه در کدام سرورها هستند و درخواست را مستقیماً به سرور A و سرور B ارسال می‌کند (به سایر سرورها کاری ندارد).
بحران اشتراک‌گیری (Intersection Bottleneck)	برای یافتن اسنادی که هر 2 کلمه را دارند، این 2 سرور باید اطلاعات خود را با هم ترکیب کنند. از آنجا که داده‌ها در 2 ماشین فیزیکی جداگانه هستند، سرور A باید کل لیست پستینگ خود را از طریق کابل شبکه برای سرور B ارسال کند (یا بالعکس، هر کدام که لیست کوتاه‌تری دارد).
پردازش نهایی (Final Processing)	سرور B پس از دریافت لیست میلیونی از سرور A ، آن‌ها را با لیست خودش تقاطع داده، اسناد را امتیازدهی می‌کند.

شبیه‌ساز پیشرفته جریان شبکه (Network Payload Simulator)

مدل‌سازی پایدار زیر تفاوت فاحش مسیر انتقال داده‌ها و گلوگاه ترافیکی شبکه را در این 2 معماری در هنگام پردازش 1 پرسمان 2 کلمه‌ای به تصویر می‌کشد:



مقایسه ریاضی و مهندسی: کدام معماری ترافیک کمتری دارد؟

پاسخ قطعی: معماری توزیع سند-محور (Document-Partitioned) ترافیک شبکه بسیار کمتری ایجاد می‌کند.

برای اثبات این موضوع، حجم داده‌های منتقل‌شده در بستر شبکه (Network Payload) را به صورت ریاضی مقایسه می‌کنیم:

شاخص ترافیک شبکه	معماری توزیع سند-محور	معماری توزیع واژه-محور
حجم داده ارسالی از سرورها	$N \times k$	$\min(PL(t_1) , PL(t_2))$
مثال عددی (برای $k = 10$)	اگر 10 سرور داشته باشیم، مجموعاً 100 رکورد (شامل شناسه سند و امتیاز که در حد چند کیلوبایت است) در شبکه جابجا می‌شود.	کلمه «دانشگاه» ممکن است در 10 میلیون سند و «صنعتی» در 5 میلیون سند وجود داشته باشد. سرورها باید 5 میلیون شناسه سند (معادل ده‌ها مگابایت داده خالص) را در کسری از ثانیه در شبکه منتقل کنند.

نکته کلیدی:

در پرسمان‌های تک‌کلمه‌ای (Single-term Queries)، معماری Term-partitioned ترافیک کمتری دارد؛ اما از آنجا که تقریباً تمام جستجوهای واقعی کاربران در موتورهای جستجو چندکلمه‌ای (Multi-term) هستند، ترافیک شبکه‌ی Term-partitioned به صورت تصاعدی بالا می‌رود. انتقال میلیون‌ها بایت داده در شبکه برای هر کوئری، باعث ایجاد گلوگاه شبکه (Network Bottleneck) و افت شدید سرعت پاسخ‌گویی می‌شود. به همین دلیل، تمام موتورهای جستجوی مدرن مقیاس‌پذیر، منحصراً از معماری توزیع سند-محور (Document-partitioned) استفاده می‌کنند.

پاسخنامه مسئله 20.15: استراتژی‌های صف جلو و عقب در مرز آدرس‌ها

(Front & Back Queues Strategy)

مقدمه: تقابل اولویت و ادب در خزشگر

یکی از زیباترین طراحی‌ها در معماری موتورهای جستجو (معماری **Mercator**)، حل تعارض ذاتی بین «اهمیت 1 صفحه» (**Priority**) و «ظرفیت سرور میزبان» (**Politeness**) است.

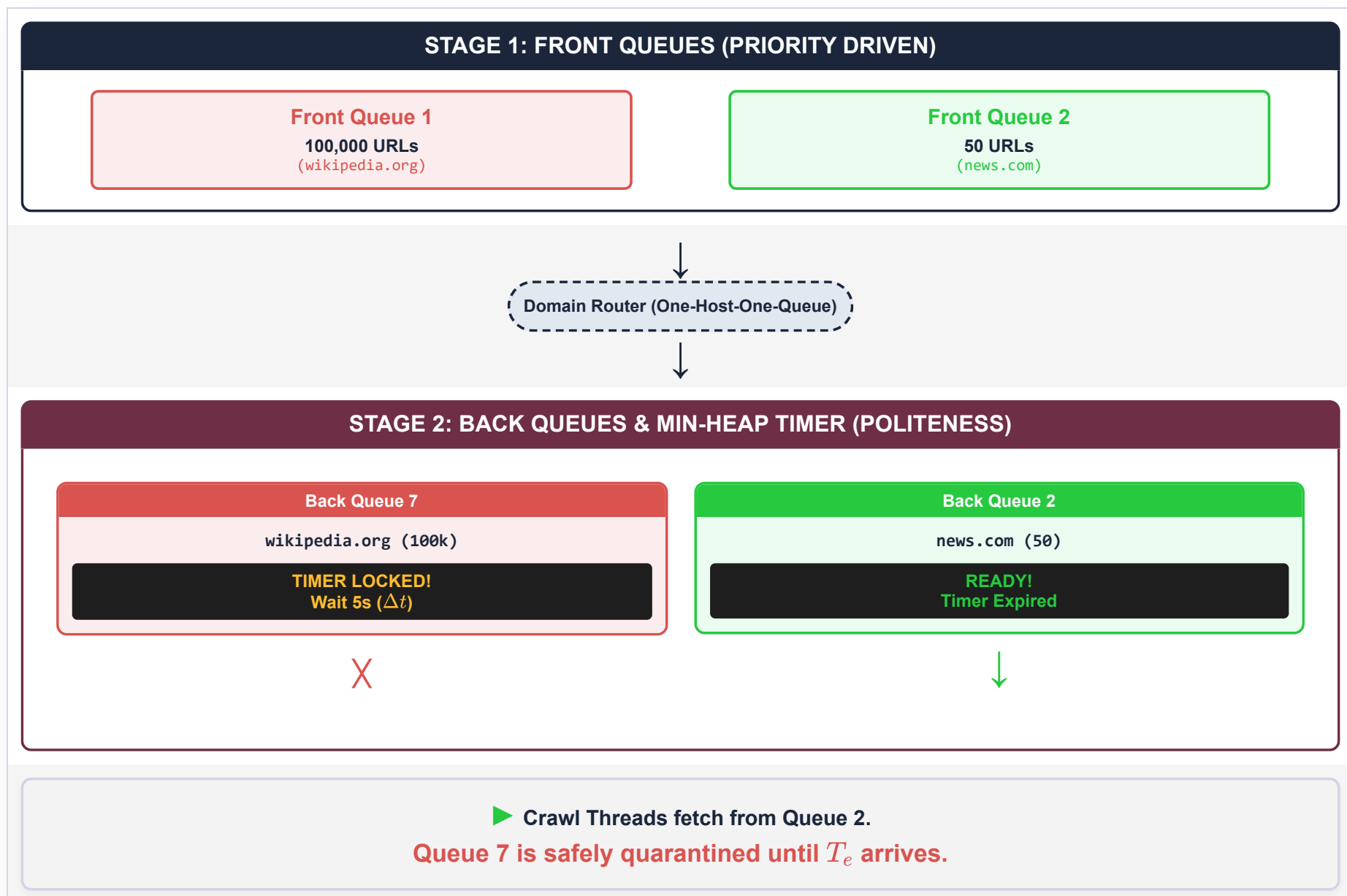
در صورت مسئله، با 1 سناریوی بحرانی مواجه هستیم: فرض کنید سایت 1 **wikipedia.org** سایت با اولویت بسیار بالا (**High Priority**) است. سیستم استخراج پیوند در 1 لحظه، 100,000 آدرس جدید از ویکی‌پدیا کشف می‌کند. از آنجا که این سایت اهمیت بالایی دارد، الگوریتم اولویت‌بند تمام این 100,000 آدرس را در بالاترین صف جلویی (مثلاً **Front Queue 1**) قرار می‌دهد.

خطر بمباران سرور:

اگر خزشگر صرفاً بر اساس اولویت کار می‌کرد، صدها نخ پردازشی (**Thread**) همزمان به این صف هجوم برده و با ارسال 100,000 درخواست متوالی، سرور ویکی‌پدیا را از کار می‌انداختند (حمله **DoS**). اما مکانیزم صف‌های پشتی (**Back Queues**) به عنوان 1 «دیواره آتشی زمان‌بندی‌شده» مانع از این فاجعه می‌شود.

شبیه‌ساز معماری: مکانیزم مهار سیلاب آدرس‌ها

نمودار پایدار زیر نشان می‌دهد که چگونه معماری 2 مرحله‌ای، ترافیک عظیم 1 دامنه را قرنطینه کرده و همزمان اجازه می‌دهد پردازش سایر دامنه‌ها متوقف نشود:



برای درک اینکه چگونه معماری **URL Frontier** این سیلاب آدرس‌های با اولویت بالا را مهار می‌کند، مسیر حرکت این داده‌ها را

در 1 فرآیند 4 مرحله‌ای تحلیل می‌کنیم:

گام اجرایی	شرح عملکرد و الگوریتم کنترلی
گام 1 مکش آدرس‌ها (Front Queues)	خروج سریع بر اساس اولویت: الگوریتم انتخاب‌گر (Biased Selector) به دلیل اولویت بالای Front Queue 1 ، به سرعت شروع به استخراج آدرس‌های <code>wikipedia.org</code> از این صف می‌کند. در این مرحله، سیستم کاملاً بر اساس اولویت عمل کرده و آدرس‌ها را به سمت لایه پایینی (مسیریاب) پمپاژ می‌کند.
گام 2 تجمیع دامنه (Back Queue Allocation)	قانون تخصیص یک‌به‌یک: در اینجا اصلی‌ترین قانون معماری اجرا می‌شود: «هر صف پشتی در هر لحظه فقط متعلق به 1 دامنه است.» مسیریاب دامنه (Router) متوجه می‌شود که تمام این 100,000 آدرس متعلق به دامنه 1 یکسانی هستند. بنابراین، به جای پخش کردن آن‌ها بین صف‌های پشتی مختلف، تمامی آن‌ها را در Back Queue 1 اختصاصی (مثلاً صف شماره 7) تلمبار می‌کند. در نتیجه، سیلاب آدرس‌ها در 1 کانال ایزوله مهار می‌شود.
گام 3 اعمال تاخیر (Min-Heap Timer)	استفاده از هرم حداقل: صف پشتی شماره 7 اکنون پر از آدرس‌های مهم است، اما خزشگرها مستقیماً از آن نمی‌خوانند. هر صف پشتی 1 ویژگی به نام «زمان مجاز بعدی» (T_e) دارد که توسط 1 ساختار داده هرم حداقل (Min-Heap) مدیریت می‌شود. هنگامی که 1 نخ کارگر اولین آدرس را از این صف برداشته و واکنشی می‌کند، سیستم محاسبه زیر را انجام می‌دهد:
	$T_e = t_c + \Delta t$
	که در آن t_c زمان حال و Δt تاخیر مودبانه (مثلاً 5 ثانیه) است. صف شماره 7 با برچسب زمانی 5 ثانیه در آینده، به درون هرم بازگردانده می‌شود.
گام 4 هدایت نخ‌ها (Thread Re-routing)	تغییر مسیر توان پردازشی: هرم مینیمم فقط صف‌هایی را در بالاترین گره خود (Root) قرار می‌دهد که T_e آن‌ها کوچکتر یا مساوی زمان حال باشد. تا زمانی که این 5 ثانیه سپری نشود، صف شماره 7 در اعماق هرم مدفون می‌ماند. در این مدت 5 ثانیه، صدها نخ پردازشی خزشگر بیکار نمی‌مانند؛ آن‌ها از بالای هرم، صف‌های پشتی دیگری را (که حاوی آدرس‌هایی از سایت‌های خبرگزاری، دانشگاهی و غیره هستند) برداشته و پردازش می‌کنند.

تحلیل:

نتیجه نهایی این معماری بسیار هوشمندانه است: آدرس‌های سایت با اولویت بالا به سرعت از صف‌های جلو به صف‌های عقب منتقل می‌شوند تا مسیر برای سایر آدرس‌های مهم باز بماند، اما در صف‌های عقب متوقف و ایزوله می‌شوند تا به صورت قطره‌چکانی (**Rate-limited**) وارد شبکه شوند.

این مکانیزم تضمین می‌کند که:

- ادب سرور:** سرور سایت محبوب هرگز در هر Δt ثانیه، بیش از 1 درخواست دریافت نمی‌کند (جلوگیری از بمباران).
- استفاده بهینه از منابع:** نخ‌های خزشگر منتظر باز شدن قفل ویکی‌پدیا نمی‌مانند، بلکه در زمان تاخیر، توان خود را صرف خزش دامنه‌های دیگر می‌کنند.

بخش 1) محاسبه حافظه مورد نیاز برای ماتریس مجاورت

در نظریه گراف، ماتریس مجاورت (**Adjacency Matrix**) 1 آرایه دو بعدی $N \times N$ است که در آن N برابر با تعداد کل گره‌ها (صفحات وب) است. اگر صفحه i به صفحه j پیوند (**Link**) داشته باشد، سلول (i, j) مقدار 1 و در غیر این صورت مقدار 0 می‌گیرد.

برای محاسبه حافظه مورد نیاز در مقیاس وب، گام‌های ریاضی زیر را طی می‌کنیم:

گام 1 تعداد کل صفحات	$N = 10,000,000,000 = 10^{10}$ Nodes
گام 2 تعداد کل سلول‌ها	Total Cells = $N \times N = (10^{10})^2 = 10^{20}$ Cells
گام 3 محاسبه حافظه خام	از آنجا که هر سلول فقط عدد 0 یا 1 را ذخیره می‌کند، در بهینه‌ترین حالت سخت‌افزاری، هر سلول دقیقاً به 1 بیت حافظه نیاز دارد. $\text{Memory (Bytes)} = \frac{10^{20} \text{ bits}}{8} = 1.25 \times 10^{19} \text{ Bytes}$
گام 4 تبدیل به مقیاس	با تقسیم این عدد بر مقیاس‌های استاندارد ذخیره‌سازی، به ارقام خیره‌کننده‌ای می‌رسیم: Gigabytes (GB): 1.25×10^{10} GB Petabytes (PB): 12,500 PB Exabytes (EB): 12.5 EB

نتیجه‌گیری بخش اول (فاجعه مهندسی):

ذخیره ماتریس مجاورت برای 10 میلیارد صفحه، نیازمند 12.5 اگزابایت حافظه RAM است. برای درک این محدودیت، کافی است بدانید که این حجم، معادل ظرفیت ذخیره‌سازی میلیون‌ها سرور پیشرفته است و بارگذاری آن در حافظه اصلی 1 موتور جستجو از نظر فیزیکی و اقتصادی کاملاً غیرممکن است.

بخش 2) چرا از ماتریس استفاده نمی‌شود؟ (پدیده گراف‌های تُنک)

دلیل اصلی شکست ماتریس مجاورت، ویژگی ساختاری وب به نام تُنک بودن (**Sparsity**) است.

در 1 ماتریس مجاورت، ما فضای حافظه را صرف ذخیره کردن «عدم وجود پیوند» می‌کنیم. میانگین پیوندهای خروجی (**Out-degree**) در 1 صفحه وب استاندارد معمولاً بین 15 تا 30 پیوند است. این یعنی در هر سطر از این ماتریس 10 میلیاردی، نهایتاً 30 سلول دارای عدد 1 هستند و 9,999,999,970 سلول دیگر با صفر پر شده‌اند.

استفاده از ماتریس به این معناست که ما میلیاردها دلار هزینه سخت‌افزار می‌کنیم تا صرفاً میلیون‌ها ترابایت عدد «صفر» را در حافظه نگه داریم.

مدل‌سازی شماتیک: ماتریس مجاورت در برابر لیست مجاورت

ADJACENCY MATRIX (WASTEFUL)	ADJACENCY LIST (OPTIMAL)
N1	
N2	
N3	
N4	
N5	

N1	0	1	0	0	1
N2	0	0	1	0	0
N3	1	0	0	0	0

99.999% of memory is wasted storing '0's (Non-links).

N1	→	[N2, N5]
N2	→	[N3]
N3	→	[N1]

Stores ONLY existing links.
Memory depends on edges, not N^2 .

بخش 3) رویکرد لیست مجاورت فشرده‌شده (Compressed Adjacency List)

برای حل این محدودیت مطلق، سرورهای اتصال (**Connectivity Servers**) ساختار داده را از ماتریس به لیست مجاورت (**Adjacency List**) تغییر می‌دهند.

در این روش، به جای اختصاص 1 سطر کامل به هر صفحه، 1 آرایه پویا ایجاد می‌شود که فقط شناسه (**ID**) صفحاتی را ذخیره می‌کند که پیوندی به آن‌ها وجود دارد.

تحلیل هزینه در معماری لیست مجاورت:

بیاپید همان گراف 10 میلیاردی را با فرض میانگین 20 پیوند خروجی برای هر صفحه، با معماری لیست مجاورت محاسبه کنیم:

مرحله 1 تعداد کل پیوندها	$\text{Total Links} = 10^{10} \text{ Nodes} \times 20 \text{ Links/Node} = 2 \times 10^{11} \text{ Links}$
مرحله 2 حجم حافظه خام	اگر هر شناسه سند (DocID) به 1 عدد صحیح 4 بایتی (32 بیتی) نیاز داشته باشد: $\text{Raw Memory} = 2 \times 10^{11} \times 4 \text{ Bytes} = 8 \times 10^{11} \text{ Bytes} \approx 800 \text{ GB}$

جدول مقایسه نهایی معماری‌های ذخیره‌سازی

شاخص ارزیابی	ماتریس مجاورت	لیست مجاورت خام	لیست مجاورت فشرده‌شده (استاندارد موتور جستجو)
نوع ذخیره‌سازی داده‌ها	ذخیره وجود و عدم وجود پیوند	ذخیره شناسه‌های متصل	ذخیره فاصله‌ها (Gaps) با الگوریتم‌های متراکم
حافظه مورد نیاز برای گراف 10 میلیاردی	12.5 EB ($12.5 \times 10^9 \text{ GB}$)	~ 800 GB	< 100 GB
امکان اجرا در حافظه RAM (در 1 کلاستر کوچک)	غیرممکن	کاملاً ممکن	بسیار ایده‌آل و سریع

جمع‌بندی نهایی:

همان‌طور که در جدول مشاهده می‌شود، با تغییر معماری به لیست مجاورت و اعمال الگوریتم‌های فشرده‌سازی تخصصی وب (نظیر **Boldi-Vigna** که با بهره‌گیری از تشابه صفحات، حجم هر پیوند را از 32 بیت به حدود 3 بیت کاهش می‌دهد)، موتور جستجو می‌تواند کل توپولوژی وب را در کمتر از 100 گیگابایت حافظه رم سرورهای اتصال بارگذاری کرده و الگوریتم‌های تکرارشونده‌ای مانند **PageRank** را در کسری از ثانیه روی کل گراف اجرا کند.

مقدمه: تله‌های عنکبوتی و گراف‌های چرخشی

گراف وب 1 ساختار داده درختی و منظم نیست، بلکه 1 گراف جهت‌دار به شدت درهم‌تنیده است. یکی از رایج‌ترین چالش‌ها در این فضا، وجود حلقه‌های بی‌نهایت (Cycles) است. این حلقه‌ها گاهی به صورت عمدی توسط هکرها برای به دام انداختن خزشگرها (Spider Traps) ایجاد می‌شوند و گاهی کاملاً تصادفی و ناشی از معماری سیستم‌های مدیریت محتوا (CMS) هستند.

اگر دایرکتوری A به B، B به C، و مجدداً C به A پیوند داشته باشد، 1 خزشگر ساده و بدون حافظه، تا ابد در این مثلث گرفتار می‌شود و منابع شبکه و پردازشگر موتور جستجو را هدر می‌دهد. برای جلوگیری از این فاجعه، قطعه‌ای ساختاری و حیاتی به نام ماژول آدرس‌های دیده‌شده (URL Seen) به عنوان 1 فیلتر یا دروازه‌بان در معماری خزشگر تعبیه شده است.

بخش 1) مکانیزم شکستن حلقه با ماژول URL Seen

ماژول URL Seen 1 پایگاه داده بسیار سریع است که تاریخچه‌ای از تمام آدرس‌هایی که تا این لحظه کشف شده‌اند را در خود نگه می‌دارد.

قانون طلایی دروازه بان:

«هیچ آدرسی حق ورود به صف خزش (URL Frontier) را ندارد، مگر اینکه قبلاً در ماژول URL Seen ثبت نشده باشد.»

برای درک دقیق عملکرد این ماژول، مسیر اجرای سناریوی چرخشی $A \rightarrow B \rightarrow C \rightarrow A$ را مرحله به مرحله بررسی می‌کنیم:

گام اجرایی	تحلیل فرآیند کشف و فیلترینگ
گام 1 پردازش گره A (کشف اولیه)	خزشگر صفحه A را واکنشی می‌کند. بلافاصله آدرس A را در ماژول URL Seen ثبت می‌کند. سپس پیوندهای داخل صفحه A را استخراج می‌کند و به آدرس B می‌رسد. از ماژول می‌پرسد: «آیا B را قبلاً دیده‌ایم؟» پاسخ منفی (Miss) است. پس B در ماژول ثبت شده و برای واکنشی به صف خزش ارسال می‌شود.
گام 2 پردازش گره B (پیمایش عمقی)	رشته کارگر (Worker Thread) صفحه B را از صف برداشته و واکنشی می‌کند. در بدنه صفحه B، پیوند C را کشف می‌کند. مجدداً از ماژول URL Seen استعلام می‌گیرد. چون C جدید است، ماژول آن را در خود ثبت کرده و به صف خزش اضافه می‌کند.
گام 3 پردازش گره C (برخورد با حلقه)	اکنون خزشگر صفحه C را واکنشی می‌کند. در کدهای این صفحه، به پیوند A می‌رسد (تکمیل حلقه). خزشگر آدرس A را استخراج کرده و پیش از هر کاری آن را به ماژول URL Seen ارسال می‌کند.
گام 4 انسداد حلقه (Drop & Rescue)	ماژول URL Seen آدرس A را در تاریخچه خود جستجو می‌کند. این بار پاسخ مثبت (Hit) است؛ سیستم به یاد می‌آورد که در گام اول صفحه A را پردازش کرده است. بنابراین، ماژول آدرس A را دور می‌ریزد (Drop) و اجازه نمی‌دهد وارد صف خزش شود. حلقه شکسته می‌شود و خزشگر به سراغ شاخه‌های دیگر گراف می‌رود.

Extracted Link	URL Seen Module (Memory)	Action Taken
A	Query: 'A' → Not Found (MISS)	Add to Frontier
B	Query: 'B' → Not Found (MISS)	Add to Frontier
C	Query: 'C' → Not Found (MISS)	Add to Frontier
A (Cycle Detected)	Query: 'A' → FOUND! (HIT)	DROP URL (Break Loop)

بخش 2) چالش مهندسی: ساختار داده در مقیاس وب

در 1 مسئله دانشگاهی، جستجو در 1 لیست 4 تایی بسیار ساده است. اما در مقیاس موتورهای جستجوی واقعی که با ده‌ها میلیارد URL سروکار دارند، ماژول **URL Seen** با چالش فلج‌کننده‌ای به نام «محدودیت حافظه» روبرو می‌شود.

اگر طول میانگین هر URL را 100 بایت در نظر بگیریم، ذخیره 10,000,000,000 آدرس در 1 ساختار داده ساده (مثل Hash Set) به حدود 1 ترابایت حافظه RAM نیاز دارد که دسترسی به آن در هر میلی‌ثانیه برای سرورها بسیار پرهزینه و نامعقول است.

راهکار صنعتی: فیلتر بلوم (Bloom Filter)

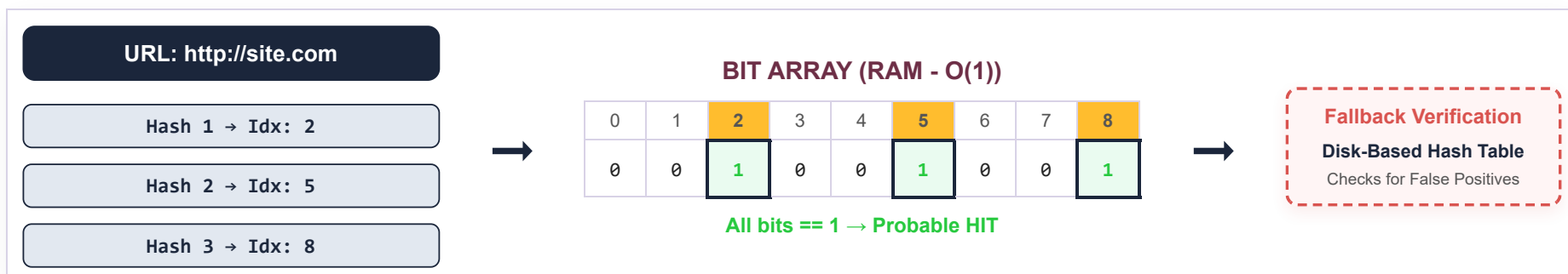
برای حل این چالش، معماران موتور جستجو از 1 ساختار داده احتمالاتی فوق‌سریع و کم‌حجم به نام **فیلتر بلوم** استفاده می‌کنند.

نحوه کار: فیلتر بلوم به جای ذخیره رشته متنی آدرس‌ها، از 1 آرایه بیتی (**Bit Array**) و چندین تابع هش (**Hash Function**) استفاده می‌کند. وقتی آدرسی استخراج می‌شود، توابع هش تولید شده و بیت‌های خاصی از آرایه را 1 می‌کنند.

مزیت ریاضی: با فیلتر بلوم می‌توان وضعیت تکراری بودن ده‌ها میلیارد آدرس را تنها با استفاده از چند گیگابایت حافظه RAM و با مرتبه زمانی $O(1)$ بررسی کرد.

حاشیه خطا (False Positive): تنها ضعف این الگوریتم احتمال خطای مثبت کاذب است (یعنی ممکن است به اشتباه بگوید 1 آدرس جدید قبلاً دیده شده است). به همین دلیل، خزشگرهای صنعتی ابتدا آدرس را از فیلتر بلوم (در حافظه RAM) عبور می‌دهند و اگر فیلتر گفت آدرس تکراری است، برای اطمینان 100 درصدی، آن را در 1 جدول هش مبتنی بر دیسک (**Disk-based Hash Table**) نیز بررسی می‌کنند.

مدل‌سازی شماتیک: معماری فیلتر بلوم



نتیجه‌گیری پایانی:

ماژول **URL Seen** با جدا کردن منطق «کشف» از «واکشی»، و با استفاده از ساختارهای داده‌ای پیشرفته مانند فیلتر بلوم، تضمین می‌کند که گراف درهم‌تنیده وب به 1 درخت پیمایشی یک‌طرفه تبدیل شده و خزشگر هرگز در تله‌های ساختاری گرفتار نشود.

پاسخنامه مسئله 20.18: توزیع بار در سرورهای شاخص (Index Server Load Balancing)

مقدمه: قانون زیف و پدیده «گره داغ» (Hot Node)

برای درک ریشه این مشکل، ابتدا باید به «قانون زیف» (**Zipf's Law**) در زبان‌شناسی و بازیابی اطلاعات اشاره کنیم. این قانون به صورت ریاضی اثبات می‌کند که توزیع فراوانی کلمات در هر زبانی به شدت نامتقارن (**Skewed**) است؛ به طوری که تعداد بسیار انگشت‌شماری از کلمات (کلمات توقف یا **Stop words** مانند «از»، «در»، «است») کسر عظیمی از کل متون را تشکیل می‌دهند.

$$f_i \propto \frac{1}{i}$$

فرکانس کلمه i ام با رتبه آن رابطه عکس دارد

در معماری توزیع واژه-محور (**Term-Partitioned**)، اگر تخصیص کلمات به سرورها صرفاً با 1 تابع درهم‌سازی (**Hash Function**) ساده انجام شود، سروری که به صورت تصادفی کلمه «است» به آن تخصیص یافته است، با 2 بحران فلج‌کننده مواجه می‌شود:

1. **چولگی ذخیره‌سازی (Data Skew):** لیست پستینگ (**Posting List**) این کلمه به تنهایی صدها مگابایت فضا اشغال می‌کند.

2. **چولگی پرسمان (Query Skew):** از آنجا که این کلمه در اکثر کوئری‌های کاربران وجود دارد، این سرور باید به هزاران درخواست در ثانیه پاسخ دهد، در حالی که سرور حاوی کلمات تخصصی کاملاً بیکار است. به این پدیده، ایجاد 1 گلوگاه یا گره داغ (**Hot Spot**) می‌گویند.

مدل‌سازی شماتیک: پدیده گره داغ در معماری واژه‌محور

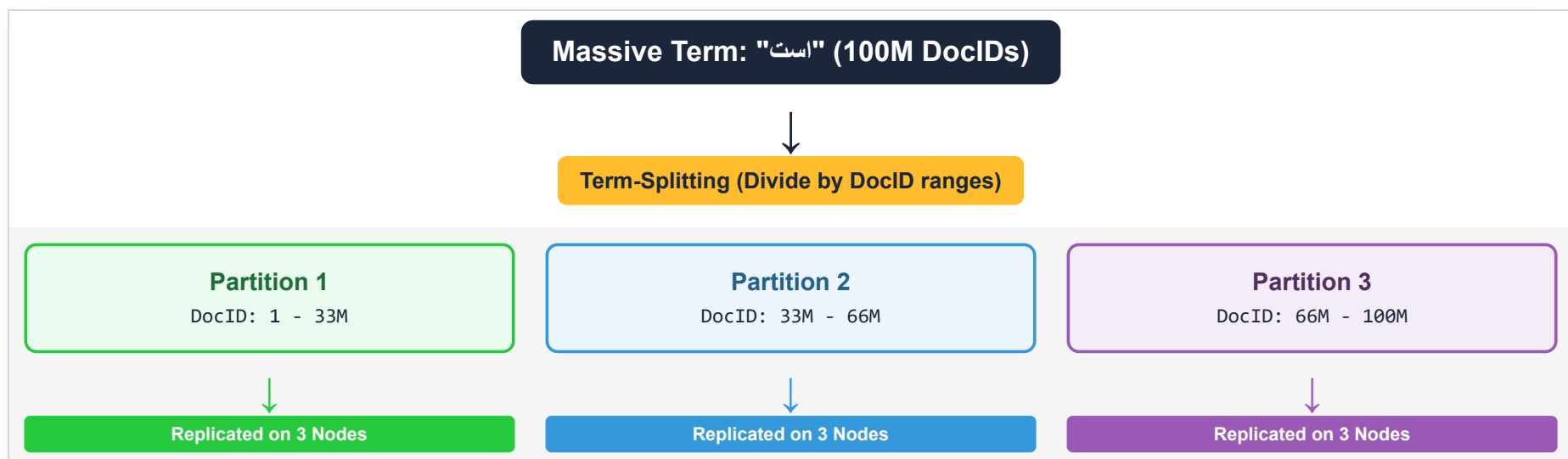
NODE A (HOT NODE)	NODE B (IDLE NODE)
Assigned Term: "است" Posting List: 80,000,000 DocIDs	Assigned Term: "می‌تواند" Posting List: 4,500 DocIDs
- CPU Usage: 99% (Overloaded) - Queries/sec: 15,000 - Network I/O: Bottlenecked	- CPU Usage: 2% (Idle) - Queries/sec: 12 - Network I/O: Minimal

راهکارهایی برای توازن بار (Load Balancing)

معماران موتور جستجو برای جلوگیری از فروپاشی سیستم در معماری **Term-Partitioned**، از ترکیبی از استراتژی‌های زیر برای توازن بار استفاده می‌کنند:

1. تکرار انتخابی (Selective Replication) منطقی‌ترین راه برای کاهش بار پردازشی روی 1 سرور داغ، تکثیر آن است. در این مکانیزم، سیستم کلمات را بر اساس فرکانس جستجو رتبه‌بندی می‌کند. سرورهایی که حاوی کلمات بسیار پرکاربرد (مانند کلمات توقف) هستند، به جای 1 بار، روی چندین ماشین فیزیکی (مثلاً 5 کپی یکسان) تکثیر یا Replicate می‌شوند. سپس 1 توزیع‌کننده بار (Load Balancer) در جلوی این گره‌ها قرار می‌گیرد و پرسرمان‌های ورودی را با الگوریتم نوبت‌چرخشی (Round-Robin) بین کپی‌ها تقسیم می‌کند تا هیچ پردازنده‌ای دچار سربار (Overload) نشود.	
2. قطعه‌بندی ترکیبی (Term-Splitting) لیست پستینگ کلمه «است» (که مثلاً شامل 100 میلیون شناسه سند است) به 5 قطعه 20 میلیونی تقسیم می‌شود (بر اساس شناسه سند یا DocID). هر قطعه به 1 سرور مجزا سپرده می‌شود. نکته تحلیلی: با این کار، ما عملاً برای کلمات بسیار پرکاربرد، معماری سیستم را از Term-partitioned به Document-partitioned تغییر داده‌ایم تا بار لیست‌های طولانی را بشکنیم.	
3. مسیریابی آگاه به بار (Workload Hashing) به جای توزیع کورکورانه کلمات با توابع هش یکنواخت، سیستم از 1 الگوریتم تخصیص وزن‌دار استفاده می‌کند. اگر قرار است کلمه سنگینی مانند «در» به گره شماره 2 تخصیص یابد، سیستم می‌داند که این کلمه 90% از منابع پردازشی گره 2 را اشغال خواهد کرد. بنابراین، تعداد کلمات دیگری که به این گره سپرده می‌شود به شدت کاهش می‌یابد تا مجموع بار پردازشی (Total Workload) در گره 2، با گره 3 (که حاوی هزاران کلمه تخصصی و کم‌جستجو است) برابر شود.	
4. نهان‌سازی پرسرمان (Query Caching) هنگامی که کاربر عبارت "دانشگاه صنعتی شریف است" را جستجو می‌کند، بهینه‌ساز پرسرمان (Query Optimizer) مداخله می‌کند: • جابجایی انتقال داده: به جای اینکه لیست 80 میلیونی کلمه «است» برای اشتراک‌گیری در شبکه جابجا شود، لیست کوتاه کلمه «شریف» به سرور کلمه «است» ارسال می‌گردد تا ترافیک شبکه به حداقل برسد. • کشینگ (Caching): از آنجا که نتایج تقاطع پرسرمان‌های دارای کلمات توقف بسیار تکراری هستند، خروجی این اشتراک‌گیری‌ها در 1 لایه کش سریع (مانند Memcached یا Redis) ذخیره می‌شود تا در دفعات بعدی، سیستم اصلاً نیازی به پردازش مجدد لیست پستینگ کلمه توقف نداشته باشد.	

مدل‌سازی شماتیک: پیاده‌سازی همزمان Replication و Term-Splitting



جمع‌بندی:

چالش‌های ناشی از کلمات توقف و قانون زیپ، به عنوان **پاشنه آشیل** در معماری **Term-partitioned** شناخته می‌شوند. پیاده‌سازی همزمان تکنیک‌های **Replication** و **Term-Splitting** برای مدیریت این چالش، پیچیدگی سیستم (**System Complexity**) و هزینه‌های همگام‌سازی شبکه را به شدت بالا می‌برد.

به همین دلیل مهندسی است که امروزه تمام موتورهای جستجوی مقیاس‌وب، معماری **Term-partitioned** را کنار گذاشته و به طور کامل از معماری **توزیع سند-محور (Document-partitioned)** استفاده می‌کنند؛ چرا که در معماری سند-محور، اسناد به صورت تصادفی توزیع می‌شوند و بار پردازشی تمام کلمات (از جمله کلمات توقف) به صورت کاملاً طبیعی و متقارن بین تمامی گره‌های شبکه تقسیم می‌گردد.

پاسخنامه مسئله 20.19: برخورد با صفحات پویا و پارامترهای تکراری (Dynamic Parameters & DUST)

مقدمه: بحران پارامترهای پویا و پدیده DUST

در معماری وب مدرن، سایت‌های فروشگاهی و پایگاه‌های داده‌محور از پارامترهای کوئری (**Query Parameters**) برای فیلتر کردن، مرتب‌سازی یا مدیریت نشست (**Session**) کاربران استفاده می‌کنند.

اضافه شدن پارامترهایی مانند `sort=price_asc?` یا `sort=name?` باعث می‌شود ترتیب نمایش کالاها تغییر کند، اما محتوای اصلی (**Core Content**) کالاها ثابت باقی می‌ماند. این مسئله منجر به بروز پدیده **DUST (Duplicate URLs with Similar Text)** می‌شود؛ جایی که هزاران URL متفاوت، عملاً به 1 محتوای واحد اشاره می‌کنند.

اگر خزشگر نتواند این پارامترهای بی‌اثر را پیش از ورود به صف خزش (**URL Frontier**) شناسایی کند، در 1 تله عنکبوتی (**Spider Trap**) گرفتار شده و منابع خود را صرف واکنشی هزاران صفحه تکراری از 1 فروشگاه خواهد کرد.

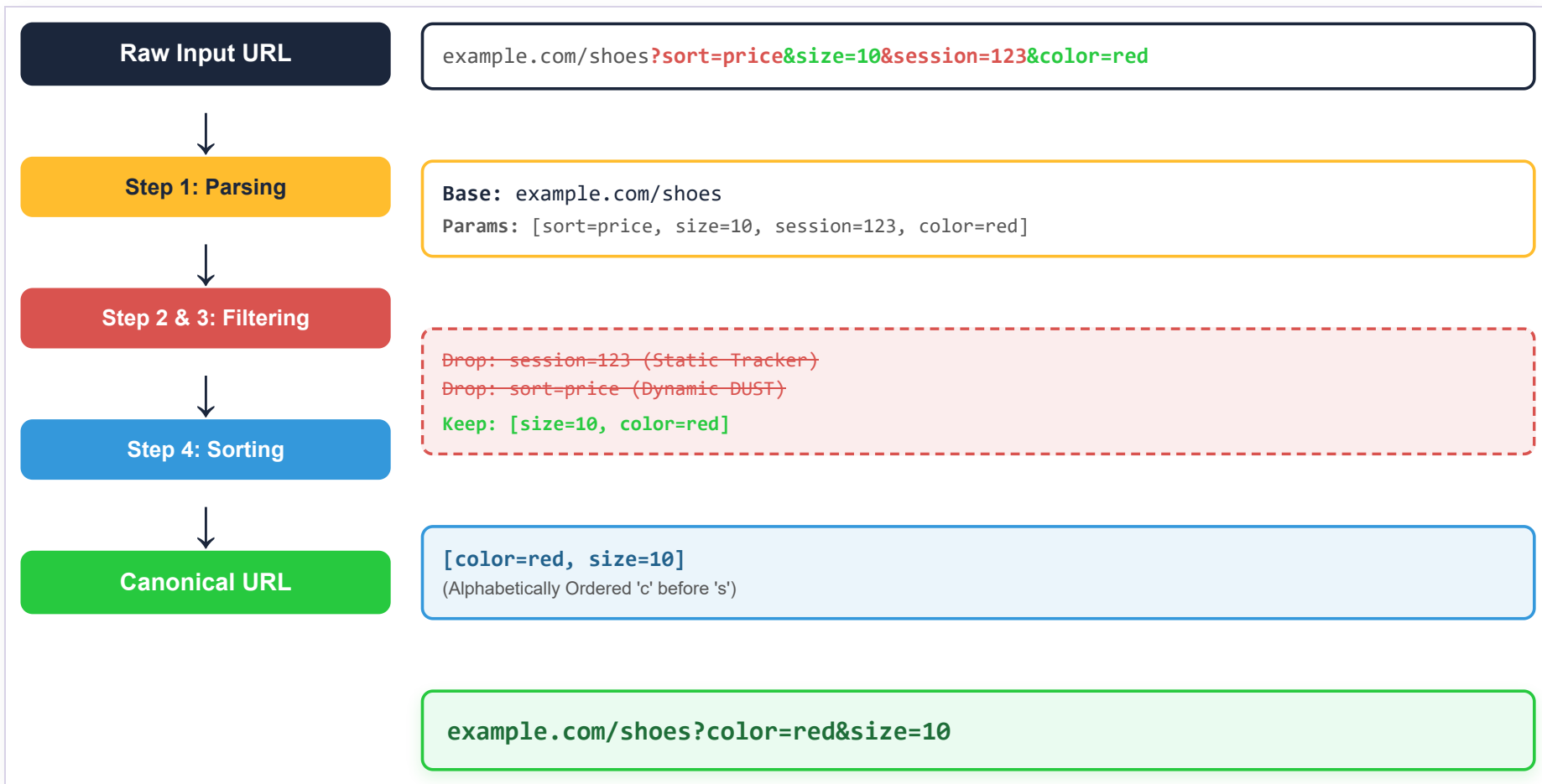
قانون و الگوریتم نرمال‌سازی پارامترها

برای حل این چالش، ماژول نرمال‌سازی URL صرفاً به توابع متنی اتکا نمی‌کند، بلکه از 1 رویکرد ترکیبی (اکتشافی و آماری) برای تدوین قانون حذف پارامترها بهره می‌برد. این قانون طی مراحل زیر اجرا می‌شود:

گام اجرایی	شرح مکانیزم و الگوریتم فیلترینگ
گام 1 تجزیه آدرس (URL Parsing)	جداسازی اجزای آدرس: خزشگر ابتدا رشته URL را به 2 بخش اصلی تقسیم می‌کند: «مسیر پایه» (Base Path) و «رشته کوئری» (Query String). مثلاً در آدرس <code>example.com/shoes?sort=price&brand=nike</code> ، مسیر پایه برابر با <code>example.com/shoes</code> و پارامترها <code>sort</code> و <code>brand</code> هستند.
گام 2 فیلترینگ ایستا (Static Blacklisting)	اعمال لیست سیاه قطعی: سیستم دارای 1 لیست سیاه از پیش‌تعریف‌شده (Global Blacklist) از پارامترهایی است که در کل وب هرگز تغییری در محتوا ایجاد نمی‌کنند. ماژول نرمال‌سازی بلافاصله هرگونه پارامتر مرتبط با رهگیری تبلیغات (مانند <code>utm_source</code>) یا نشست کاربری (مانند <code>session_id</code>) را حذف می‌کند.
گام 3	تحلیل آماری شباهت محتوا: برای پارامترهای خاص 1 سایت (مانند <code>sort</code> یا <code>order</code>)، خزشگر به صورت پویا یاد می‌گیرد. سیستم، صفحه پایه و صفحه دارای پارامتر را واکنشی کرده و اثر انگشت آن‌ها (مثلاً از طریق الگوریتم Simhash) را مقایسه می‌کند.

گام اجرایی	شرح مکانیزم و الگوریتم فیلترینگ
کشف پویای پارامتر (Dynamic Discovery)	اگر شباهت ژاکارد (Jaccard Similarity) بین 2 صفحه از آستانه مشخصی ($\theta \approx 0.95$) بیشتر باشد، سیستم 1 قانون جدید ثبت می‌کند: «برای این دامنه، پارامتر sort محتوا را تغییر نمی‌دهد.» از این پس، این پارامتر از تمام URLهای این سایت حذف می‌شود.
گام 4 مرتب‌سازی الفبایی (Lexicographical Sort)	جلوگیری از افزونگی جایگزینی: یکی از مشکلات رایج این است که آدرس <code>?</code> با <code>color=red&size=10</code> آدرس <code>size=10&color=red?</code> دقیقاً 1 محتوا را برمی‌گردانند، اما از نظر متنی متفاوت‌اند. در گام آخر، ماژول نرمال‌سازی، پارامترهای مجاز را بر اساس حروف الفبا مرتب کرده و آدرس نهایی را بازسازی می‌نماید.

مدل‌سازی شماتیک: خطوله رفع افزونگی (DUST Resolution Pipeline)



شبه‌کد (Pseudo-code)

```
url_normalizer.algo

Let URL = BasePath + "?" + QueryString
Let params = split(QueryString, "&")
Let valid_params = empty list

For each p in params:
  Let key = extract_key(p)

  // 1. check global blacklist (trackers and sessions)
  If key in GLOBAL_BLACKLIST:
    continue
  // 2. check local blacklist (learned for this domain)
  If key in DOMAIN_BLACKLIST[URL.domain]:
    continue
  // otherwise, parameter is valid
  add p to valid_params

// sort valid params alphabetically to prevent permutation duplicates
sort_alphabetically(valid_params)

Return BasePath + "?" + join(valid_params, "&")
```

نتیجه‌گیری تحلیلی:

این الگوریتم نشان می‌دهد که 1 خزشگر مقیاس‌پذیر، 1 سیستم کاملاً استاتیک نیست. خزشگرهای مدرن دارای مکانیزم یادگیری ماشین (Machine Learning) در لایه URL Frontier هستند تا رفتار هر وب‌سایت را به صورت کاملاً مستقل

تحلیل کرده و قوانینی (Rulesets) اختصاصی برای حذف پارامترهای بی‌اثر همان سرور ایجاد کنند. این امر ضمن حفظ ساختار گراف وب، از اتلاف پهنای باند و منابع پردازشی به شدت جلوگیری می‌کند.

پاسخنامه مسئله 20.20: تخصیص زمان در Min-Heap

مقدمه: نقش Min-Heap در صف‌های پشتی (Back Queues)

در معماری صف خزش (URL Frontier)، صف‌های پشتی (Back Queues) وظیفه تضمین سیاست «ادب» (Politeness) را بر عهده دارند. قانون اصلی این است که هر صف پشتی منحصرراً آدرس‌های 1 میزبان (Host) خاص را در خود نگه می‌دارد و بین هر 2 درخواست متوالی به آن میزبان، باید 1 وقفه زمانی (Crawl-delay) اعمال شود.

برای اینکه رشته‌های خزشگر (Worker Threads) بتوانند بهینه‌ترین صف آماده‌ی واکنشی را پیدا کنند، از ساختار داده Min-Heap (هرم کمینه) به عنوان 1 صف اولویت‌دار (Priority Queue) استفاده می‌شود. در این هرم، مقدار کلید (Key) برای هر گره، برابر با زمان مجاز برای واکنشی بعدی (T_e) است. ماهیت Min-Heap تضمین می‌کند صفی که زودتر از همه زمان انتظارش به پایان می‌رسد، همیشه در ریشه (Root) هرم قرار داشته باشد.

فرمول ریاضی محاسبه زمان بیدار شدن

هنگامی که خزشگر 1 URL را از 1 صف پشتی خارج کرده و به سمت شبکه ارسال می‌کند، آن صف موقتاً از هرم خارج می‌شود. پس از اتمام واکنشی، سیستم باید زمان بیدار شدن بعدی (T_{next}) را برای این صف محاسبه کند تا مجدداً آن را در هرم قرار دهد. این زمان بر اساس فرمول جبری زیر محاسبه می‌شود:

$$T_{next} = T_{now} + \Delta t$$

که در آن:

T_{now} : زمان فعلی سیستم (بر حسب ثانیه یا میلی‌ثانیه).

Δt : میزان تاخیر تعیین‌شده برای آن سرور (مستخرج از فایل robots.txt یا سیاست‌های پیش‌فرض موتور جستجو).

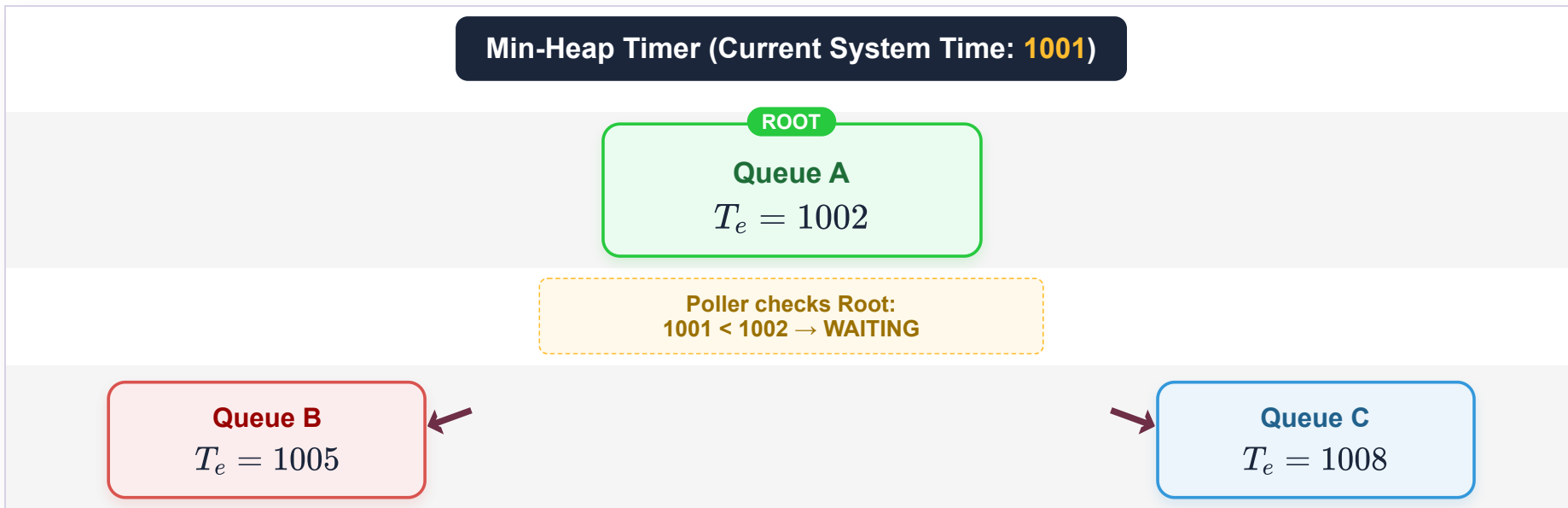
تحلیل سناریو: سرور A و سرور B

برای حل دقیق مسئله، 1 زمان فرضی را به عنوان زمان حال در نظر می‌گیریم. فرض کنید زمان فعلی سیستم، ثانیه 1000 ($T_{now} = 1000$) است و 2 رشته خزشگر به طور همزمان 1 آدرس از صف سرور A و 1 آدرس از صف سرور B را واکنشی می‌کنند.

1. به‌روزرسانی صف سرور A	2. به‌روزرسانی صف سرور B
سرور A دارای محدودیت تاخیر 2 ثانیه ($\Delta t_A = 2$) است.	سرور B دارای محدودیت تاخیر 5 ثانیه ($\Delta t_B = 5$) است.
محاسبه زمان بعدی:	محاسبه زمان بعدی:

2. به روزرسانی صف سرور B	1. به روزرسانی صف سرور A
$T_{next,B} = 1000 + 5 = 1005$	$T_{next,A} = 1000 + 2 = 1002$
درج مجدد: صف مربوط به سرور B با کلید 1005 به درون Min-Heap اضافه می‌شود و در جایگاهی پایین‌تر از سرور A تثبیت می‌گردد.	درج مجدد: صف مربوط به سرور A با کلید 1002 مجدداً به درون Min-Heap اضافه (Insert) می‌شود. عملیات Swim-up (بالا رفتن در هرم) روی آن اجرا می‌شود تا در جایگاه مناسب خود قرار گیرد.

مدل سازی شماتیک پیشرفته: وضعیت Min-Heap پس از درج



توصیف وضعیت هرم و رفتار خزشگر پس از درج

پس از درج این 2 صف، رفتار ساختار داده و سیستم به شرح زیر خواهد بود:

1. **مرتب سازی خودکار:** به دلیل ویژگی ذاتی Min-Heap (گره والد همیشه کوچکتر یا مساوی فرزندان است)، گره مربوط به سرور A (با کلید 1002) در سطحی بالاتر از گره مربوط به سرور B (با کلید 1005) قرار می‌گیرد و دقیقاً در جایگاه ریشه مستقر می‌شود.

2. **بررسی پیوسته سیستم:** رشته‌های خزشگر مرتباً مقدار کلید ریشه هرم را بررسی می‌کنند.

• اگر زمان سیستم به 1001 برسد، خزشگر می‌بیند که زمان ریشه حداقل 1002 است. در این حالت، هیچ درخواستی به این سرورها ارسال نمی‌شود و سیستم منتظر می‌ماند (یا به سراغ صف‌های دامنه‌های دیگر در بالای هرم می‌رود).

• به محض اینکه زمان ساعت سیستم به 1002 برسد، صف سرور A از ریشه استخراج شده و پردازش می‌شود. صف سرور B همچنان تا رسیدن به ثانیه 1005 در داخل هرم مسدود (Block) باقی خواهد ماند.

نتیجه‌گیری پایانی:

زمان درج مجدد هر گره، جمع جبری زمان اتمام واکنشی قبلی به علاوه محدودیت تاخیر اختصاصی آن میزبان است. استفاده از Min-Heap در این فرآیند باعث می‌شود خزشگر نیازی به پیمایش خطی در میان هزاران صف برای یافتن «صف آماده‌ی واکنشی» نداشته باشد و این عملیات با پیچیدگی زمانی بسیار پایین و لگاریتمی $O(\log N)$ به بهینه‌ترین شکل ممکن از نظر مهندسی نرم‌افزار انجام پذیرد.

پاسخنامه مسئله 20.21: پیاده‌سازی تشخیص تله‌های عنکبوتی (Spider Trap Detection)

مقدمه: چرا تله‌های تقویمی ماژول URL Seen را فریب می‌دهند؟

تله‌های عنکبوتی ناشی از تقویم‌های پویا (Dynamic Calendars) یکی از چالش‌برانگیزترین خطاهای ساختاری در وب هستند. وقتی خزشگر روی لینک «ماه بعد» کلیک می‌کند، سرور 1 صفحه جدید با 1 URL کاملاً جدید (مثلاً `/events/2026/08`) و سپس `/events/2026/09` تا بی‌نهایت) تولید می‌کند.

از آنجا که هر رشته متنی کاملاً یکتا و جدید است، ماژول `URL_Seen` (و فیلترهای بلوم) فریب می‌خورند و این آدرس‌ها را به عنوان صفحات جدید به صف خزش اضافه می‌کنند. برای مهار این فاجعه، نیازمند 1 الگوریتم ناظر (Watchdog Algorithm) هستیم که الگوهای ناهنجار در ساختار URL را پیش از ورود به صف شناسایی کند.

طراحی الگوریتم ناظر (Watchdog Algorithm)

این الگوریتم به عنوان 1 میان‌افزار (Middleware) پیش از افزودن آدرس به صف خزش اجرا می‌شود و آدرس را از 3 فیلتر کنترلی عبور می‌دهد. برای پیاده‌سازی ریاضی، 3 پارامتر آستانه (Threshold) تعریف می‌کنیم:

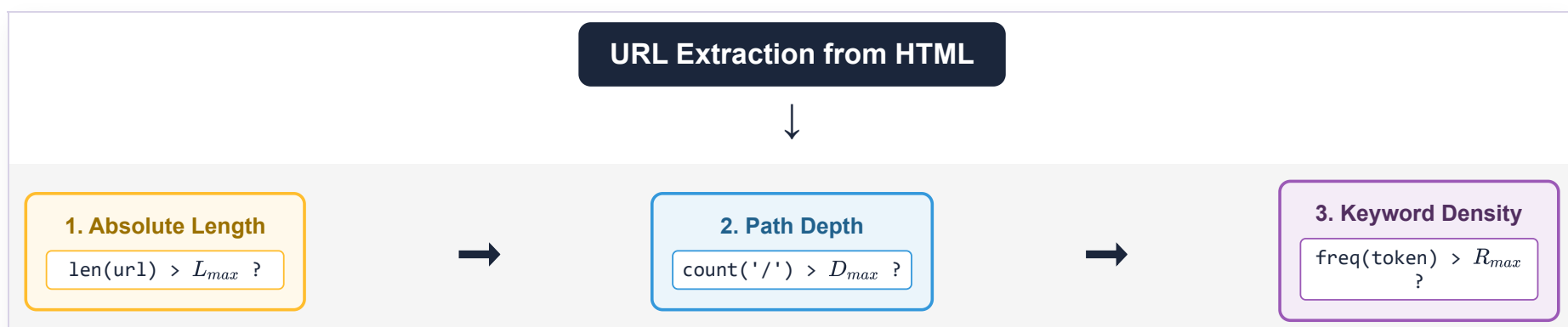
L_{max} : حداکثر طول مجاز 1 رشته URL.

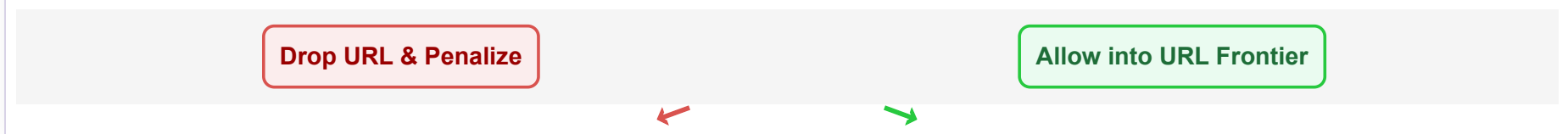
D_{max} : حداکثر عمق مجاز مسیر (تعداد پوشه‌ها یا اسلش‌ها).

R_{max} : حداکثر دفعات مجاز تکرار 1 کلمه کلیدی در مسیر.

گام اجرایی	شرح مکانیزم و الگوریتم فیلترینگ
فیلتر 1 بررسی طول مطلق (Length Check)	ساده‌ترین و سریع‌ترین سد دفاعی است. الگوریتم طول رشته متنی URL را محاسبه می‌کند. اگر طول آن از مرز L_{max} (مثلاً 300 کاراکتر) عبور کرده باشد، فوراً به عنوان تله برچسب‌گذاری می‌شود.
فیلتر 2 بررسی عمق مسیر (Path Depth)	برخی تله‌ها طول کل کوتاهی دارند اما مسیرهای تودرتو ایجاد می‌کنند (مانند <code>/a/b/c/d/e/f/g/...</code>). الگوریتم با شمارش کاراکتر اسلش <code>/</code> در بخش مسیر (Path)، عمق درخت را محاسبه می‌کند. تجاوز از D_{max} نشان‌دهنده 1 تله دایرکتوری چرخشی است.
فیلتر 3 تراکم واژگان (Repetition)	خطرناک‌ترین تله‌ها، تله‌های پارامتریک و تقویمی هستند. الگوریتم مسیر را بر اساس کاراکترهای جداکننده (مانند <code>/</code> ، <code>?</code> ، <code>&</code> و <code>=</code>) توکنایز (Tokenize) می‌کند. سپس فرکانس تکرار هر توکن را می‌شمارد. اگر توکنی (مانند <code>calendar</code> یا اعداد متوالی سال/ماه) بیش از R_{max} بار تکرار شده باشد، الگوریتم وقوع تله را اعلام می‌کند.

مدل‌سازی شماتیک پیشرفته: خط‌لوله نظارتی (Watchdog Pipeline)





پیاده‌سازی اسکریپت در پایتون (مخصوص Autograder)

کد زیر به گونه‌ای طراحی شده است که به عنوان 1 ماژول ارزیاب مستقل عمل کند. این تابع 1 شیء (شامل وضعیت توقف و دلیل آن) برمی‌گرداند تا به راحتی به پارامترهای کنترلی اتوگریدر (**Autograder**) متصل شود.

```
watchdog_detector.py

from urllib.parse import urlparse
from collections import Counter
import re

class SpiderTrapException(Exception):
    """کلاس استثنای سفارشی برای مدیریت خطا در اتوگریدر"""
    pass

def watchdog_trap_detector(url: str, max_len: int = 300, max_depth: int = 10, max_repeats: int = 3) -> dict:
    """
    الگوریتم ناظر برای تشخیص تله‌های عنکبوتی در سیستم‌های ارزیابی خودکار.
    """
    # گام 1: بررسی طول مطلق
    if len(url) > max_len:
        return {"is_trap": True, "penalty_code": "LENGTH_EXCEEDED", "value": len(url)}

    try:
        parsed_url = urlparse(url)
    except Exception:
        return {"is_trap": True, "penalty_code": "MALFORMED_URL", "value": 0}

    # استخراج مسیر و کوئری‌ها
    path_string = parsed_url.path.lower()
    query_string = parsed_url.query.lower()

    # گام 2: بررسی عمق مسیر (تعداد پوشه‌ها)
    # فیلتر کردن رشته‌های خالی برای جلوگیری از خطای اسلش‌های متوالی
    path_segments = [seg for seg in path_string.split('/') if seg]
    if len(path_segments) > max_depth:
        return {"is_trap": True, "penalty_code": "DEPTH_EXCEEDED", "value": len(path_segments)}

    # گام 3: تحلیل تراکم واژگان (تجزیه تمام کلمات موجود در مسیر و پارامترها)
    # برای استخراج واژگان معنادار (حروف و اعداد) Regex استفاده از
    combined_string = f"{path_string} {query_string}"
    tokens = re.findall(r'[a-z0-9]+', combined_string)

    # شمارش فرکانس هر توکن
    token_counts = Counter(tokens)

    # بررسی اینکه آیا هیچ توکنی بیش از حد مجاز تکرار شده است؟
    if token_counts:
        most_common_token, count = token_counts.most_common(1)[0]
        if count > max_repeats:
            return {
                "is_trap": True,
                "penalty_code": "KEYWORD_REPETITION",
                "value": f"Token '{most_common_token}' repeated {count} times"
            }

    return {"is_trap": False, "penalty_code": "SAFE", "value": None}
```

نحوه اتصال الگوریتم به سیستم ارزیاب خودکار (Autograder Integration)

برای استفاده از این اسکریپت در فرآیند تصحیح و نمره‌دهی پروژه‌های دانشجویان، می‌توانید این ناظر را در کلاس **Autograder** به عنوان 1 هوک (**Hook**) قبل از اجرای متد **fetch** یا در زمان آپدیت صف دانشجویان متصل کنید:

1. تست مقاومت (Resilience Test)	اتوگریدر لیستی از URLهای مسموم (Poisoned URLs) را به کد دانشجو تزریق می‌کند.
------------------------------------	---

هر بار که کد دانشجو می‌خواهد 1 URL جدید را به صف خود (Frontier) اضافه کند، این عملیات توسط watchdog_trap_detector رهگیری می‌شود.	2. فعال‌سازی ناظر (Intercept)
اگر تابع is_trap: True را برگرداند، به این معناست که کد دانشجو نتوانسته تله را تشخیص دهد و در حال ورود به آن است. اتوگریدر در این لحظه اجرای اسکریپت دانشجو را متوقف کرده (Raise SpiderTrapException)، نمره بخش Robustness را کسر نموده و penalty_code را به عنوان بازخورد (Feedback) برای دانشجو چاپ می‌کند تا دقیقاً بداند در کدام فیلتر امنیتی شکست خورده است.	3. اعمال جریمه (Penalization)

نتیجه‌گیری تحلیلی:

این استراتژی در اتوگریدرها و همچنین خزشگرهای صنعتی به وضوح اثبات می‌کند که سیستم‌های واکنشی اطلاعات نمی‌توانند کورکورانه به موتورهای کشف آدرس متکی باشند. 1 میان‌افزار امنیتی هوشمند، با تجزیه چندلایه‌ی ساختار آدرس‌ها، ایمنی کلاستر پردازشی را تضمین کرده و از فلج شدن معماری در مواجهه با حلقه‌های مخرب و تله‌های بی‌نهایت تقویمی جلوگیری می‌کند.

پاسخنامه مسئله 20.22: طراحی الگوریتم صف‌های اولویت‌دار دوگانه (Dual Priority Queues)

مقدمه: معماری دوگانه (Dual Architecture)

طراحی ساختمان داده برای مدیریت صف خزش (URL Frontier)، نیازمند پیاده‌سازی معماری مشهور مرکاتور (Mercator) است. در این معماری، ما با 2 چالش کاملاً متضاد روبرو هستیم:

1. کیفیت (Priority): آدرس‌های مهم باید زودتر پردازش شوند (از طریق صف‌های Front).

2. ادب (Politeness): هیچ سروری نباید بمباران شود و باید بین درخواست‌ها تاخیر وجود داشته باشد (از طریق

صف‌های Back و Min-Heap).

در ادامه، ساختار داده‌های مورد نیاز و شبه‌کد (Pseudo-code) دقیق این سیستم با استانداردهای دانشگاهی مدل‌سازی شده است.

بخش 1) ساختار داده‌های پایه (Data Structures)

پیش از نوشتن توابع اصلی، باید اجزای کلاسی که این معماری را مدیریت می‌کند، تعریف کنیم. سیستم از 4 بخش اصلی تشکیل می‌شود:

1 آرایه شامل K صف FIFO (مثلاً از اولویت 1 تا اولویت 10).	front_queues
1 آرایه شامل M صف FIFO. هر صف در هر لحظه فقط متعلق به 1 دامنه (Host) است.	back_queues
1 هرم کمینه که صف‌های Back را بر اساس ویژگی next_fetch_time (زمان مجاز واکنشی بعدی) مرتب می‌کند.	min_heap
1 جدول درهم‌سازی (Hash Map) که نام دامنه را به شناسه (ID) صف پشتی تخصیص‌یافته نگاشت می‌کند تا از قانون «1 میزبان - 1 صف» محافظت شود.	host_map

بخش 2) تابع درج در صف: insert(url, priority)

وظیفه این تابع بسیار سبک و سریع است. آدرس‌های جدید کشف‌شده، صرفاً بر اساس امتیاز اولویتی که دارند، وارد یکی از صف‌های **Front** می‌شوند و هیچ درگیری با منطق تاخیر سرورها ندارند.

```
insert_module.algo

function insert(url, priority):
    1. محاسبه شناسه صف جلویی بر اساس اولویت (نگاشت امتیاز به اندیس آرایه)
    index = calculate_front_queue_index(priority)

    2. درج آدرس در انتهای صف جلویی مربوطه.
    front_queues[index].enqueue(url)
```

بخش 3) الگوریتم تخصیص و واکنشی: get_next()

قلب تپنده این معماری، تابع `get_next()` است که توسط رشته‌های خزشگر (**Worker Threads**) فراخوانی می‌شود. این الگوریتم وظیفه دارد 1 آدرس امن را برای واکنشی تحویل دهد و همزمان، جریان داده را از **Front** به **Back** هدایت کند.

مرحله اجرایی	شرح مکانیزم و منطق کنترلی
مرحله 1 یافتن صف مجاز (استخراج امن از هرم)	خزشگر ابتدا به ریشه هرم (Min-Heap) نگاه می‌کند. اگر زمان <code>next_fetch_time</code> برای صفِ ریشه هنوز فرانسیده باشد، خزشگر منتظر می‌ماند (Sleep). به محض رسیدن زمان، صف پشتی (Back Queue) از ریشه هرم استخراج (Extract-Min) می‌شود.
مرحله 2 پر کردن صف پشتی (عملیات تخصیص پویا)	اگر صف پشتی استخراج‌شده خالی باشد، باید بلافاصله از صف‌های جلویی تغذیه شود. سیستم 1 صف جلویی را (با شانس بیشتر برای صف‌های دارای اولویت بالاتر) انتخاب کرده و آدرسی را برمی‌دارد. نقطه بحرانی: اگر میزبان (Host) این آدرس جدید از قبل در 1 صف پشتی دیگر فعال باشد، آدرس به آن صف منتقل می‌شود و سیستم دوباره تلاش می‌کند تا آدرسی با 1 میزبان جدید و آزاد برای صفِ خالی فعلی پیدا کند.
مرحله 3 استخراج آدرس (آماده‌سازی واکنشی)	اکنون که صف پشتی حداقل 1 آدرس مجاز دارد، آدرسِ سر صف برداشته (Dequeue) می‌شود. سیستم زمان تاخیر سرور (Crawl-delay) را به زمان فعلی جمع کرده و به عنوان زمان بیدار شدن بعدی (T_{next}) برای این صف تنظیم می‌کند و مجدداً صف را درون هرم می‌اندازد.

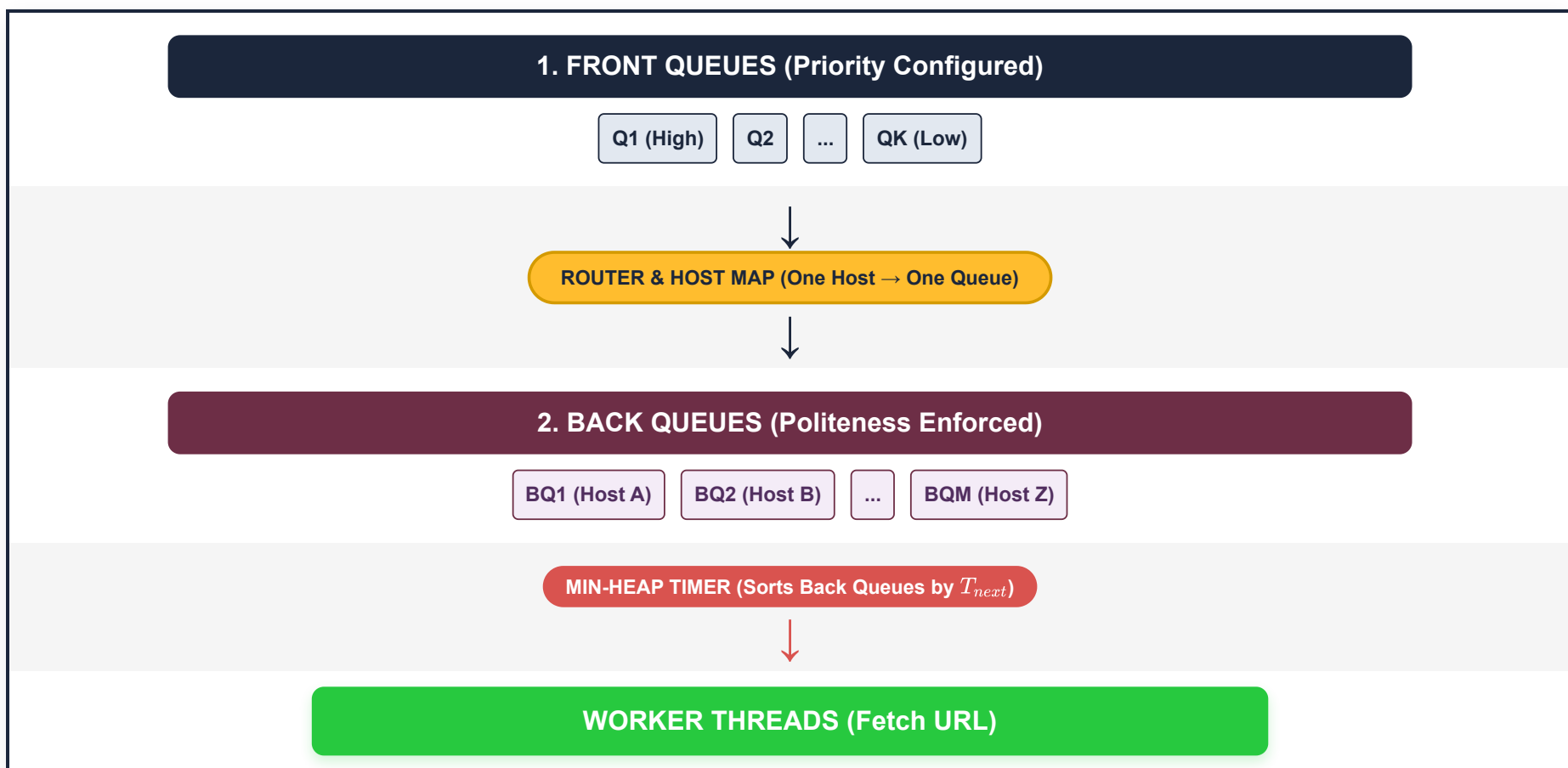
شبیه‌کد (Implementation Logic)

```
frontier_router.algo

function get_next():
    1. استخراج صف مجاز از هرم
    while min_heap.peek().next_fetch_time > current_time():
        sleep()
    b_queue = min_heap.extract_min()
    2. منطق تغذیه صف پشتی از صف‌های جلویی در صورت خالی بودن
    if b_queue.is_empty():
        while True:
            if all_front_queues_are_empty():
                return NULL // کل سیستم خالی است
            بر اساس اولویت (Biased Random) انتخاب سوگیری‌شده
            f_queue = biased_select(front_queues)
            new_url = f_queue.dequeue()
            host = extract_host(new_url)
            if host in host_map and host_map[host] != b_queue.id:
                این میزبان صف پشتی خودش را دارد، آدرس را به آنجا بفرست
                target_b_queue_id = host_map[host]
                back_queues[target_b_queue_id].enqueue(new_url)
                continue
            else:
                تخصیص میزبان جدید به صف پشتی فعلی
                b_queue.host = host
```

```
host_map[host] = b_queue.id
b_queue.enqueue(new_url)
break // خروج از حلقه
برداشتن آدرس نهایی برای واکنشی.
3.
target_url = b_queue.dequeue()
در صورتی که صف با این برداشتن خالی شد، نگاشت میزبان را پاک می‌کنیم
if b_queue.is_empty():
    remove host_map[b_queue.host]
4. به‌روزرسانی زمان و درج مجدد در هرم Politeness: اعمال
delay = get_politeness_delay(target_url.host)
b_queue.next_fetch_time = current_time() + delay
min_heap.insert(b_queue)
return target_url
```

مدل‌سازی شماتیک: معماری جریان داده مرکاتور (Mercator Data Flow)



جمع‌بندی تحلیلی برای دانشجویان:

این الگوریتم شاهکار مهندسی همزمانی (**Concurrency**) در موتورهای جستجو است. جداسازی کامل توابع `insert` و `get_next` باعث می‌شود:

- پیمایشگرهای تجزیه‌کننده (**Parsers**) با سرعت فوق‌العاده بالا و بدون درگیری با قفل‌های زمانی، آدرس‌ها را در **front_queues** انباشته کنند.
- رشته‌های خزشگر (**Workers**) تنها با **min_heap** درگیر شوند که زمان‌بندی ایمن و مودبانه را تضمین می‌کند.
- منطق حلقه در مرحله 2 (**While loop**) تضمین می‌کند که مهم‌ترین آدرس‌ها (از بالاترین صف **Front**) هرگز سرور مقصد خود را بمباران نکنند، زیرا به صورت خودکار به صف محدودشده‌ی همان سرور در **Back Queue** سرازیر می‌شوند.

پاسخنامه مسئله 20.23: سناریو خرابی گره در پارتیشن‌بندی ترم (Node Failure in Term-Partitioning)

مقدمه: آسیب‌پذیری ذاتی در معماری واژه‌محور

همان‌طور که در مسائل پیشین بررسی شد، در معماری توزیع واژه‌محور (**Term-Partitioned**)، دامنه لغات (**Vocabulary**) بین گره‌های مختلف تقسیم می‌شود. این یعنی کل لیست پستینگ (**Posting List**) کلمه «الگوریتم» که شامل شناسه تمام

اسنادِ دارای این کلمه در کل وب است منحصراً روی 1 گره فیزیکی قرار دارد.

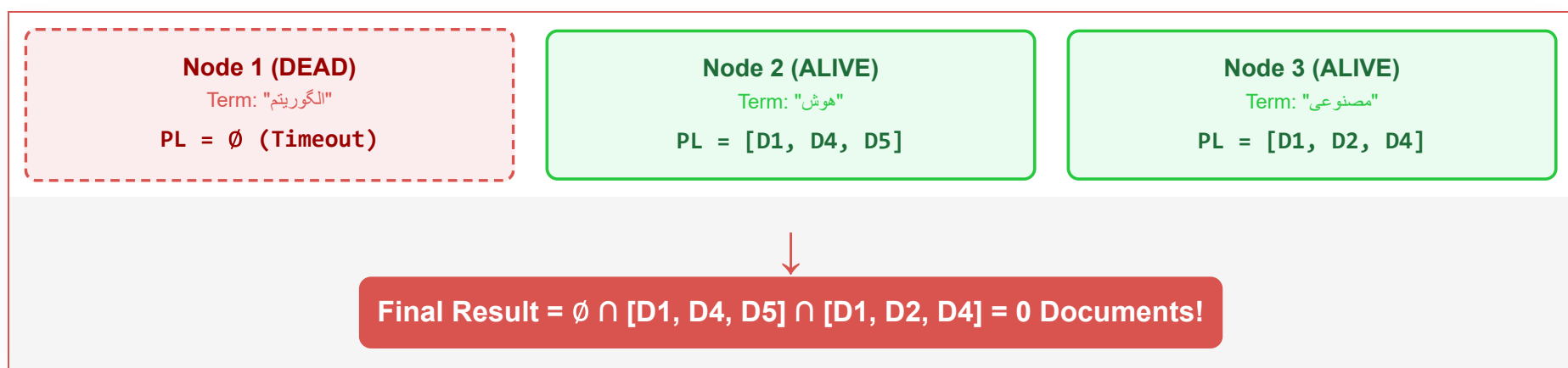
این معماری دارای 1 نقطه شکست یگانه (**Single Point of Failure**) برای هر کلمه است. در معماری سند-محور (**Document-Partitioned**)، خرابی 1 گره صرفاً باعث می‌شد درصد کوچکی از کل اسناد از نتایج حذف شوند (تخریب تدریجی یا **Graceful Degradation**)، اما در معماری واژه‌محور، خرابی 1 گره به معنای نابودی کامل دسترسی سیستم به آن کلمه خاص است.

بخش 1) پاسخ سیستم در غیاب مکانیزم Fallback و تکتیر

اگر سیستم فاقد تکنیک تکتیر (**Replication**) و مکانیزم‌های جایگزین باشد، پاسخ کاربر مستقیماً به نوع پرسمان (**Query**) او بستگی دارد و در هر 2 حالت، 1 شکست فاجعه‌بار (**Catastrophic Failure**) رخ می‌دهد:

سناریو 1 پرسمان تک‌کلمه‌ای (Single-term Query)	مانند جستجوی دقیق کلمه "الگوریتم": گره مرکزی (Broker) درخواست را به گره مسئول ارسال می‌کند. پس از پایان یافتن مهلت زمانی (Timeout)، گره مرکزی هیچ پاسخی دریافت نمی‌کند. کاربر با 1 صفحه خطای سیستمی (مثل 500 Internal Server Error) یا پیام «هیچ نتیجه‌ای یافت نشد» مواجه می‌شود، در حالی که میلیون‌ها سند مرتبط در سیستم وجود دارد!
سناریو 2 پرسمان چندکلمه‌ای (Multi-term Query)	مانند جستجوی "الگوریتم هوش مصنوعی": موتورهای جستجو به طور پیش‌فرض از عملگر عطفی (AND) بین کلمات استفاده می‌کنند. از نظر ریاضی، گره مرکزی باید اشتراک لیست پستینگ این 3 کلمه را محاسبه کند: $PL("متیروگل") \cap PL("شوه") \cap PL("ی‌عونصم")$ از آنجا که گره کلمه «الگوریتم» از کار افتاده است، سیستم مقدار $PL("متیروگل")$ را برابر با تهی ($\emptyset$) یا خطا در نظر می‌گیرد. اشتراک هر مجموعه‌ای با مجموعه تهی، برابر با تهی است. در نتیجه، کاربر با وجود دسترسی کامل سیستم به کلمات «هوش» و «مصنوعی»، 0 نتیجه دریافت می‌کند.

مدل‌سازی شماتیک: فاجعه تقاطع با مجموعه تهی (Null Intersection)



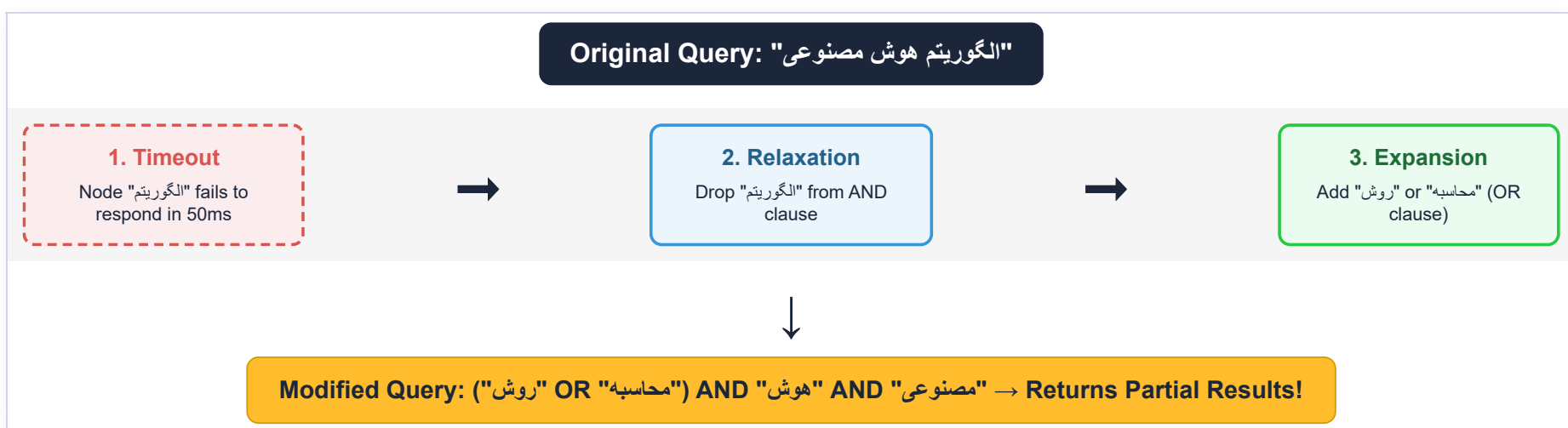
بخش 2) طراحی مکانیزم Fallback

از آنجا که طبق صورت مسئله استفاده از **Replication** (کپی کردن داده‌ها روی گره‌های دیگر) ممنوع است، ما داده‌های اصلی (لیست پستینگ دقیق کلمه «الگوریتم») را برای همیشه تا زمان رفع خرابی از دست داده‌ایم.

برای جلوگیری از نمایش خطا و ارائه نتایج جزئی (**Partial Results**)، باید از تکنیک‌های تحلیلی در لایه گره مرکزی (**Broker**) استفاده کنیم. الگوریتم پیشنهادی، ترکیبی از 3 تکنیک آرام‌سازی پرسمان (**Query Relaxation**)، بسط معنایی

گام اجرایی	شرح مکانیزم و الگوریتم بازیابی
گام 1 مدیریت خطا (Timeout Detection)	تشخیص و تغییر مسیر: گره مرکزی (Broker) برای هر درخواست به گره‌های کارگر، 1 محدودیت زمانی سخت (مثلاً 50 میلی‌ثانیه) تعیین می‌کند. به محض پایان این زمان و عدم پاسخ از گره کلمه «الگوریتم»، گره مرکزی این کلمه را به عنوان «گره مرده» (Dead Node) نشانه گذاری می‌کند تا کل پرسمان مسدود نشود.
گام 2 آرام‌سازی پرسمان (Query Relaxation)	تغییر عملگرهای منطقی: در پرسمان‌های چندکلمه‌ای (مثل «الگوریتم هوش مصنوعی»), سیستم به صورت پویا، کلمه ازدست‌رفته را از شروط الزامی (AND) حذف کرده و پرسمان را تقلیل می‌دهد. سیستم اکنون صرفاً به دنبال اسنادی می‌گردد که حاوی «هوش مصنوعی» هستند. این کار تضمین می‌کند که خروجی پرسمان هرگز صفر نشود.
گام 3 جایگزینی معنایی (Synonym Substitution)	استفاده از گره‌های سالم: برای جبران حذف کلمه «الگوریتم» (و به ویژه برای پرسمان‌های تک‌کلمه‌ای)، گره مرکزی به 1 دیکشنری سبک و درون‌حافظه‌ای (In-memory Thesaurus/Word2Vec) مراجعه می‌کند. سیستم کلمات مترادفی مانند «روش»، «محاسبه» یا «فلوچارت» را استخراج می‌کند. از آنجا که این کلمات روی گره‌های سالم دیگری قرار دارند، گره مرکزی پرسمان جدید را با این مترادف‌ها بازنویسی کرده و به گره‌های سالم ارسال می‌کند.
گام 4 تجمیع نتایج (Aggregation & UX)	حفظ اعتماد کاربر: گره مرکزی نتایج جایگزین‌شده را دریافت کرده، آن‌ها را بر اساس مدل‌های رتبه‌بندی (مانند BM25) مرتب می‌کند. نکته حیاتی: برای حفظ شفافیت، سیستم این نتایج را به همراه 1 پیام هشدار (Warning Snippet) در رابط کاربری نمایش می‌دهد. مثلاً: "به دلیل اختلال موقت در پردازش کلمه «الگوریتم»، نتایج برای کلمات مشابه («هوش مصنوعی»، «روش‌های محاسباتی») نمایش داده شده است."

مدل‌سازی شماتیک: خط‌لوله بازیابی (Fallback Pipeline)



نتیجه‌گیری تحلیلی:

در غیاب Replication، داده‌ها به صورت فیزیکی غیرقابل دسترس هستند و هیچ جادویی نمی‌تواند لیست پستینگ ازدست‌رفته را برگرداند. هنر مهندسی در چنین مواقعی، تغییر رویکرد از "تطابق دقیق متنی" (Exact Match) به "تطابق مفهومی و جزئی" (Soft/Semantic Match) در لایه مسیریاب (Broker) است تا قابلیت اطمینان سیستم (Reliability) از دید کاربر نهایی حفظ شود و از ارائه 1 صفحه خطای مطلق جلوگیری گردد.

مقدمه: پیش‌نیاز فشرده‌سازی در گراف وب

در سرورهای اتصال (**Connectivity Servers**) و همچنین در ساختار نمایه وارونه (**Inverted Index**)، شناسه‌های اسناد (**DocIDs**) در هر لیست مجاورت معمولاً به صورت صعودی مرتب می‌شوند.

با توجه به پدیده «مکان‌مندی ارجاع» (**Locality of Reference**) در وب، صفحاتی که از نظر ساختاری به هم نزدیک‌اند (مثلاً در 1 دامنه یا سایت قرار دارند)، شناسه‌های عددی نزدیک به هم دریافت می‌کنند. تکنیک **رمزگذاری فاصله یا دلتا (Delta Encoding / Gap Encoding)** از همین ویژگی استفاده می‌کند تا به جای ذخیره اعداد بزرگ مطلق، صرفاً «فاصله» بین آن‌ها را ذخیره کند.

بخش 1) پیاده‌سازی الگوریتم رمزگذاری دلتا

```
def delta_encode(doc_ids: list[int]) -> list[int]:
    """
    دریافت 1 لیست مرتب‌شده از شناسه‌های اسناد و تبدیل آن به فرمت رمزگذاری دلتا.
    """
    # اگر لیست خالی باشد (Edge case): بررسی حالت لبه
    if not doc_ids:
        return []
    # عنصر اول همیشه بدون تغییر باقی می‌ماند (نقطه مرجع)
    delta_encoded = [doc_ids[0]]
    # پیمایش آرایه از عنصر دوم تا انتها
    for i in range(1, len(doc_ids)):
        # محاسبه فاصله (گپ) بین عنصر فعلی و عنصر قبلی
        gap = doc_ids[i] - doc_ids[i - 1]
        delta_encoded.append(gap)
    return delta_encoded

# تست و اجرای الگوریتم بر اساس صورت مسئله
adjacency_list = [1020, 1025, 1035, 1050, 1055]
encoded_list = delta_encode(adjacency_list)

print("Original Adjacency List: ", adjacency_list)
print("Delta Encoded List:      ", encoded_list)
```

خروجی اجرای کد:

Original Adjacency List: [1020, 1025, 1035, 1050, 1055]

Delta Encoded List: [1020, 5, 10, 15, 5]

بخش 2) تحلیل مهندسی: چرا دلتا برای کدهای طول متغیر (VLC) مناسب است؟

عملیات رمزگذاری دلتا (تفریق اعداد) به تنهایی حجم حافظه را در ساختارهای داده استاندارد کاهش نمی‌دهد (زیرا عدد 5 و عدد 1055 هر 2 در 1 متغیر 32 بیتی ذخیره می‌شوند). جادوی واقعی زمانی رخ می‌دهد که خروجی الگوریتم بالا، به عنوان ورودی به الگوریتم‌های رمزگذاری طول متغیر (**Variable-length Codes**) مانند **V-Byte** (بایت متغیر) یا **Elias Gamma** داده شود.

دلیل این هم‌افزایی بی‌نظیر، در 3 منطق ریاضی و سخت‌افزاری زیر نهفته است:

در حالت عادی، کامپیوتر برای ذخیره هر شناسه سند (مانند عدد 1055) 1 بلوک کامل 32 بیتی (4 بایتی) اختصاص می‌دهد. اگر لیست ما شامل 1 میلیون سند با شناسه‌هایی در محدوده میلیارد باشد، به 4 مگابایت حافظه فقط برای همین لیست نیاز داریم. اما کدهای طول متغیر به سیستم اجازه می‌دهند که از تعداد بیت‌های شناور بر اساس "اندازه عدد" استفاده کنند.	الف) رهایی از محدودیت 32 بیتی (Hardware Independence)
الگوریتم‌های طول متغیر (مانند گاما کد)، برای ذخیره اعداد کوچکتر به بیت‌های بسیار کمتری نیاز دارند.	ب)

کاهش ابعاد اعداد (Domain Reduction)	• ذخیره عدد مطلق 1055 در کد گاما نیازمند 21 بیت است. • ذخیره عدد دلتای 5 در کد گاما تنها نیازمند 5 بیت است.
ج) فشرده‌سازی تصاعدی (Exponential Compression)	به دلیل مرتب بودن لیست و مکان‌مندی وب، بیش از 80% فواصل (گپ‌ها) در گراف وب اعدادی کوچکتر از 15 هستند. این مسئله باعث می‌شود حجم داده‌ها پس از تبدیل به دلتا، در ترکیب با VLC به شدت فرو بریزد.

مدل‌سازی شماتیک: شگفتی فشرده‌سازی دلتا + VLC

System State	Array Sequence	Required Memory Allocation
Before Delta (Standard 32-bit)	[1000001, 1000002, 1000003, 1000004]	32 + 32 + 32 + 32 = 128 Bits
After Delta + VLC (Elias Gamma)	[1000001, 1, 1, 1]	32 + 1 + 1 + 1 = 35 Bits! (Nearly 75% Compression)

نتیجه‌گیری پایانی برای دانشجویان:

رمزگذاری دلتا (Delta Encoding) پیش‌شرط و آماده‌ساز داده‌ها برای فشرده‌سازی است. این تکنیک، اعداد بزرگ و تصادفی را به «اعداد کوچک و تکراری» تبدیل می‌کند تا الگوریتم‌های طول متغیر بتوانند با تخصیص بیت‌های کمتر به اعداد کوچکتر، حجم گراف ده‌ها اگزابایتی وب را به چند گیگابایت در حافظه RAM تقلیل دهند و پردازش اطلاعات را برای موتورهای جستجو ممکن سازند.

پاسخنامه مسئله 20.25: رمزگذاری ارجاعی با بلوک‌های همسایه (Reference Encoding)

مقدمه: چرا رمزگذاری ارجاعی (Reference Encoding) معجزه می‌کند؟

در گراف وب، صفحات بر اساس حروف الفبای آدرس‌هایشان (URL) مرتب می‌شوند. این مرتب‌سازی باعث می‌شود صفحاتی که شماره شناسایی (DocID) متوالی یا نزدیک به هم دارند، معمولاً متعلق به یک وب‌سایت واحد باشند (مثلاً site.com/page1 و site.com/page2).

از آنجا که صفحات یک سایت از الگوهای ناوبری (منوها، فوترها و لینک‌های ثابت) مشترکی استفاده می‌کنند، لیست پیوندهای خروجی آن‌ها به شدت شبیه به هم است. تکنیک رمزگذاری ارجاعی (طراحی شده توسط Boldi و Vigna) به جای ذخیره مجدد این لینک‌های تکراری در حافظه، به سادگی به لیست صفحه قبلی ارجاع می‌دهد و از نظر منطقی بیان می‌کند: "من هم دقیقاً همان لینک‌های صفحه قبلی را دارم، به جز چند تغییر کوچک!"

بخش 1: تشریح الگوریتم و محاسبه خروجی

فرض مسئله به این شرح است:

لیست مرجع (Reference List - صفحه 99): [8, 15, 22, 35, 40]

لیست هدف (Target List - صفحه 100): [8, 15, 28, 35, 45]

برای فشرده‌سازی لیست 100 با استفاده از لیست 99، الگوریتم 3 گام ریاضی زیر را طی می‌کند:

گام 1: تولید بیت‌های کنترلی (Copy Vector)

الگوریتم، لیست هدف را با لیست مرجع مقایسه می‌کند. به ازای هر پیوند موجود در لیست مرجع (صفحه 99)، یک بیت کنترلی (0 یا 1) تولید می‌شود:

اگر آن پیوند در لیست صفحه 100 نیز وجود داشته باشد، بیت 1 (کپی کن) ثبت می‌شود.

اگر آن پیوند در لیست صفحه 100 وجود نداشته باشد، بیت 0 (نادیده بگیر) ثبت می‌شود.

گام 2: استخراج اضافی‌ها (Residuals)

پس از کپی‌برداری از گره‌های مشترک، الگوریتم باید بررسی کند که آیا صفحه 100 پیوند مستقلی دارد که در صفحه 99 نبوده است؟ این پیوندهای جدید به عنوان «لیست باقیمانده‌ها» (Residual List) جدا می‌شوند. در این مثال، اعداد 28 و 45 در لیست مرجع وجود ندارند.

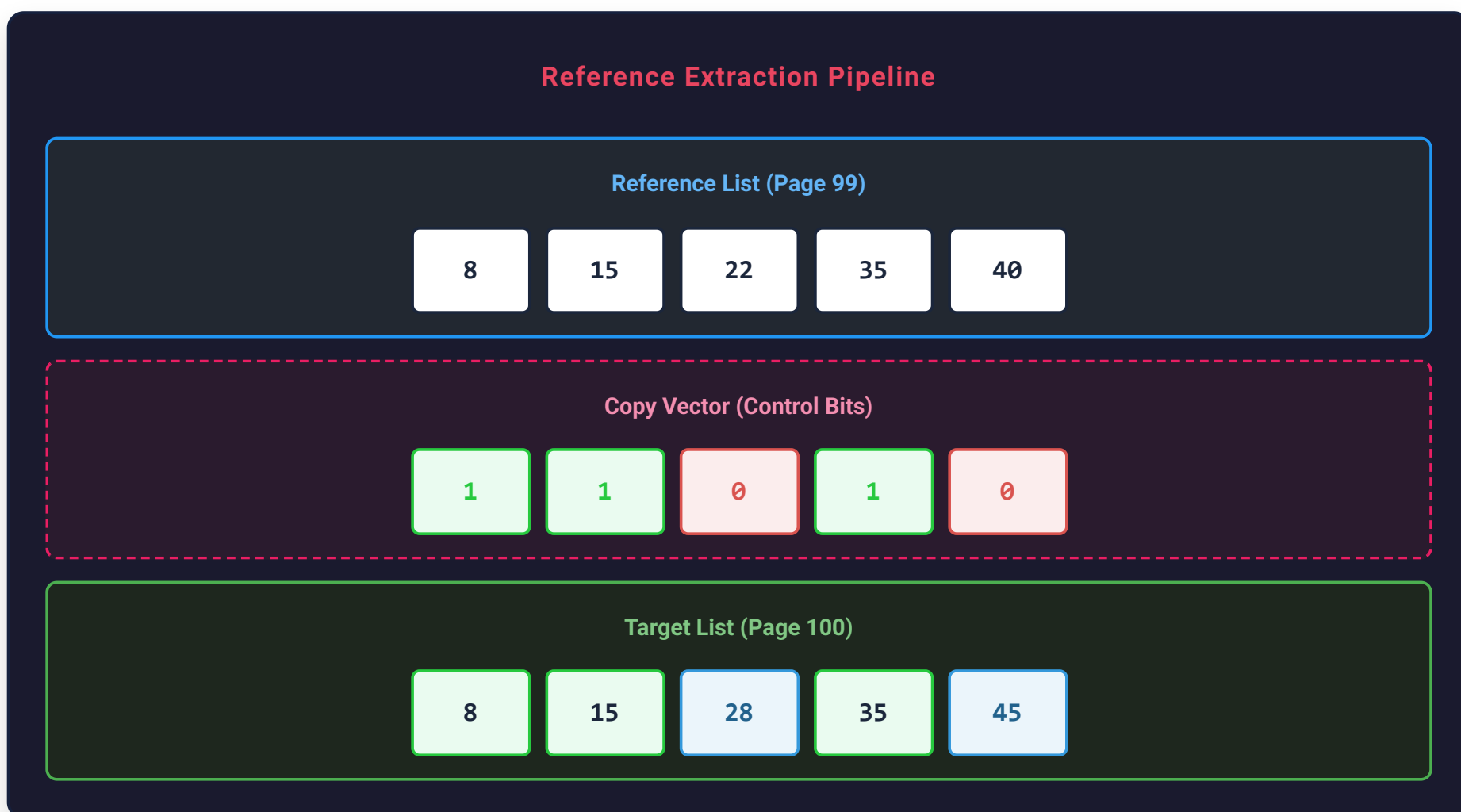
گام 3: ادغام و دلتا کدینگ (Delta Compression)

خروجی نهایی شامل یک اشاره‌گر به گره مرجع، بیت‌های کنترلی برای کپی کردن داده‌ها و در نهایت لیست باقیمانده‌هاست. برای حداکثر فشرده‌سازی، لیست باقیمانده‌ها نیز از فیلتر «رمزگذاری دلتا» (Delta Encoding) عبور داده می‌شود.

بخش 2: محاسبه دقیق مقادیر و فرآیند Reference Encoding

بر اساس گام‌های فوق، محاسبات دقیق برای لیست صفحه 100 به شکل زیر است:

1. محاسبه بیت‌های کنترلی (Copy Bitmap)



بنابراین، رشته بیت‌های کنترلی برابر با 11010 است.

2. محاسبه پیوندهای اضافی (Residuals)

با کپی کردن بیت‌های 1، تا این لحظه پیوندهای [8, 15, 35] برای صفحه 100 ساخته شده است.

پیوندهای موجود در صفحه 100 که در صفحه 99 نبوده‌اند عبارتند از: [28, 45]. این اعداد لیست باقیمانده‌ها را تشکیل می‌دهند.

3. دلتا کدینگ باقیمانده‌ها

برای فشرده‌سازی بیشتر، لیست باقیمانده‌ها [28, 45] به فواصل دلتا تبدیل می‌شود:

عضو اول تغییر نمی‌کند: 28

عضو دوم تفریق می‌شود: $45 - 28 = 17$

لیست باقیمانده‌های نهایی (Residual Deltas) برابر با [28, 17] خواهد بود.

بخش 3: خروجی نهایی الگوریتم (Output Format)

اگر خزشگر بخواهد لیست مجاورت صفحه 100 را روی دیسک (یا RAM) ذخیره کند، به جای ذخیره 5 عدد 32 بیتی بزرگ [8, 15, 28, 35, 45]، دقیقاً مقادیر ساختاریافته زیر را ذخیره می‌کند:

1. فاصله تا گره مرجع: 1 (نشان‌دهنده اینکه از گره قبلی یعنی $99 = 100 - 1$ الگوبرداری کن).

2. بیت‌های کنترلی (Copy Vector): 11010 (یک رشته 5 بیتی بسیار سبک).

3. تعداد باقیمانده‌ها: 2 عدد.

4. مقادیر باقیمانده فشرده‌شده: [28, 17].

بخش 4: جمع‌بندی

تکنیک رمزگذاری ارجاعی (Reference Encoding) که در چارچوب فریم‌ورک‌های متراکم‌سازی گراف نظیر WebGraph معرفی شده است، یک شاهکار مهندسی در تقلیل افزونگی داده‌ها (Data Redundancy) است. در این فرآیند ساختاریافته، می‌بینیم که چگونه 3 عدد بزرگ صحیح (یعنی 8، 15 و 35) به طور کامل از حافظه فیزیکی حذف شده و صرفاً با 3 بیتِ ارزان‌قیمت جایگزین گردیدند.

در معماری موتورهای جستجو در مقیاس وب، با ترکیب همزمان این تکنیک مبتنی بر ارجاع به بلوک‌های همسایه و پروتکل‌های رمزگذاری نظیر Variable-Byte Encoding (برای لیست باقیمانده‌های دلتا)، حجم گراف پیوندهای وب تا 90 درصد فشرده می‌شود. این تراکم بی‌نظیر، بارگذاری و نگهداری ساختار گول‌پیکر شبکه جهانی اینترنت را در حافظه اصلی (RAM) خوشه‌های سرور امکان‌پذیر ساخته و محاسبه درنگام الگوریتم‌هایی نظیر PageRank را با تأخیر نزدیک به صفر عملیاتی می‌سازد.

پاسخنامه مسئله 20.26: مدیریت بن‌بست در استخراج موازی (Deadlock in Parallel Extraction)

مقدمه: ریشه‌یابی بن‌بست (Deadlock) در معماری حافظه اشتراکی

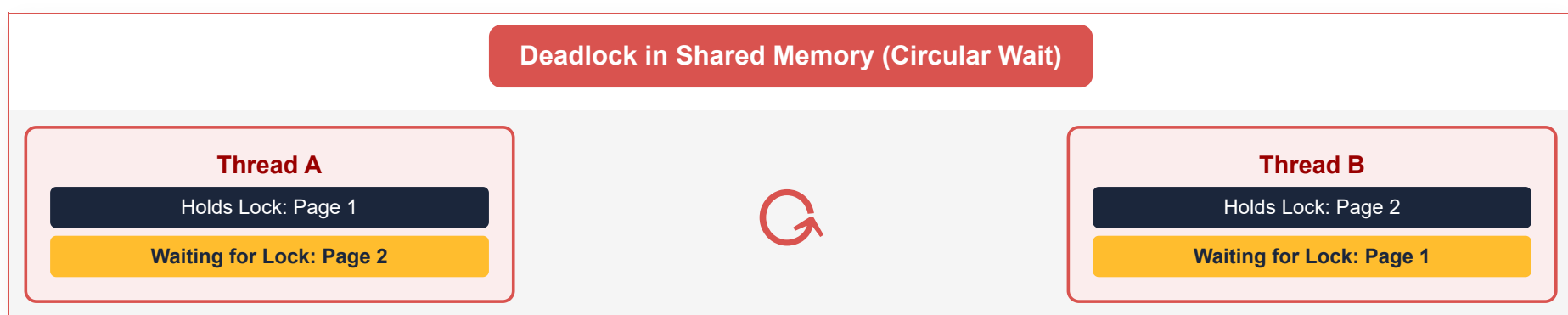
در 1 خزشگر توزیع شده، اگر تمامی نخ‌های کارگر (**Worker Threads**) بخواهند 1 جدول متمرکز **URL_Seen** را به صورت مستقیم (**Synchronous**) بررسی و به‌روزرسانی کنند، سیستم به شدت در برابر بن‌بست (**Deadlock**) و گلوگاه‌های رقابتی آسیب‌پذیر می‌شود.

در سناریوی صورت مسئله، پدیده بن‌بست به این شکل رخ می‌دهد (شرط انتظار چرخشی **Circular Wait**):

1. نخ A در حال پردازش صفحه 1 است و قفل رکورد صفحه 1 را در اختیار دارد. همزمان متوجه لینک به صفحه 2 می‌شود و درخواست قفل برای رکورد صفحه 2 را می‌دهد.

2. نخ B در حال پردازش صفحه 2 است و قفل رکورد صفحه 2 را در اختیار دارد. همزمان متوجه لینک به صفحه 1 می‌شود و درخواست قفل برای رکورد صفحه 1 را می‌دهد.

3. هر 2 نخ تا ابد منتظر آزاد شدن قفل توسط دیگری می‌مانند و سیستم کاملاً فلج می‌شود.



برای حل این بحران، باید پارادایم معماری را از حالت حافظه اشتراکی (**Shared Memory**) به حالت ارسال پیام (**Message Passing**) و معماری هیچ‌چیز-اشتراکی (**Shared-Nothing**) تغییر دهیم.

تغییر معماری: حذف قفل‌ها با هشینگ و صف‌بندی پیام

راهکار اساسی این است که دسترسی مستقیم نخ‌های واکنشی‌کننده به جدول **URL_Seen** را به طور کامل قطع کنیم. این معماری جدید طی 4 گام مهندسی پیاده‌سازی می‌شود:

گام اجرایی	شرح مکانیزم مهندسی
گام 1 قطعه‌بندی جدول (Sharding)	حذف تمرکز پایگاه داده: به جای داشتن 1 جدول عظیم URL_Seen ، این ساختار داده را به N قطعه (Shard) کاملاً مستقل و مجزا تقسیم می‌کنیم (مثلاً 64 قطعه). این قطعات می‌توانند روی سرورهای مختلف یا در بخش‌های مجزای حافظه قرار گیرند و هیچ ارتباطی با یکدیگر ندارند.
گام 2 توزیع با هشینگ (Consistent Hashing)	مسیریابی قطعی و بدون تداخل: هنگامی که نخ کارگر 1 URL جدید استخراج می‌کند، به جای تلاش برای ثبت آن در پایگاه داده، 1 تابع هش یکنواخت (مانند MurmurHash) روی رشته متنی آن URL اجرا می‌کند. خروجی تابع هش را به پیمانه N محاسبه می‌کند: $\text{Shard_ID} = \text{Hash}(\text{URL}) \pmod{N}$ این فرمول ریاضی تضمین می‌کند که مسئولیت بررسی وضعیت 1 URL خاص (مثلاً صفحه 1)، همیشه و تحت هر شرایطی فقط بر عهده 1 قطعه ثابت است.

شرح مکانیزم مهندسی	گام اجرایی
تغییر رویکرد همگام به غیرهمگام: در این مرحله، نخ کارگر صبر نمی‌کند تا ببیند آیا URL تکراری است یا خیر. بلکه صرفاً 1 پیام حاوی دستور (CHECK_AND_INSERT, URL) را تولید کرده و آن را درون 1 صف پیام غیرهمگام (Async Message Queue) که اختصاصاً به آن قطعه متصل است، رها می‌کند (رویکرد Fire-and-Forget). سپس بلافاصله به استخراج لینک‌های بعدی می‌پردازد.	گام 3 صف‌بندی پیام‌ها (Message Queuing)
راز حذف قفل‌ها: هر قطعه (Shard) دقیقاً و فقط 1 نخ اختصاصی دارد که پیام‌ها را یکی‌یکی از صف خود برداشته و در ساختار داده (مثلاً فیلتر بلوم آن قطعه) بررسی و ثبت می‌کند. از آنجا که فقط همین 1 نخ حق نوشتن و خواندن از آن قطعه را دارد، امکان تداخل داده (Race Condition) مطلقاً صفر است و در نتیجه هیچ نیازی به پیاده‌سازی قفل (Lock) وجود ندارد.	گام 4 پردازش تک‌نخی (Single-Threaded)

تحلیل سیستم جدید در برابر بن‌بست

بیا ببینیم سناریوی صورت مسئله را در معماری جدید شبیه‌سازی کنیم:

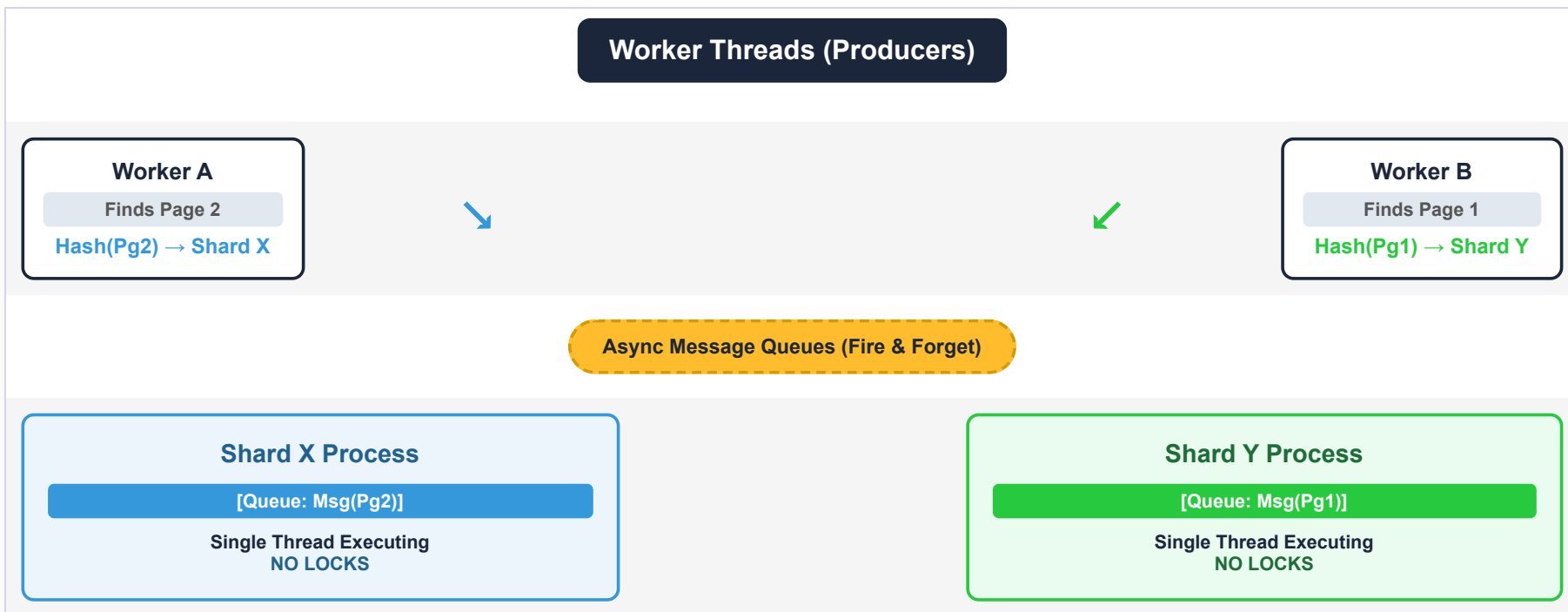
• نخ **A** صفحه **2** را کشف می‌کند. هش صفحه **2** محاسبه شده و پیام آن به صف قطعه شماره **X** ارسال می‌شود.

• نخ **B** صفحه **1** را کشف می‌کند. هش صفحه **1** محاسبه شده و پیام آن به صف قطعه شماره **Y** ارسال می‌شود.

چرا بن‌بست غیرممکن است؟

نخ‌های **A** و **B** هیچ درخواستی برای مسدود کردن رکوردی نمی‌دهند و اصلاً منتظر پاسخ نمی‌مانند تا بخواهند وارد چرخه انتظار متقابل شوند. از سوی دیگر، نخ پردازشی قطعه **X** و نخ قطعه **Y** نیز کاملاً مستقل از هم پیام‌های خود را پردازش می‌کنند. تمام قفل‌های سطح رکورد (**Record-level Locks**) حذف شده‌اند و گلوگاه ترافیکی به صورت ریاضی بی‌اثر شده است.

مدل‌سازی شماتیک پیشرفته: معماری ارتباط پیام (Lock-Free Message Passing)



نتیجه‌گیری پایانی:

در طراحی سیستم‌های توزیع‌شده با مقیاس وب، استفاده از قفل (**Locking**) برای همگام‌سازی، ضدالگویی (**Anti-pattern**) است که مقیاس‌پذیری را به صورت مطلق نابود می‌کند. ترکیب هوشمندانه قطعه‌بندی (**Partitioning**)، توابع درهم‌سازی و صف‌های پیام (**Message Queues**) تضمین می‌کند که داده‌ها از نظر منطقی به گونه‌ای

مسیردهی شوند که پردازش همزمان، به پردازش مستقلِ تکنخی تبدیل شده و ریسک بن‌بست و رقابت از ریشه خشکانده شود.

پاسخنامه تشریحی مسئله 20.27: بهینه‌سازی کوئری‌های چندکلمه‌ای توزیع‌شده

(Distributed Query Optimization)

مقدمه: فاجعه ترافیک در معماری یک‌مرحله‌ای

در معماری توزیع سند-محور (**Document-Partitioned**)، اسناد بین N گره مختلف (در اینجا 50 گره) تقسیم شده‌اند. در 1 رویکرد ساده‌لوحانه (**Naïve Approach**)، گره مرکزی کوئری را به تمام 50 گره ارسال می‌کند. هر گره، پرسمان را روی اسناد خود اجرا کرده و رکورد کامل 1000 نتیجه برتر خود را (شامل شناسه سند، امتیاز، عنوان، آدرس URL و قطعه‌متن یا **Snippet**) به گره مرکزی برمی‌گرداند.

تحلیل حجم ترافیک:

اگر هر رکورد کامل نمایشی (**Full Record**) به طور میانگین 1000 بایت حجم داشته باشد، حجم داده‌های منتقل شده در شبکه برای 1 کوئری برابر است با:

$$50 \text{ Nodes} \times 1000 \text{ Records} \times 1000 \text{ Bytes} = 50,000,000 \text{ Bytes} \approx 50 \text{ MB}$$

انتقال 50 مگابایت داده روی شبکه کلاستر فقط برای 1 کوئری و استخراج 10 نتیجه، باعث ایجاد گلوگاه شبکه (**Network Bottleneck**) و نابودی کامل توان عملیاتی موتور جستجو می‌شود.

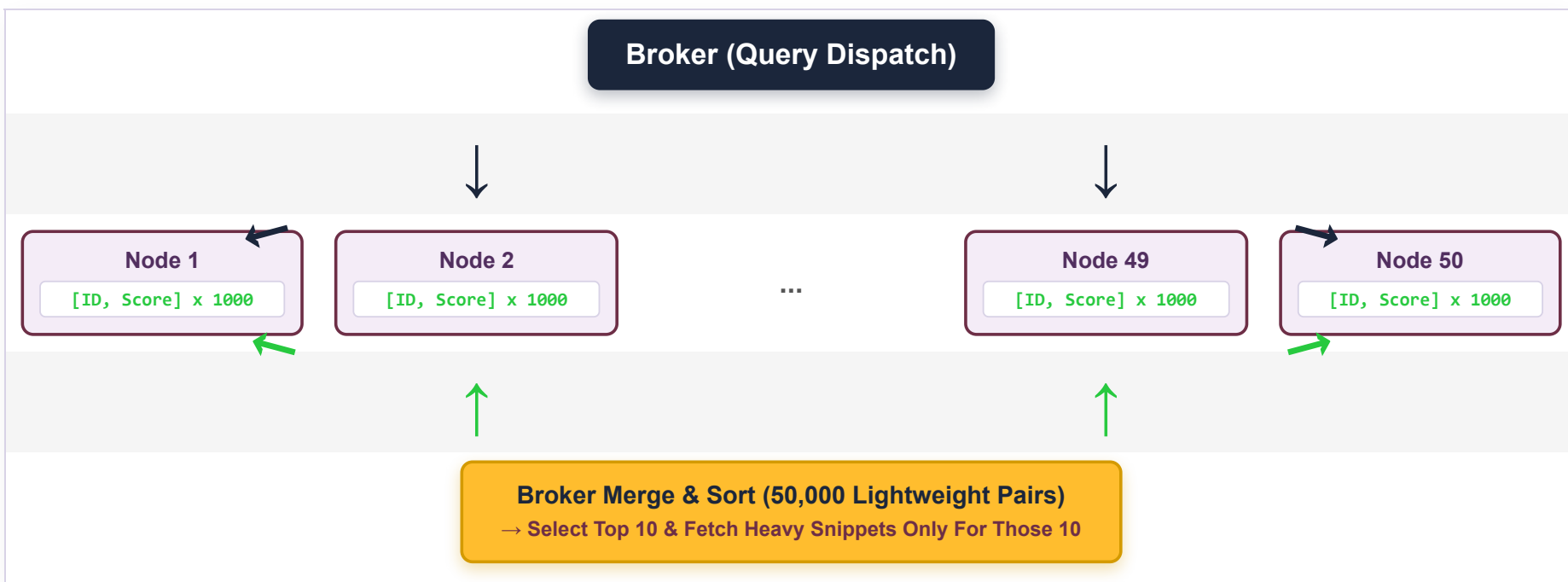
راهکار مهندسی: الگوریتم واکنشی 2 مرحله‌ای (Two-Phase Query Processing)

برای حل این چالش، معماران سیستم از الگوریتم 2 مرحله‌ای «امتیازدهی-سپس-واکنشی» (**Score-then-Fetch**) استفاده می‌کنند. این الگوریتم شباهت بنیادینی به پارادایم **MapReduce** (پخش‌کردن و جمع‌کردن) دارد و از انتقال داده‌های سنگین پیش از تعیین برندگان نهایی جلوگیری می‌کند.

گام اجرایی	شرح مکانیزم و الگوریتم پردازشی
فاز 1 انتخاب نامزدها (Map Operation)	گره مرکزی (Broker) کوئری "پردازش سیگنال پیشرفته" را به 50 گره ارسال می‌کند و از آن‌ها می‌خواهد که فقط نتایج سبک (Lightweight Records) را برگردانند. هر گره 1000 نتیجه برتر خود را پیدا می‌کند، اما به جای ارسال کل داده‌های صفحه، فقط 1 آرایه دو-تایی شامل [DocID, Score] را به گره مرکزی ارسال می‌کند. این آرایه فاقد هرگونه متن، عنوان یا URL است.
مرحله میانی تجمیع و رتبه‌بندی (Reduce Operation)	گره مرکزی اکنون 50,000 جفت [DocID, Score] را دریافت کرده است. از آنجا که این داده‌ها بسیار سبک هستند، به سرعت در حافظه رم گره مرکزی ادغام (Merge) شده و بر اساس Score به صورت نزولی مرتب می‌شوند. پس از مرتب‌سازی، گره مرکزی 10 سند برتر نهایی (Global Top 10) را از بین این 50,000 رکورد انتخاب کرده و مابقی 49,990 رکورد را دور می‌ریزد.

گام اجرایی	شرح مکانیزم و الگوریتم پردازشی
فاز 2 واکشی نمایشی (Fetch Operation)	گره مرکزی اکنون دقیقاً می‌داند کدام 10 سند برنده شده‌اند و هر کدام در کدام گره فیزیکی قرار دارند. در این مرحله، گره مرکزی 1 درخواست نقطه-به-نقطه (Point-to-Point) فقط به گره‌های دارنده این 10 سند ارسال می‌کند و از آن‌ها می‌خواهد که رکوردهای سنگین و کامل (متن، عنوان و Snippet) را منحصراً برای همین 10 شناسه سند ارسال کنند.

مدل‌سازی شماتیک پیشرفته: خط‌لوله 2 مرحله‌ای (Two-Phase Execution)



اثبات ریاضی: کاهش ترافیک و حفظ دقت

1. محاسبه کاهش ترافیک شبکه

فرض کنید اندازه 1 رکورد سبک [DocID, Score] برابر با 8 بایت و اندازه 1 رکورد سنگین (شامل متون نمایشی) برابر با 1000 بایت باشد.

ترافیک معماری یک مرحله‌ای (ساده‌لوحانه)	$50 \times 1000 \times 1000 = 50,000,000 \text{ Bytes} \approx 50 \text{ MB}$
ترافیک معماری 2 مرحله‌ای (بهینه‌شده)	Phase 1: $50,000 \times 8 \text{ Bytes} = 400,000 \text{ Bytes}$ Phase 2: $10 \times 1000 \text{ Bytes} = 10,000 \text{ Bytes}$ Total: $410,000 \text{ Bytes} \approx 400 \text{ KB}$

نتیجه: حجم داده‌های منتقل‌شده از 50 MB به 400 KB کاهش یافته است. این یعنی بیش از 99 درصد کاهش ترافیک، که بسیار فراتر از شرط 50 درصدی صورت مسئله است.

2. اثبات حفظ دقت (Accuracy Guarantee)

چرا دقت نتایج نهایی (Top 10) در این الگوریتم افت نمی‌کند؟

زیرا امتیازدهی به صورت قطعی (**Deterministic**) روی گره‌های محلی انجام می‌شود. اگر سندی در رتبه‌بندی سراسری (**Global**) در جایگاه اول تا دهم باشد، از نظر ریاضی **غیرممکن است** که در رتبه‌بندی محلی (**Local**) گره خودش، رتبه‌ای بدتر از دهم داشته باشد.

بنابراین، با ارسال 1000 نتیجه برتر از هر گره در فاز اول، اطمینان 100 درصدی داریم که هیچیک از نامزدهای اصلی لیست Top 10 سراسری از قلم نیفتاده‌اند.

نکته پیشرفته معماری:

برای اینکه امتیازات گره‌ها با هم کاملاً قابل قیاس باشند، موتورهای جستجو پیش از اجرای کوئری، متغیرهای آماری سراسری مانند Global IDF را بین تمام گره‌ها همگام‌سازی (Sync) می‌کنند تا ارزش هر کلمه در تمام 50 گره با 1 استاندارد یکپارچه محاسبه گردد.

پاسخنامه مسئله 20.28: سناریوی محاسبه پیچ‌رنک در حافظه محدود (PageRank in Limited RAM)

مقدمه: بحران دسترسی تصادفی دیسک (Random I/O Thrashing)

در محاسبه الگوریتم رتبه صفحه (PageRank)، فرمول پایه به این شکل است:

$$PR_{new}(u) = \sum_{v \in B_u} \frac{PR_{old}(v)}{L(v)}$$

در این فرمول، برای محاسبه امتیاز جدید گره u ، باید امتیاز قبلی تمام گره‌هایی که به آن لینک داده‌اند (v) را از بردار PR_{old} استخراج کنیم.

برای 1 گراف وب با 1 میلیارد گره، اگر هر مقدار PageRank 1 عدد اعشاری مضاعف (Double - 8 Bytes) باشد، بردار PR_{old} حدود 8 گیگابایت و بردار PR_{new} نیز 8 گیگابایت (مجموعاً 16 گیگابایت) فضا اشغال می‌کند. ساختار گراف (لیست مجاورت) نیز ده‌ها گیگابایت حجم دارد. سرور 32 گیگابایتی ما نمی‌تواند همه این داده‌ها را به همراه سیستم‌عامل و سایر فرآیندها همزمان در حافظه RAM نگه دارد و باید به دیسک سخت متوسل شود.

ریشه گلوگاه:

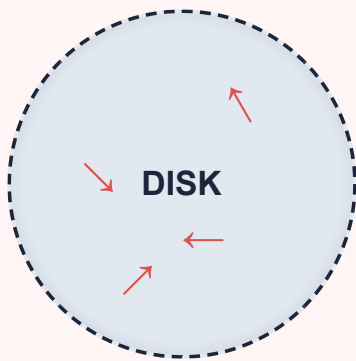
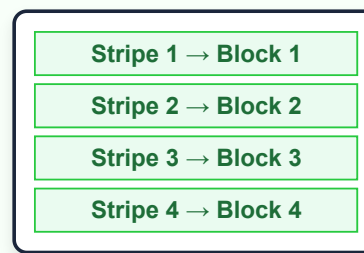
از آنجا که پیوندهای وب کاملاً پراکنده و درهم‌تنیده هستند، خواندن مقادیر $PR_{old}(v)$ از روی دیسک نیازمند میلیاردها پرش تصادفی (Random Disk Seek) است. دیسک‌های سخت در دسترسی تصادفی به شدت کند عمل می‌کنند و این رویکرد باعث قفل‌شدگی سیستم در اثر بارگذاری مداوم از دیسک (Disk Thrashing) می‌شود و محاسبه 1 تکرار (Iteration) از الگوریتم ممکن است هفته‌ها طول بکشد.

استراتژی Block-Stripe Update (تکنیک Haveliwala)

برای حل این بحران، از معماری Block-Stripe Update استفاده می‌کنیم. ایده کلیدی این الگوریتم، تبدیل تمام دسترسی‌های تصادفی به دسترسی‌های کاملاً ترتیبی و جریانی (Sequential I/O) با استفاده از قطعه‌بندی حافظه و پیش‌پردازش یال‌هاست. اجرای این الگوریتم در سرور اتصال (Connectivity Server) طی گام‌های زیر انجام می‌شود:

گام اجرایی	شرح مکانیزم و الگوریتم فیزیکی دیسک
گام 1 قطعه‌بندی گره‌ها (Node Partitioning)	مدیریت محدودیت حافظه: کل N گره گراف به K بلوک مساوی تقسیم می‌شوند، به گونه‌ای که بردار PR_{new} برای تنها 1 بلوک به راحتی در حافظه RAM جا شود. در سناریوی ما (1 میلیارد گره)، می‌توانیم آن را به 4 بلوک 250 میلیونی تقسیم کنیم ($K = 4$). با این کار، هر بلوک بردار جدید فقط به حدود 2 گیگابایت RAM نیاز خواهد داشت.
گام 2 مرتب‌سازی یال‌ها (Edge Bucketing)	آماده‌سازی داده‌ها برای خواندن خطی: سرور اتصال، ساختار گراف را 1 بار برای همیشه بازآرایی می‌کند. یال‌ها بر اساس گره مقصد (Destination Node) مرتب شده و در K فایل یا نوار (Stripe) جداگانه روی دیسک ذخیره می‌شوند. به عنوان مثال، Stripe شماره 1 فقط شامل یال‌هایی است که به گره‌های موجود در بلوک 1 اشاره می‌کنند. درون هر Stripe نیز، یال‌ها بر اساس گره مبدا مرتب می‌گردند.
گام 3 بارگذاری ترتیبی (Sequential Streaming)	حذف پرش‌های تصادفی: اکنون سیستم محاسبه را بلوک به بلوک آغاز می‌کند. برای محاسبه بلوک i : - آرایه 2 گیگابایتی PR_{new} مربوط به بلوک i در RAM تخصیص یافته و با صفر مقداردهی می‌شود. - فایل نوار یال‌ها (Stripe i) و بردار کامل 8 گیگابایتی PR_{old} به صورت همزمان و کاملاً ترتیبی (Streaming) از دیسک خوانده می‌شوند. چون یال‌های درون Stripe i بر اساس گره مبدا مرتب شده‌اند، هدایتگر دیسک (Disk Head) هر 2 فایل را خط به خط و به سمت جلو می‌خواند و هیچ نیازی به بازگشت یا پرش (Seek) ندارد.
گام 4 نوشتن ترتیبی (Sequential Write)	پایان پردازش بلوک: پس از پایان یافتن خواندن Stripe i ، مقادیر بردار PR_{new} برای آن بلوک کاملاً محاسبه شده است. این بلوک 2 گیگابایتی به صورت یکجا و کاملاً ترتیبی روی دیسک ذخیره (Flush) می‌شود و RAM برای پردازش بلوک $i + 1$ آزاد می‌گردد.

مدل‌سازی شماتیک پیشرفته: اثبات مهندسی حذف گلوگاه

NAÏVE APPROACH (RANDOM I/O)	BLOCK-STRIPE (SEQUENTIAL I/O)
 Disk Head jumps millions of times. System locks up (Thrashing).	 Disk Head reads continuously. Maximum transfer speed utilized.

مقایسه هزینه I/O

شاخص ارزیابی	روش ساده‌لوحانه (Naive)	روش Block-Stripe
الگوی دسترسی به دیسک	میلیاردها پرش تصادفی (Random Seek)	کاملاً خطی و جریانی (Sequential Scan)
دفعات خواندن گراف از دیسک	1 بار به صورت پراکنده و بسیار کند	1 بار خواندن کل K نوار به صورت ترتیبی
دفعات خواندن بردار PR_{old}	ده‌ها بار خواندن تصادفی هر بلوک	دقیقاً K بار خواندن کامل و ترتیبی
گلوگاه اصلی سیستم	مکانیک دیسک سخت (قفل‌شدگی مطلق)	پهنای باند گذرگاه دیسک و سرعت CPU

نتیجه‌گیری پایانی:

با استفاده از الگوریتم **Block-Stripe**، ما زمان اجرای 1 تکرار (**Iteration**) از الگوریتم **PageRank** را از مرتبه «زمان پرش دیسک» (**Seek Time**) به مرتبه «زمان انتقال داده» (**Transfer Rate**) کاهش می‌دهیم، که در سرورهای مدرن هزاران بار سریع‌تر است و امکان پردازش گراف‌های میلیارد گرهی را روی سخت‌افزارهای بسیار محدود نیز فراهم می‌کند.

پاسخنامه مسئله 20.29: شبیه‌سازی صف‌های ادب با لاگ‌های خروجی

(Politeness Queue Simulation)

مقدمه: ارزیابی جعبه‌سیاه (Black-box Testing) در سیستم‌های توزیع‌شده

در سیستم‌های ناهمگام (**Asynchronous**) مانند معماری صف خزش (**URL Frontier**)، بررسی صحت کدهای نوشته‌شده توسط دانشجویان از طریق خواندن مستقیم متغیرها یا قفل‌های حافظه، کاری به شدت پیچیده و مستعد خطاست.

بهترین روش برای ارزیابی این سیستم‌ها، استفاده از رویکرد «ارزیابی جعبه‌سیاه مبتنی بر رویداد» است. در این روش، خزشگر موظف است هر بار که 1 URL را برای واکنش به شبکه ارسال می‌کند، 1 رکورد در 1 فایل لاگ (**Log File**) ثبت کند. سپس اسکرپت ارزیاب (**Autograder**) مستقل از معماری داخلی خزشگر، این لاگ‌ها را تحلیل کرده و صحت زمان‌بندی را از نظر ریاضی اثبات می‌کند.

بخش 1) شبیه‌ساز لاگ‌های خروجی خزشگر

ابتدا قطعه کدی را می‌نویسیم که رفتار خروجی صف‌های **Front** و **Back** را شبیه‌سازی کرده و فایلی با فرمت استاندارد **[Timestamp, Host_IP, URL]** تولید می‌کند.

```
simulate_frontend.py

import time
import random

def simulate_frontend_output(log_filename: str):
    """
    شبیه‌سازی خروج آدرس‌ها از صف و ثبت آن‌ها در فایل لاگ.
    عمل می‌کند (Mock Data) این تابع به عنوان ابزار تولید داده آزمایشی.
    """
    mock_hosts = ["192.168.1.10", "10.0.0.55", "172.16.0.8"]

    with open(log_filename, 'w') as log_file:
        current_time = time.time()
        # تولید 20 رویداد خروج
        for i in range(20):
            # انتخاب تصادفی 1 میزبان
            host = random.choice(mock_hosts)
            url = f"http://{host}/page_{i}.html"
            # ثبت رویداد در فایل لاگ
            log_file.write(f"{current_time:.4f},{host},{url}\n")
            # شبیه‌سازی گذشت زمان متغیر بین درخواست‌ها
            current_time += random.uniform(0.5, 3.0)

# فراخوانی شبیه‌ساز برای تولید فایل
simulate_frontend_output("crawler_events.log")
```

بخش 2) اسکرپت ارزیاب خودکار (Autograder)

اسکرپت زیر به عنوان هسته سیستم ارزیابی پروژه‌های دانشجویی عمل می‌کند. این الگوریتم با 1 بار پیمایش خطی (با پیچیدگی زمانی $O(N)$) روی فایل لاگ، کوچکترین تخطی از سیاست **Politeness** را شناسایی کرده و جریمه متناسب را اعمال

```
autograder.py

from typing import Dict, List, Any

def politeness_autograder(log_filename: str, politeness_delay: float) -> Dict[str, Any]:
    """
    الگوریتم ناظر برای بررسی رعایت فاصله زمانی در فایل لاگ خزشگر

    پارامترها:
    - log_filename: مسیر فایل لاگ تولید شده توسط کد دانشجو
    - politeness_delay: حداقل تاخیر مجاز بین 2 درخواست به 1 سرور
    """
    # دیکشنری برای نگهداری آخرین زمان دیده شده برای هر میزبان
    # ساختار: { "Host_IP": Last_Timestamp }
    last_seen_map: Dict[str, float] = {}

    violations: List[dict] = []
    line_number = 0

    try:
        with open(log_filename, 'r') as file:
            for line in file:
                line_number += 1
                parts = line.strip().split(',')

                # نادیده گرفتن خطوط نامعتبر
                if len(parts) != 3:
                    continue

                timestamp_str, host_ip, url = parts
                current_ts = float(timestamp_str)

                # بررسی شرط در صورت وجود تاریخچه برای این میزبان
                if host_ip in last_seen_map:
                    delta = current_ts - last_seen_map[host_ip]

                    # اگر فاصله زمانی کمتر از حد مجاز باشد، تخلف ثبت می‌شود
                    if delta < politeness_delay:
                        violations.append({
                            "line": line_number,
                            "host": host_ip,
                            "url": url,
                            "actual_delay": round(delta, 4),
                            "expected_delay": politeness_delay
                        })

                # به‌روزرسانی زمان برای درخواست‌های بعدی به این میزبان
                last_seen_map[host_ip] = current_ts

    except FileNotFoundError:
        return {"status": "ERROR", "message": "Log file not found."}

    # تولید گزارش نهایی برای سیستم نمره‌دهی
    total_violations = len(violations)
    if total_violations > 0:
        # کسر 5 نمره به ازای هر تخلف
        penalty = total_violations * 5
        return {
            "status": "FAILED",
            "penalty_score": penalty,
            "violations_count": total_violations,
            "details": violations
        }
    else:
        return {
            "status": "PASSED",
            "penalty_score": 0,
            "message": "Politeness policy strictly followed."
        }

# اجرای اتوگریدر
report = politeness_autograder("crawler_events.log", politeness_delay=2.0)
print(report)
```

تحلیل منطق ارزیاب (State Management)

الگوریتم ارزیاب از 1 مکانیزم ردیابی وضعیت (State Tracking) بسیار بهینه استفاده می‌کند:

گام اجرایی	شرح مکانیزم بهینه‌سازی حافظه و پردازش
گام 1	اسکرپیت کل فایل لاگ را در RAM بارگذاری نمی‌کند (زیرا لاگ 1 خزشگر واقعی ممکن است گیگابایت‌ها حجم داشته باشد). فایل خط به خط خوانده شده و پس از استخراج current_ts

گام اجرایی	شرح مکانیزم بهینه‌سازی حافظه و پردازش
پردازش حافظه‌دوست (Streaming Read)	و <code>host_ip</code> ، آن خط بلافاصله از حافظه پاک می‌شود.
گام 2 مدیریت وضعیت (Hash Map Tracking)	الگوریتم 1 جدول درهم‌سازی (<code>last_seen_map</code>) در RAM نگه می‌دارد. اگر آدرس IP میزبان برای اولین بار دیده شود، صرفاً زمان آن در جدول ثبت می‌شود. اگر IP از قبل در جدول وجود داشته باشد، یعنی این 1 «درخواست تکراری به سرور قبلی» است و باید بررسی شود.
گام 3 کشف گلوگاه (Delta Calculation)	سیستم تفاوت زمان فعلی و زمان ثبت‌شده در جدول را محاسبه می‌کند (Δt). این محاسبه مشخص می‌کند که آیا خزشگر به تعهد زمانی خود در قبال سرور میزبان عمل کرده است یا خیر.

کشف باگ همزمانی (Race Condition Detection):

اگر کد دانشجویی دارای باگ همزمانی در **Min-Heap** باشد، ممکن است 2 نخ کارگر به طور همزمان 1 سرور را واکنشی کنند. در این حالت Δt عددی بسیار کوچک و نزدیک به صفر خواهد شد و اتوگریدر بلافاصله این تخطی را شکار کرده و آن را در لیست **violations** برای کسر نمره درج می‌کند. این روش، ارزیابی دقیق سیستم‌های موازی را بدون نیاز به دیباگ کدهای پیچیده امکان‌پذیر می‌سازد.

پاسخنامه مسئله 20.30: توزیع مقیاس‌پذیر در برابر حملات تزریق لینک (Link Injection & Spam Mass)

مقدمه: آناتومی مزارع پیوند (Link Farms) و شکست PageRank

حملات تزریق لینک بر پایه 1 ضعف بنیادین در الگوریتم **PageRank** استاندارد شکل می‌گیرند. الگوریتم **PageRank** بر این فرض استوار است که «هر پیوند، 1 رای اعتماد است». اسپمرها با ایجاد شبکه‌ای متراکم از هزاران سایت بی‌محتوا که همگی به یکدیگر پیوند داده‌اند (1 کلیک یا **Clique**)، 1 «چاه پیچ‌رنک» (**PageRank Sink**) می‌سازند.

در مدل استاندارد، به دلیل وجود بردار پرش یکنواخت (**Uniform Teleportation**) – که احتمال پرش تصادفی کاربر به هر صفحه‌ای در وب را برابر می‌داند – این شبکه‌های اسپم به مرور زمان امتیاز بالایی جمع‌آوری می‌کنند. اگر خزشگر (**Crawler**) برای تعیین اولویت صف‌های جلویی (**Front Queues**) صرفاً به این امتیاز خام اتکا کند، در تله افتاده و پهنای باند ارزشمند خود را صرف واکنشی میلیون‌ها صفحه زباله می‌کند.

راهکار TrustRank: اعتماد از هسته مرکزی نشأت می‌گیرد

برای خنثی کردن این حملات پیش از ورود به صف خزش، معماران موتور جستجو الگوریتم **TrustRank** (ارائه‌شده توسط محققان دانشگاه استنفورد) را جایگزین یا مکمل **PageRank** می‌کنند.

تفاوت ریاضی این 2 الگوریتم، منحصراً در بردار پرش تصادفی (**Teleportation Vector**) نهفته است. در الگوریتم **TrustRank**، توزیع پرش یکنواخت نیست، بلکه بردار ایستا (s) به گونه‌ای دستکاری می‌شود که کاربر تصادفی، فقط به مجموعه‌ای از صفحات به شدت معتبر (**Seed Pages**) پرش کند:

$$TR(u) = \alpha \sum_{v \in B_u} \frac{TR(v)}{L(v)} + (1 - \alpha)s(u)$$

در این فرمول، $s(u)$ تنها در صورتی مقدار غیرصفر دارد که صفحه u جزء صفحات دانه (مثل دامنه‌های **.edu**، **.gov** یا سایت‌های بررسی‌شده توسط انسان مانند ویکی‌پدیا) باشد.

خاصیت میرایی اعتماد (Trust Decay):

اعتماد از این هسته‌های مرکزی در گراف وب به سمت بیرون جریان می‌یابد. از آنجا که سایت‌های معتبر تقریباً هرگز به مزارع پیوند (Spam) لینک نمی‌دهند، گراف اسپمرها از هسته اعتماد ایزوله می‌ماند. هر چه سایت اسپم از هسته دورتر باشد، امتیاز TrustRank آن به صورت تصاعدی افت می‌کند (Decay) و در حد صفر باقی می‌ماند، حتی اگر میلیون‌ها لینک داخلی مصنوعی داشته باشد.

بازنویسی فرمول محاسبه اولویت (Priority Score)

برای محافظت از URL Frontier، باید منطق ریاضی استخراج آدرس‌ها را تغییر دهیم. به جای استفاده از PageRank خام، از مفهوم حجم اسپم (Spam Mass) استفاده می‌کنیم.

1. محاسبه حجم اسپم

ابتدا محاسبه می‌کنیم که چه کسری از امتیاز 1 صفحه، ناشی از ساختارهای مشکوک است:

$$SpamMass(u) = \frac{PR(u) - TR(u)}{PR(u)}$$

• اگر SpamMass نزدیک به 1 باشد: صفحه قطعاً اسپم است (رتبه بالا اما اعتماد صفر).

• اگر SpamMass نزدیک به 0 یا منفی باشد: صفحه کاملاً معتبر و ارگانیک است.

2. فرمول جدید اولویت در URL Frontier

اکنون تابع اولویت‌بندی $P(u)$ را (که تعیین می‌کند آدرس u در کدام صف **Front** قرار گیرد) با اعمال 1 «جریمه سنگین» (Penalty) برای صفحات مشکوک بازنویسی می‌کنیم:

$$P(u) = TR(u) \times Freshness(u) \times e^{-\lambda \cdot SpamMass(u)}$$

که در آن:

$TR(u)$: امتیاز خالص اعتماد صفحه.

$Freshness(u)$: میزان تغییرات و تازگی محتوای صفحه (Crawl History).

$e^{-\lambda \cdot SpamMass(u)}$: 1 تابع جریمه‌کننده نمایی. متغیر λ 1 ضریب سخت‌گیری است. اگر حجم اسپم بالا باشد، این بخش

به سرعت به سمت صفر میل کرده و کل اولویت صفحه را نابود می‌کند.

مکانیزم خنثی‌سازی در لایه معماری

با اعمال این فرمول در سیستم، حمله اسپرها در همان مراحل اولیه به شکل زیر در هم می‌شکند:

گام اجرایی	شرح مکانیزم دفاعی
گام 1 محاسبه اولیه (Extraction)	کشف لینک‌های مصنوعی: خزشگر لینک‌های تولید شده توسط مزرعه پیوند را می‌بیند و آدرس‌های جدید را به ماژول اولویت‌سنج (Priority Assgner) ارسال می‌کند.
گام 2 تحلیل تقاطع (PR vs TR)	برخورد با سد اعتماد: ماژول محاسبه می‌کند که این صفحات دارای PageRank بالایی (به دلیل کثرت لینک‌های داخلی) هستند، اما TrustRank آن‌ها به دلیل عدم دریافت لینک از دامنه‌های معتبر (Seeds) عملاً صفر است.
گام 3 فعال‌سازی جریمه (Penalty)	سقوط آزاد اولویت: فرمول جدید اعمال می‌شود. به دلیل بالای بودن متغیر $SpamMass(u)$ ، تابع نمایی مقدار بسیار کوچکی تولید کرده و اولویت نهایی $P(u)$ را به شدت کاهش می‌دهد.
گام 4 تنزل صف (Queue Drop)	دفع در لایه صف: به جای قرار گرفتن در Front Queue های رده بالا (مثلاً صف 1 یا 2)، این آدرس‌ها به پایین‌ترین و کندترین صف‌های خزش (مثلاً صف 64) پرتاب می‌شوند. خزشگر هرگز منابع پردازشی و پهنای باند خود را برای رسیدن به انتهای این صف‌ها هدر نخواهد داد.

مدل‌سازی شماتیک پیشرفته: فیلترینگ Spam Mass در معماری صف

Source	Metrics (PR vs TR)	Spam Mass Penalty	Front Queue Assigned
Valid Page	PR(u) = 0.85 TR(u) = 0.80	~ 0.05 (Low)	Queue 1 (High Priority)
Link Farm Page	PR(u) = 0.90 (Fake) TR(u) = 0.01	~ 0.98 (Critical)	Queue 64 (Dumped)

نتیجه‌گیری ساختاری:

ادغام مکانیزم TrustRank با لایه URL Frontier، گراف وب را از 1 شبکه صرفاً کمی به 1 شبکه تحلیلی کیفی تبدیل می‌کند. تابع نمایی جریمه، مانند 1 فیلتر ضداسپم بی‌رحم عمل کرده و صفحاتی که سعی در فریب ریاضی الگوریتم PageRank دارند را با «سقوط آزاد اولویت» در صف‌های مرده محبوس می‌سازد. بدین ترتیب منابع پردازشی موتور جستجو منحصراً معطوف به هسته معتبر وب باقی می‌ماند.

پاسخنامه مسئله 20.31: معماری تکثیر متقاطع با همگام‌سازی ناهمگام (Async Cross-Replication)

مقدمه: کالبدشکافی بحران در معماری ناهمگام (Asynchronous)

طراحی چنین سناریوهای بحرانی و پیچیده‌ای، 1 مورد مطالعاتی عالی برای سنجش تسلط بر مهندسی قابلیت اطمینان (Reliability Engineering) در سیستم‌های توزیع‌شده است. در این مسئله، ما با 1 تله مفهومی روبرو هستیم:

تفاوت ظریف و حیاتی:

تفاوت بنیادین بین «از دست رفتن داده‌های در حال نوشتن (In-flight Writes)» و «از دست رفتن داده‌های در حال ادغام

در تکثیر ناهمگام (**Asynchronous Replication**)، گره رهبر (**Leader**) به محض دریافت 1 سند جدید، آن را در حافظه موقت (**MemTable**) و گزارش پیش‌نوشت خود (**WAL**) می‌نویسد، به کلاینت تاییدیه (**ACK**) می‌دهد و سپس در پس‌زمینه، این تغییرات را به گره‌های پیرو (**Followers**) ارسال می‌کند. این رویکرد توان عملیاتی (**Throughput**) را به شدت بالا می‌برد، اما در زمان بروز خرابی، پیامدهای خاص خود را دارد.

مکانیزم ادغام نمایه (Index Merge) و ویژگی تغییرناپذیری

عملیات ادغام نمایه (یا **Compaction**) 1 فرآیند سنگین پس‌زمینه است که برای جلوگیری از تکه‌تکه شدن داده‌ها، قطعات کوچک نمایه (**Segments**) را خوانده و آن‌ها را به 1 قطعه یکپارچه و بزرگ‌تر تبدیل می‌کند.

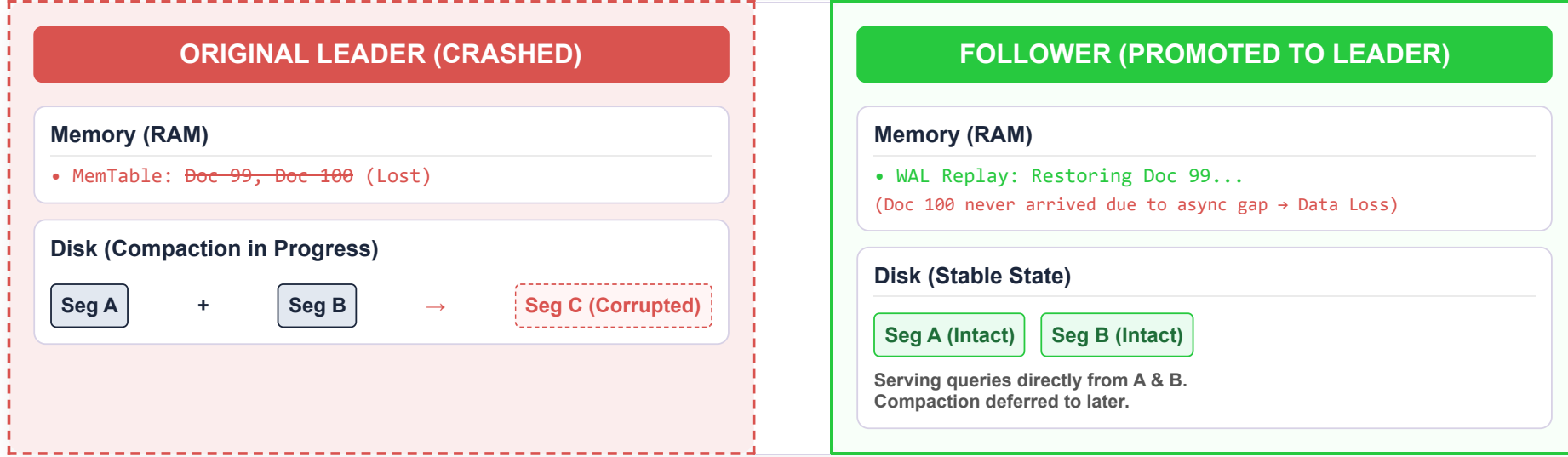
اگر گره رهبر دقیقاً در وسط ادغام قطعات **A** و **B** به قطعه جدید **C** دچار فروپاشی سخت‌افزاری (**Hard Crash**) شود، مسیر بازیابی به ساختار بنیادین موتورهای جستجو مدرن (مانند **Elasticsearch** یا **Apache Lucene**) بستگی دارد. این موتورها بر پایه **تغییرناپذیری (Immutability)** بنا شده‌اند. این یعنی فرآیند ادغام 1 عملیات «مخرب» نیست که داده‌های قبلی را پاک کند، بلکه 1 عملیات «تولیدی» است.

مسیر بازیابی سیستم (Failover Sequence)

بازیابی سیستم بدون هیچ‌گونه نیاز به مداخله دستی، طی مراحل زیر در کلاستر انجام می‌شود:

گام اجرایی	شرح مکانیزم و وضعیت سیستم
گام 1 ارتقای پیرو (Leader Election)	جابجایی قدرت در کلاستر: هماهنگ‌کننده کلاستر (مانند ZooKeeper) متوجه قطع شدن ضربان قلب (Heartbeat) گره رهبر می‌شود. بلافاصله یکی از گره‌های پیرو که به‌روزترین وضعیت همگام‌سازی را دارد، به عنوان رهبر جدید ارتقا (Promote) می‌یابد تا ترافیک خواندن و نوشتن را بر عهده بگیرد.
گام 2 ایزوله کردن فایل‌ها (Idempotency)	تضمین تغییرناپذیری قطعات نمایه: از آنجا که قطعات A و B تغییرناپذیرند، در حین فرآیند ادغام روی گره رهبر سوخته، کاملاً دست‌نخورده باقی مانده بودند. بر روی گره رهبر جدید، قطعات A و B به صورت سالم و مستقل وجود دارند. قطعه نیمه‌کاره C که روی سرور سوخته در حال ساخته شدن بود، صرفاً 1 زباله (Orphaned File) است که با سوختن سرور از بین می‌رود و هیچ آسیبی به یکپارچگی داده‌های اصلی وارد نمی‌کند.
گام 3 بازسازی حافظه (WAL Replay)	بازیابی اسناد در حال پرواز: رهبر جدید برای اسنادی که به تازگی وارد شده بودند و هنوز در حافظه موقت قرار داشتند، به سراغ گزارش پیش‌نوشت (WAL) خود می‌رود و تمام تراکنش‌های ثبت‌شده را در حافظه بازسازی (Replay) می‌کند. بهای معماری ناهمگام: تنها اسنادی که در چند میلی‌ثانیه آخر قبل از سوختن رهبر وارد سیستم شده و به دلیل قطعی شبکه هنوز به WAL گره پیرو نرسیده بودند، برای همیشه از دست می‌روند (Data Loss). کاربر تصور می‌کند این داده‌ها ثبت شده‌اند (چون ACK دریافت کرده بود)، اما در واقعیت از بین رفته‌اند. این هزینه قطعی معماری‌های ناهمگام برای رسیدن به تاخیر پایین است.
گام 4 از سرگیری ادغام (Resume Compaction)	ادغام 1 بهینه‌سازی است، نه 1 الزام: رهبر جدید اکنون کاملاً عملیاتی است. از آنجا که فرآیند ادغام صرفاً 1 بهینه‌سازی برای افزایش سرعت جستجو است، لزومی ندارد بلافاصله پس از بازیابی از سر گرفته شود. سیستم به پاسخگویی کوئری‌ها با استفاده از همان قطعات مجزای A و B ادامه می‌دهد و در زمان خلوتی شبکه، مجدداً آن‌ها را برای ادغام کاندید می‌کند.

مدل‌سازی شماتیک پیشرفته: فروپاشی رهبر و ارتقای پیرو



جمع‌بندی:

در این معماری، تفاوت حیاتی بین داده‌های در حال پرواز (In-flight Writes) و داده‌های در حال ادغام (Merging Segments) مشخص می‌شود.

فرآیند ادغام کاملاً در برابر خرابی ایمن (Crash-safe) است و سوختن سرور در میان این عملیات هیچ دیتای تثبیت‌شده‌ای را از بین نمی‌برد. تنها نقطه آسیب‌پذیری در این طراحی، شکاف زمانی (Time Gap) در انتقال ناهمگام اطلاعات به گره‌های پیرو است که می‌تواند منجر به از دست رفتن کسری از ثانیه از آخرین تراکنش‌های کلاینت شود.

پاسخنامه مسئله 20.32: طراحی جامع خزشگر متمرکز با ارزیابی خودکار (Focused Crawler Pipeline)

مقدمه: معماری خزشگر متمرکز (Focused Crawler)

طراحی 1 خزشگر متمرکز برای استخراج مقالات پژوهشی، نیازمند عبور از پارادایم «خزش کورکورانه» (Blind Crawling) و حرکت به سمت «ناوبری هدایت‌شده» است. در جلسه دفاعیه، ارزش اصلی این معماری نه در توانایی دانلود فایل، بلکه در مکانیزم‌های دفاعی آن برای حفظ پهنای باند و دقت بالای طبقه‌بند نهفته است.

برای تحقق این هدف، سیستم را در قالب 1 خط لوله پردازشی (Processing Pipeline) چندمرحله‌ای طراحی می‌کنیم که در آن هر ماژول به عنوان 1 فیلتر عمل کرده و از هدررفت منابع در ماژول بعدی جلوگیری می‌کند.

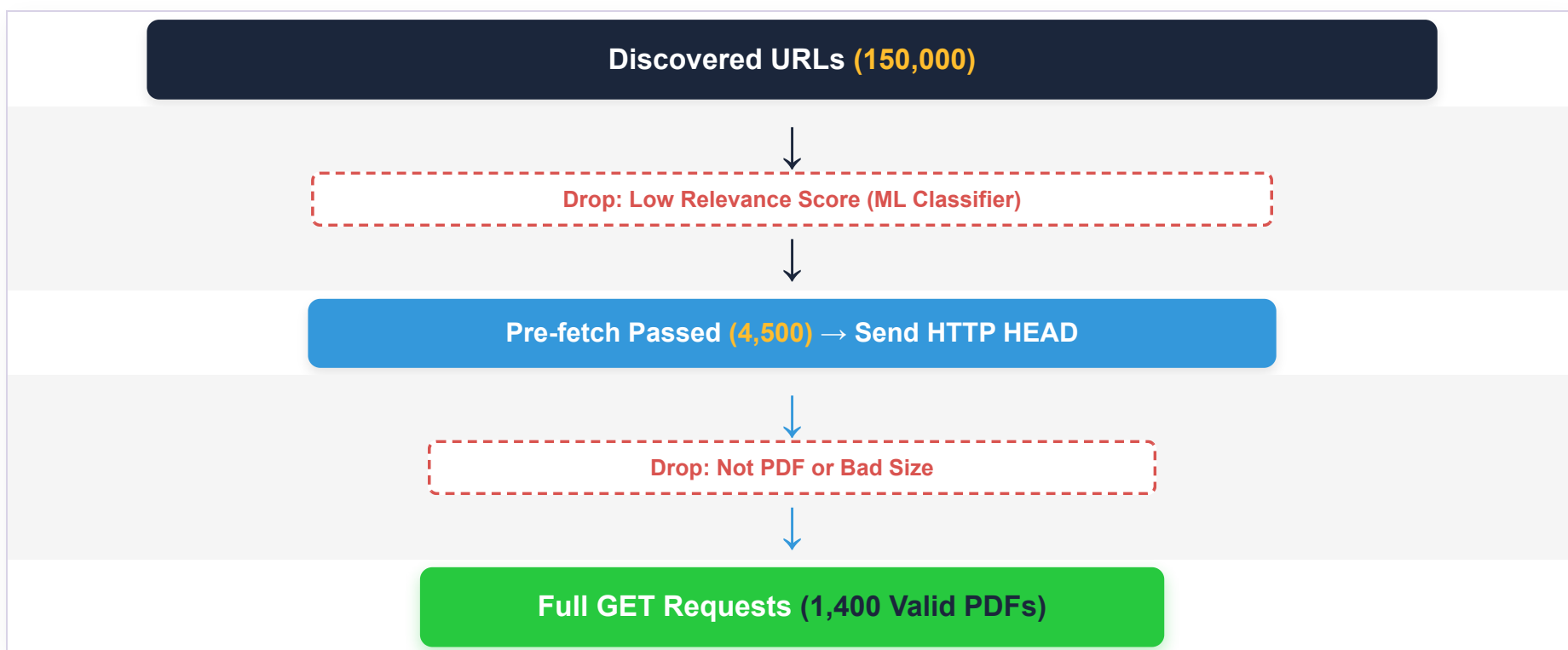
جریان داده و معماری خط لوله (Data Flow Architecture)

مسیر حرکت 1 URL کشف‌شده تا تبدیل شدن به 1 مقاله ذخیره‌شده و ثبت در گزارش، طی گام‌های زیر انجام می‌پذیرد:

گام اجرایی	شرح مکانیزم و فیلترینگ ماژول
گام 1 طبقه‌بند پیش‌واکشی (Pre-fetch Classifier)	فیلترینگ بدون نیاز به شبکه: این ماژول پیش از ارسال هرگونه درخواست HTTP وارد عمل می‌شود. الگوریتم (مانند Naive Bayes یا SVM) روی 2 ویژگی کلیدی کار می‌کند: ساختار URL (وجود کلماتی نظیر pdf, article, proceedings یا شناسه doi) و متن پیوند (Anchor Text). اگر طبقه‌بند تشخیص دهد که پیوند «احتمالاً» به 1 مقاله دانشگاهی اشاره دارد، به آن 1 امتیاز ارتباط (Relevance Score) می‌دهد. تنها آدرس‌هایی که امتیازشان از آستانه θ بیشتر باشد، مجوز ورود به مرحله شبکه را دریافت می‌کنند.

گام اجرایی	شرح مکانیزم و فیلترینگ ماژول
گام 2 استخراج متادیتا (HTTP HEAD Request)	دیواره آتش پهنای باند: این مرحله شاه‌کلید بهینه‌سازی سیستم است. به جای ارسال درخواست GET (که کل فایل را دانلود می‌کند)، خزشگر 1 درخواست سبک HEAD به سرور می‌فرستد. سرور تنها سرآیندها (Headers) را برمی‌گرداند. ماژول بررسی می‌کند: آیا Content-Type دقیقاً برابر با application/pdf است؟ آیا Content-Length در محدوده منطقی 1 مقاله (مثلاً بین 500 کیلوبایت تا 50 مگابایت) قرار دارد؟ اگر پاسخ منفی باشد، ارتباط بلافاصله قطع شده و فایل دانلود نمی‌شود.
گام 3 واکشی کامل (Full GET Request)	استخراج نهایی: تنها اگر URL از سد ماژول HEAD با موفقیت عبور کند، خزشگر دستور GET را صادر کرده و بایت‌های فایل PDF را دریافت می‌کند. در اینجا سیستم حجم دقیق داده‌های دریافت شده را برای ماژول گزارش‌گیر ثبت می‌کند.
گام 4 تولید گزارش (JSON Generator)	تله‌متری و ارزیابی: 1 ماژول ناظر (Monitor) در پس‌زمینه، تمام رویدادهای موفق و ناموفق را می‌شمارد. در پایان فرآیند (یا در فواصل زمانی مشخص)، این ماژول وضعیت کلی سیستم، نرخ موفقیت و میزان مصرف شبکه را در قالب 1 فایل JSON ساختاریافته خروجی می‌دهد تا توسط سیستم‌های ارزیاب (Autograder) یا مانیتورینگ تحلیل شود.

مدل‌سازی شماتیک پیشرفته: قیف فیلترینگ خزشگر متمرکز



تحلیل ریاضی و فرمول‌بندی معیارها برای جلسه دفاع

در جلسه دفاعیه پروژه‌های سیستم‌های بازیابی، صرف‌کار کردن کد کافی نیست؛ بلکه باید کارایی سیستم از نظر ریاضی اثبات شود. ماژول تولیدکننده گزارش، 2 فرمول حیاتی زیر را محاسبه می‌کند:

1. نرخ موفقیت (Hit Rate)

نشان‌دهنده دقت (Precision) خزشگر در یافتن مقالات هدف است.

$$Hit Rate = \frac{\text{Total PDFs Downloaded}}{\text{Total HEAD Requests Sent}}$$

2. بهره‌وری پهنای باند (Bandwidth Efficiency)

نشان‌دهنده موفقیت ماژول **HEAD** در جلوگیری از دانلود فایل‌های هرز (تصاویر بزرگ، ویدیوها یا صفحات **HTML** حجیم) است.

$$Bandwidth\ Efficiency = \frac{\text{Bytes of Saved PDFs}}{\text{Total Network Bytes Transferred}}$$

هرچه این عدد به 1 (یا 100%) نزدیک‌تر باشد، نشان می‌دهد که سیستم پهنای باند را صرفاً برای مقالات واقعی خرج کرده است.

ساختار خروجی ماژول گزارش‌ساز (JSON Schema)

خروجی سیستم باید به گونه‌ای طراحی شود که به راحتی توسط اسکریپت‌های ارزیاب (**Autograders**) قابل پارس و نمره‌دهی باشد. ساختار استاندارد فایل `report.json` به شکل زیر خواهد بود:

```
{
  "crawler_session": {
    "start_time": "2026-07-16T10:00:00Z",
    "end_time": "2026-07-16T10:45:00Z",
    "total_duration_seconds": 2700
  },
  "classification_metrics": {
    "total_urls_discovered": 150000,
    "urls_passed_classifier": 4500,
    "urls_rejected_by_classifier": 145500
  },
  "network_metrics": {
    "head_requests_sent": 4500,
    "head_requests_failed_mime_type": 3100,
    "full_get_requests_sent": 1400,
    "total_bandwidth_used_mb": 1850.5
  },
  "performance_results": {
    "total_pdfs_downloaded": 1390,
    "hit_rate_percentage": 30.88,
    "bandwidth_efficiency_percentage": 98.2
  }
}
```

نکته کلیدی:

در این معماری، از میان **150,000** آدرس کشف‌شده، تنها برای **4,500** مورد درخواست سبک **HEAD** ارسال شده و در نهایت فقط **1,400** فایل دانلود شده است. این یعنی سیستم بدون بارگیری ده‌ها گیگابایت داده اضافی در بستر شبکه، مقالات را با دقتی جراحی‌گونه استخراج کرده است. این الگو، تبلور مهندسی مقیاس‌پذیر در سیستم‌های پیچیده بازیابی اطلاعات است.